

HE-based On-the-Fly MPC, Revisited: Universal Composability, Approximate and Imperfect Computation, Circuit Privacy

Ganyuan Cao¹[0009–0003–7299–3347] ^{*}, Sylvain Chatel²[0000–0002–1275–8367] and
Christian Knabenhans³[0000–0003–4645–7295]

¹ Télécom Paris, Institut Polytechnique de Paris, Palaiseau, France
`ganyuan.cao@telecom-paris.fr`

² CISP, Helmholtz Center for Information Security, Saarbrücken, Germany
`sylvain.chatel@cispa.de`

³ EPFL, Lausanne, Switzerland
`christian.knabenhans@epfl.ch`

Abstract. On-the-fly multi-party computation (MPC), introduced by López-Alt, Tromer, and Vaikuntanathan (STOC 2012), enables clients to dynamically join a computation without remaining continuously online. Yet, the original proposal suffers from substantial efficiency and expressivity limitations hindering practical deployments. Even though various techniques have been proposed to mitigate these shortcomings, seeing on-the-fly MPC as a combination of independent building blocks jeopardizes the security of the original model.

Thus, we revisit on-the-fly MPC in light of recent advances and extend its formal framework to incorporate efficiency and expressivity improvements. Our approach is built around *multi-group homomorphic encryption* (MGHE), which generalizes threshold and multi-key HE and serves as the core primitive for on-the-fly MPC. Our contributions are fourfold: i) We propose new security notions for MGHE (e.g., IND-CPA with partial decryption, circuit privacy) and justify their suitability to the on-the-fly MPC. ii) We present the first ideal functionality for MGHE in the Universal Composability (UC) framework and characterize the conditions under which it can be realized, via reductions to our proposed security notions. iii) We present a generic protocol that securely realizes our on-the-fly MPC functionality against a semi-malicious adversary from our MGHE functionality. iv) Finally, we provide two generic compilers that lift these protocols to withstand a fully malicious adversary by leveraging zero-knowledge arguments.

Our analysis in the UC framework enables modular protocol analysis, where more efficient schemes can be seamlessly substituted as long as they meet the required security defined by the functionalities, retaining the security guarantees offered by the original construction.

Keywords: On-the-Fly MPC · Multi-Group Homomorphic Encryption · zkSNARK · Universal Composability (UC).

^{*} Part of this work was done while the author was affiliated with EPFL.

Table of Contents

1	Introduction	3
1.1	Our results	4
2	Technical Overview	5
2.1	UC-Secure Multi-Group HE	6
	UC Functionality for MGHE	6
	New Security Notions for MGHE	6
2.2	UC-Secure On-the-Fly MPC	7
3	UC Security of Multi-Group HE	9
3.1	Functionality $\mathcal{F}_{\text{MGHE}}$	10
3.2	What Are the “Right” Security Notions for MGHE?	12
3.3	Realizing $\mathcal{F}_{\text{MGHE}}$	18
4	UC-Secure On-the-Fly MPC from MGHE and zkSNARK/NIZK	19
4.1	Functionality $\mathcal{F}_{\text{OTF-MPC}}$	19
4.2	Realizing $\mathcal{F}_{\text{OTF-MPC}}$ against Semi-Malicious Adversary	22
4.3	Compiling with zkSNARK for Malicious Security	22
4.4	Compiling with NIZK/zkSNARG for Malicious Security	26
	References	29
A	Related Work	34
B	Universal Composability (UC) with Global Subroutines	35
B.1	System of ITMs	35
B.2	Global Subroutines	35
B.3	UC Emulation and Realization	36
B.4	UC Composition Theorem	37
C	UC-secure Zero-Knowledge Non-Interactive Arguments	38
C.1	Global Random Oracle $\mathcal{G}_{\text{RO}}$	38
C.2	Ideal Functionality $\mathcal{F}_{\text{zkSNARK}}$ and $\mathcal{F}_{\text{NIZK}}$	40
D	Global Key Registry $\mathcal{G}_{\text{KRK}}$	41
E	Authenticated Channel $\mathcal{F}_{\text{AUTH}}$	43
F	Full Proofs	43
F.1	Full proof of Theorem 2	43
F.2	Full proof of Theorem 3	48
F.3	Full proof of Corollary 1	51
F.4	Full proof of Theorem 4	53
F.5	Full proof of Theorem 5	55
F.6	Full proof of Theorem 6	59
G	Protocol with Single-Round Decryption	60

1 Introduction

Secure multi-party computation (MPC) enables a set of mutually distrusting parties to jointly compute a function over their private inputs while keeping these inputs secret. *On-the-fly* multi-party computation [49] is a variant of MPC where parties can dynamically join during the execution of the protocol for a specific computation task without the need to be constantly online. Thanks to this flexibility, on-the-fly MPC serves as a generalized framework for numerous real-world applications [41,59,60].

The canonical construction of on-the-fly MPC is given by the LTV compiler [49]. This compiler leverages a *multi-key homomorphic encryption* (MKHE) scheme, which enables computations over data encrypted under different keys, as well as a non-interactive zero-knowledge argument (NIZK) and a succinct non-interactive argument of knowledge (SNARK). The on-the-fly multi-party computation is performed by an untrusted server, which stores global state and performs encrypted computations on behalf of clients.

To the best of our knowledge, the LTV compiler has not been revisited since its inception more than a decade ago. In the meantime, there has been tremendous progress on multi-party homomorphic encryption (a primitive closely related to, but a strict superset of LTV’s MKHE). Separately, the security of NIZKs and zkSNARKs is now much better understood, particularly in terms of composability and nuanced simulation-security notions. This leads to the paradoxical situation in which many applications of on-the-fly MPC are now concretely efficient and on the verge of deployment, yet the foundational compiler and analysis for on-the-fly MPC is inadequate for virtually all of these applications, and fails to give any provable security guarantees.

In this work, we revisit the LTV compiler, adding support for optimized schemes resulting from a decade of improvements in (multi-party) HE and NIZKs/zkSNARKs. Devising a more modern analog of the LTV compiler poses significant conceptual and technical challenges. This is due mainly to three factors: First, while the improvements of each building blocks have been supported by new ad-hoc security notions and analyses, it is often unclear how security notions for the same primitive but different threat models compare to each other (e.g., honest-but-curious adversary vs (semi-)malicious adversary; different oracles provided to the adversary). Second, from this plethora of security notions, it is unclear which are the “right” security notions for the purpose of building on-the-fly MPC. Finally, when applying different (and often orthogonal) optimizations together, it is unclear how security degrades in this composed setting.

We discuss three main challenges on the way to modernizing the LTV compiler, and state our main results in Section 1.1. We then give an overview of our techniques in Section 2.

Challenge 1: Approximate and imperfect computations. Many modern on-the-fly MPC applications leverage approximate HE for efficiency (e.g., for machine learning tasks, which greatly benefit from approximate arithmetic). Some state-of-the-art implementations of HE schemes are also non-perfect and allow for

a small probability of decryption failure for efficiency. Non-perfect or non-exact schemes are susceptible to key-recovery attacks [45,24]. While many security notions for non-exact or non-perfect schemes have been proposed [45,24,7,8], the relationship between these security notions are still underexplored, and it is unclear which of these security notions are sufficient in the setting of on-the-fly MPC.

Challenge 2: Server inputs and circuit privacy. Modern HE schemes enable efficient plaintext-ciphertext operations. This capability is often leveraged in practical HE-based MPC protocols to delegate additional work to a *helper* server (e.g., evaluating a pre-trained machine learning model whose weights are only known to the server). While security analyses of HE-based MPC protocols typically focus on guaranteeing the privacy in the presence of a corrupted server, in real-world applications, ensuring the privacy of the server’s circuit [40,33,13] or other private inputs [43] is equally important.

Challenge 3: Beyond multi-key HE. Since the introduction of multi-key HE in [49], the more flexible and efficient primitive of multi-*group* HE (MGHE) [2,56,21,44,23] has emerged. MGHE combines MKHE with *threshold* HE (ThHE) [3,10,12] by organizing participants into groups; groups form logical parties in a multi-key scheme, and evaluations and encryptions are performed at the group level, thus reducing the number of involved parties. MGHE (which captures MKHE and ThHE as special cases) allows for more efficient protocols, and also aligns more with real-world applications (where computation often occurs within organizational or federated group boundaries [58,62]). However, advanced security notions (such as partial decryption simulation security or robustness) have not been satisfactorily studied for MGHE in its full generality.

Challenge 4: Composability. Challenges 1–3, individually, warrant a careful security analysis. However, we strive to combine all these improvements into a single compiler. This necessitates understanding how security notions for approximate/imperfect HE and circuit privacy interact with multi-group security considerations. Also, the LTV compiler only considers single-instance security, and does not cover cases where multiple instances of on-the-fly MPC are run concurrently, or if any other cryptographic protocol is run alongside. This severely limits the applicability of the LTV compiler in practice, since most applications of of-MPC are intended to be deployed as part of larger systems.

1.1 Our results

In order to address these challenges, we provide the first ideal functionality for multi-group HE (Section 2.1) and on-the-fly MPC (Section 2.2) in the Universal Composability (UC) framework. Both ideal functionalities model approximate and imperfect computations, as well as circuit privacy.

UC-Secure MGHE. We show the following sufficient conditions on an MGHE scheme to UC-realize our MGHE functionality (Fig. 4) against a semi-malicious adversary that can inject malformed partial decryptions.

- Client privacy: $\text{IND-CPA}^{\text{PD}}$ security (Definition 2) a notion which we introduce, and which generalizes the notions of $\text{IND-CPA}^{\text{D}}$, $\text{IND-CPA}^{\text{DSH}}$ [7], $s\text{IND-CPA}^{\text{D}}$ [8], and $\text{IND-CPA}^{\text{DR}}$ [48];
- Server privacy: a notion of IND-CIRC security which we introduce (Definition 4), which generalizes a notion from [42] to the multi-group setting.
- Threshold security: i) simulatability of partial decryption (SIM-PDEC) from [10] and ii) a notion of robustness (Rob, Definition 6), extending [10,12] to address security under HE evaluation in a generalized t -out-of- n setting.

Theorem 1 (informal). *Let $\text{MGHE} = (\text{PGen}, \text{KGen}, \text{KJoin}, \text{Enc}, \text{Eval}, \text{PDec}, \text{Combine})$ be a multi-group homomorphic encryption scheme. If MGHE satisfies $\text{IND-CPA}^{\text{PD}}$, IND-CIRC , SIM-PDEC and Rob, then the protocol Π_{MGHE} (Fig. 8) UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ (Fig. 4) in the presence of a global subroutine $\mathcal{G}_{\text{KRK}}$ against a semi-malicious adversary that injects malformed partial decryptions.*

We discuss our definitions and proof techniques in Section 2.1. Theorems 2 and 3 give more detailed statements of Theorem 1.

UC-Secure On-the-Fly MPC. After this result, which we deem of independent interest, we turn our attention to on-the-fly MPC. First, we give an ideal functionality for on-the-fly MPC in the UC framework (Fig. 10, which captures approximate and imperfect computations, as well as circuit privacy).

Equipped with a UC-secure approximate/imperfect MGHE protocol with server and client privacy, we show that this immediately gives rise to a UC-secure on-the-fly MPC protocol against a semi-malicious adversary in Section 4.2 and Theorem 4.

Next, we define three compilers that compile MGHE into a UC-secure on-the-fly MPC protocol against a fully malicious adversary. The first compiler leverages an *any-simulation-extractable* NIZK and SNARK. (Theorem 5) in spirit to the original LTV compiler.

The second compiler uses the Naor–Yung paradigm [55], and leverages an *any-simulation-sound* SNARG (Theorem 6). This compiler is less efficient than our first compiler (since it requires twice as many encryptions, encrypted evaluations, and decryptions), but is closely related to constructions of IND-CCA1 and Target-Malleable HE schemes [11,20]. Interestingly, this compiler does not require an argument of knowledge but merely a simulation-sound argument, which makes it more widely applicable.

Our third compiler (Appendix G, Fig. 18) is an optimization of our second compiler, where we rely on the Naor–Yung paradigm for encrypted and evaluated ciphertexts, but leverage an argument of knowledge for decryption, allowing for more efficient multi-party decryption protocol.

2 Technical Overview

We now provide an overview of the multi-group HE and on-the-fly MPC functionalities in the *Universal Composability* (UC) framework [16,5] and their respective key security properties.

2.1 UC-Secure Multi-Group HE

UC Functionality for MGHE We present an ideal functionality $\mathcal{F}_{\text{MGHE}}$ for multi-group HE in the *Universal Composability* (UC) framework [16] (with *global subroutines* [5]), capturing security for both clients and the server. To the best of our knowledge, this is the *first* UC ideal functionality for MGHE in the literature; and we present this formalization as a contribution of independent interest. Our functionality applies to both exact and approximate HE schemes; in the latter case, we explicitly model approximate computations by tracking noise growth throughout the execution.

New Security Notions for MGHE We introduce new security notions specifically for the multi-group structure that explicitly address a *semi-malicious* adversary [3] who is allowed to choose randomness during the execution of the protocol, and that cover both approximate and exact HE.

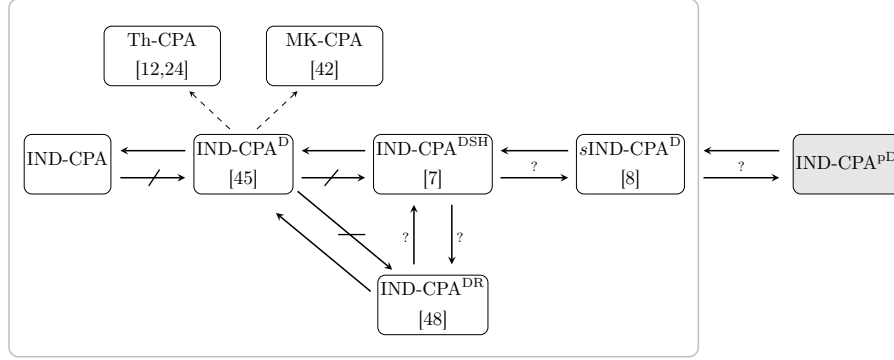


Fig. 1: Security notions for (multi-party) homomorphic encryption. Arrows denote relationships: $A \dashrightarrow B$ means that Notion B is a generalization of Notion A in multi-key or threshold setting; $A \rightarrow B$ means that A implies B; $A \nrightarrow B$ means that A does not imply B; $A \stackrel{?}{\rightarrow} B$ indicates that the implication from A to B is currently unknown. Our notion $\text{IND-CPA}^{\text{pD}}$ generalizes and implies existing notions.

Client Privacy. We introduce a new security notion $\text{IND-CPA}^{\text{pD}}$ (Fig. 5). In [7,48], the adversary is semi-honest and only learns the encryption or evaluation randomness honestly sampled by the $\mathcal{O}_{\text{Enc}}$ and $\mathcal{O}_{\text{Eval}}$ oracles after each query. In contrast, in [8], the adversary may only choose the encryption randomness. In our model, we explicitly allow the adversary to set the randomness for *any* part of the protocol execution. This captures scenarios where both clients and the server are corrupted. Furthermore, since multi-group HE naturally generalizes both threshold and multi-key HE, our notion extends the security of threshold HE

[12,24] and multi-key HE [42]. In Fig. 1, we illustrate the relationship between our proposed IND-CPA^{PD} notion and existing CPA(-D) notions for HE and its multi-party variants.

Server Privacy. We revisit circuit privacy, examining both its simulation-based [33,40,13] and game-based [42] formalizations, and analyze the security of exact and approximate HE with them respectively. For the simulation-based security, we adopt the relaxed notion of [13], in which the topology of the circuit is revealed to the adversary, because the server (publicly) receives inputs from multiple sources, and the structure of the circuit determines how these inputs are combined. Consequently, the very act of collecting inputs from different parties inherently exposes information about the circuit’s topology. For the game-based setting, we introduce a new notion IND-CIRC (Definition 4), which extends [42] to the multi-group setting by accounting for client corruptions and a semi-malicious adversary.

Threshold Security. We generalize security guarantees beyond the restrictive (n, n) case, which is typically considered in existing works [49,54,44,38], to more flexible (t, n) -threshold structure which aligns more with our goal that not all parties have to be online at the same time. We consider two notions for threshold security.

- *simulatability of partial decryption* (SIM-PDEC) [10], which formalizes the security that partial decryption does not reveal the key shares of parties participating in the decryption. We justify its necessity along with our IND-CPA^{PD} security in MGHE.
- *robustness* (Rob) [9,12], ensuring that correct message is reconstructed as long as sufficient number of valid partial decryptions are obtained, regardless the *malformed* partial decryption injected by the adversary. We define our own Rob notion (Fig. 7) in multi-group setting and considering a semi-malicious adversary as earlier, as well as capturing the correctness in terms of exact and approximate HE.

Having these security notions, we show the conditions for realizing our functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global subroutine for parameter and key setup, as shown in Theorem 1.

2.2 UC-Secure On-the-Fly MPC

Security against Semi-Malicious Adversary. Having $\mathcal{F}_{\text{MGHE}}$ as a building block, we revisit the established claim that multi-party HE alone suffices to construct on-the-fly MPC protocols secure against a semi-malicious (SM) adversary by presenting a protocol $\Pi_{\text{OF-MPC}}^{\text{SM}}$ in the $\mathcal{F}_{\text{MGHE}}$ -hybrid model, which makes calls to our UC functionality for MGHE. This further explains how our notions support and justify the security of on-the-fly MPC.

Malicious Security with (Straightline) Simulation-Extractability. We show the composition of (multi-group) HE with a zkSNARK (or a NIZK) to

achieve security against a *malicious* adversary. We first present the compiler $\Pi_{\text{OtF-MPC}}^{\text{SE-1P}}$ that utilizes a (straightline) *simulation-extractable* zkSNARK to guarantee soundness and non-malleability. We adopt the ideal functionality $\mathcal{F}_{\text{zkSNARK}}$ from [25] but additionally require it to have *any* simulation security [29] to enable our simulation. The protocol adopts a single-pass structure (1P) and proceeds in three phases, specifically:

- *Input phase*: each client encrypts its input under MGHE and provides a zkSNARK proof of well-formedness for the ciphertext.
- *Computation phase*: the server homomorphically evaluates a circuit on the encrypted inputs and produces a zkSNARK proof certifying the correctness of the evaluation.
- *Output phase*: the clients collaboratively perform partial decryptions, which are then aggregated to reconstruct the final plaintext; each partial decryption is accompanied by a zkSNARK proof of validity.

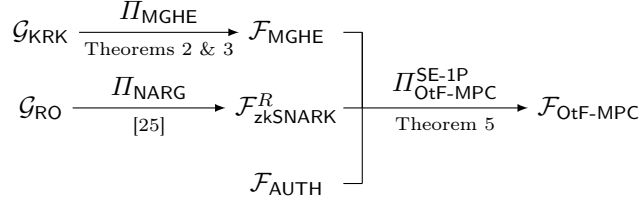


Fig. 2: Protocol $\Pi_{\text{OtF-MPC}}^{\text{SE-1P}}$ UC-realizes $\mathcal{F}_{\text{OtF-MPC}}$ against a (full) malicious adversary in the $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{zkSNARK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of global subroutines $\mathcal{G}_{\text{KRK}}$ and $\mathcal{G}_{\text{RO}}$.

Throughout, ciphertexts and proofs are transmitted over an authenticated channel $\mathcal{F}_{\text{AUTH}}$,⁴ following standard assumptions in the literature.

Malicious Security with Simulation-Soundness. Simulation-extractability leads to a straightforward construction, but it is a relatively strong property to require from a zkSNARK. Indeed, some state-of-the-art zkSNARK constructions (such as Plonk [32]) are not known to be simulation extractable in some idealized models [47]. For generality, and to distill the minimal properties for the building blocks of our compilers, we additionally define a compiler which only requires *simulation soundness* rather than simulation extractability.

In the compiler $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$, we replace the simulation-extractable zkSNARK with a *simulation-sound* NIZK. We define our functionality $\mathcal{F}_{\text{NIZK}}$ by combining

⁴ We omit the construction of an authenticated layer in this work. It can easily be constructed from a digital signature scheme and certificate authority (CA) as discussed by Canetti [16]. We also present an ideal functionality $\mathcal{F}_{\text{AUTH}}$ for the authenticated channel that fits our need in the appendix.

the ideas from [15] and [35], where we split a witness into a knowledge one and an existential one. This compiler employs a *two-pass* (2P) structure in each phase, specifically:

- *Input phase*: each input is doubly encrypted following the Naor–Yung paradigm [55], with a proof that the ciphertexts are well-formed.
- *Computation phase*: circuit evaluation is performed on the paired ciphertexts, accompanied by a proof that the evaluation is faithful.
- *Output phase*: both ciphertexts in each pair are partially decrypted, along with a proof showing the validity of both partial decryptions.

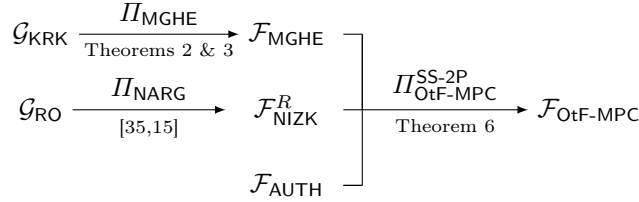


Fig. 3: Protocol $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$ UC-realizes $\mathcal{F}_{\text{OtF-MPC}}$ against a (full) malicious adversary in the $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of global subroutines $\mathcal{G}_{\text{KRK}}$ and $\mathcal{G}_{\text{RO}}$.

Similarly, all the messages are also transmitted via the authenticated channel $\mathcal{F}_{\text{AUTH}}$.

3 UC Security of Multi-Group HE

For the confidential layer of our protocol, we employ *Multi-Group Homomorphic Encryption* (MGHE) [44]. MGHE is a hybrid primitive that combines the features of multi-key and threshold homomorphic encryption. Informally, each *group* of parties, via a threshold HE scheme, behaves as a single entity⁵ within a multi-key HE framework. We formalize the syntax of MGHE in Definition 1.

Definition 1 (Multi-Group Homomorphic Encryption). A Multi-Group Homomorphic Encryption *scheme* is a tuple of probabilistic polynomial-time

⁵ In our syntax, the key aggregation algorithm takes as input both public and secret keys. This choice reflects the fact that, in most existing works [3,56,52], the generation of a group’s public *evaluation key* requires access to the parties’ secret keys. Consequently, the key aggregation step must, in practice, be implemented via an MPC protocol. An exception is the recent construction of Kwak et al. [44], where the evaluation key can be derived in a fully *non-interactive* manner. For generality, however, we adopt the more conservative definition where both public and secret keys are required for key aggregation.

(PPT) algorithms $\text{MGHE} = (\text{PGen}, \text{KGen}, \text{KJoin}, \text{Enc}, \text{Eval}, \text{PDec}, \text{Combine})^6$ defined as follows:

- $\text{pp} \leftarrow \text{PGen}(1^\lambda, 1^\kappa)$: On input the security parameter λ and the bound κ on the circuit depth, the algorithm outputs public parameters pp .
- $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(\text{pp})$: On input the public parameter pp , the algorithm outputs a key pair (pk, sk) .
- $\text{jpg} \leftarrow \text{KJoin}((\text{pk}_i, \text{sk}_i)_{i \in I})$: On input the key pairs $(\text{pk}_i, \text{sk}_i)_{i \in I}$ of parties indexed by I , the distributed protocol outputs a joint public key jpg .
- $c \leftarrow \text{Enc}(\text{jpg}, m)$: On input the joint public key jpg and a message m , the algorithm outputs a ciphertext c .
- $\hat{c} \leftarrow \text{Eval}(f, (\text{jpg}_j, c_j)_{j \in [\ell]})$: On input a circuit f of depth at most κ , ciphertexts $c_1, \dots, c_\ell$, and their associated joint public keys $\text{jpg}_1, \dots, \text{jpg}_\ell$, the algorithm outputs an evaluated ciphertext $\hat{c}$.
- $m \leftarrow \text{DistDec}(\{\text{sk}_i\}_{i \in I}, c)$: On input a ciphertext c and the secret keys $\{\text{sk}_i\}_{i \in I}$ of parties in I , the distributed protocol outputs the decrypted message m . This is realized in two steps:
 - $d \leftarrow \text{PDec}(\text{sk}_i, c)$: On input a ciphertext c and the secret key sk_i of a party, the algorithm outputs the partial decryption d .
 - $m \leftarrow \text{Combine}(c, \{d_i\}_{i \in I})$: On input a ciphertext c and the partial decryptions $\{d_i\}_{i \in I}$ of c , the algorithm outputs the message m .

3.1 Functionality $\mathcal{F}_{\text{MGHE}}$

In Fig. 4, we present the ideal functionality $\mathcal{F}_{\text{MGHE}}$ for a multi-group homomorphic encryption scheme, with a global subroutine $\mathcal{G}_{\text{KRK}}$ (Appendix D, Fig. 16) for management of keys and public parameters, inspired by [6,18].

- *Input phase*: a client $\mathcal{P}_{\text{cli}}$ sends a message m to $\mathcal{F}_{\text{MGHE}}$. Depending on the corruption state of the client and its group, $\mathcal{F}_{\text{MGHE}}$ reveals either m or its length $|m|$ to the (ideal-process) adversary $\mathcal{S}$, who then simulates the corresponding ciphertext c .
- *Computation phase*: the server $\mathcal{P}_{\text{srv}}$ sends a circuit f to $\mathcal{F}_{\text{MGHE}}$. The functionality asks $\mathcal{S}$ to simulate the evaluation of f on the ciphertexts provided by the clients. If the server is uncorrupted, only the circuit size $|f|$ is provided to $\mathcal{S}$ to model circuit privacy. We note that we also model *multi-hop* evaluation in this functionality where an evaluated result can further serve as the input of the next evaluation.
- *Output phase*: a client $\mathcal{P}_{\text{cli}}$ first requests its partial decryption d of a ciphertext c from $\mathcal{F}_{\text{MGHE}}$. To obtain d , $\mathcal{F}_{\text{MGHE}}$ queries $\mathcal{S}$, providing it with either c or the pair

⁶ Throughout the remainder of this work, unless an algorithm explicitly specifies a random coin (e.g., $\text{MGHE.Enc}(\text{jpg}, m; \omega)$), it is assumed that the random coin is sampled honestly according to the distribution prescribed by the scheme.

⁷ In this case, either an insufficient number of partial decryptions are available, which causes the protocol to abort, or the adversary has corrupted more than $\lceil \frac{g \cdot p}{t} \rceil$ clients, enabling it to forge an incorrect result. Since we assume the adversary corrupts no more than the threshold number of clients, we omit this case from the description.

Functionality: $\mathcal{F}_{\text{MGHE}}$

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\text{srv}}$; an ideal-process adversary $\mathcal{S}$.

- $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m ; \text{ } \overline{\omega}] \rangle$ from $\mathcal{P}_{\text{cli}}$:
 1. Retrieve jpg' from $\mathcal{G}_{\text{KRK}}$. If $\text{jpg} \neq \text{jpg}'$, abort.
 2. If fewer than $\lceil \frac{|\text{grp}|}{t} \rceil$ clients have been corrupted and $\mathcal{P}_{\text{cli}}$ is not corrupted:
 - (a) Output $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, |m|] \rangle$ to $\mathcal{S}$.
 3. If $\mathcal{P}_{\text{cli}}$ is corrupted, output $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m ; \text{ } \overline{\omega}] \rangle$ to $\mathcal{S}$.
 4. Otherwise, output $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m] \rangle$ to $\mathcal{S}$.
 5. When $\mathcal{S}$ returns $(c ; \omega)$:
 - (a) Set $\delta := \text{GetEncNoise}(\omega)$. // Extract noise from encryption randomness and track it.
 - (b) Record $(m ; \delta, c)$ if $\mathcal{P}_{\text{cli}}$ is uncorrupted.
 - (c) Record $(m ; \omega, c)$ if $\mathcal{P}_{\text{cli}}$ is corrupted.
 6. Output $\{\text{"ct"} : [\text{grp}, \text{sid}, c]\}$ to $\mathcal{P}_{\text{cli}}$.
- $\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]} ; \text{ } \overline{\sigma}] \rangle$ from $\mathcal{P}_{\text{srv}}$:
 1. For each grp_j , retrieve jpg'_j from $\mathcal{G}_{\text{KRK}}$. If there is $j \in [\ell]$ such that $\text{jpg}_j \neq \text{jpg}'_j$, abort.
 2. If there exists $j \in [\ell]$ such that there is no $(m_j ; \delta_j, c_j)$ store, abort.
 3. If $\text{sid} \neq \text{sid}_j$ for any $j \in [\ell]$, abort.
 4. Set $\text{grp} = \cup_{j \in [\ell]} \text{grp}_j$ and compute $\hat{m} = f(m_1, \dots, m_\ell)$.
 5. If $\mathcal{P}_{\text{srv}}$ is not corrupted:
 - (a) Output $\langle \text{Evaluate} : [\text{sid}, |f|, \hat{m}, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}] \rangle$ to $\mathcal{S}$.
 6. If $\mathcal{P}_{\text{srv}}$ is corrupted:
 - (a) Output $\langle \text{Evaluate} : [\text{sid}, f, \hat{m}, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]} ; \text{ } \overline{\sigma}] \rangle$ to $\mathcal{S}$.
 7. When $\mathcal{S}$ returns $(\hat{c}, \hat{f} ; \sigma)$:
 - (a) Set $\hat{\delta} := \text{GetEvalNoise}(f, \{\delta_j\}_{j \in [\ell]}, \sigma)$ // Compute accumulated noise.
 - (b) Record $(\hat{m} ; \hat{\delta}, \hat{c})$ if $\mathcal{P}_{\text{srv}}$ is uncorrupted.
 - (c) Record $(\hat{m} ; \sigma, \hat{c})$ if $\mathcal{P}_{\text{srv}}$ is corrupted.
 8. Output $\{\text{"ct"} : [\text{grp}, \text{sid}, \hat{c}]\}$ to $\mathcal{P}_{\text{srv}}$.
- $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c ; \text{ } \overline{\eta}] \rangle$ from $\mathcal{P}_{\text{cli}}$:
 1. If there is no $(m ; \delta, c)$ stored, abort.
 2. If $\mathcal{P}_{\text{cli}}$ is not corrupted: Output $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c, m] \rangle$ to $\mathcal{S}$.
 3. If $\mathcal{P}_{\text{cli}}$ is corrupted: Output $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c ; \text{ } \overline{\eta}] \rangle$ to $\mathcal{S}$.
 4. When $\mathcal{S}$ returns $(d ; \eta)$, record $(c, m, d ; \eta)$ if $\mathcal{P}_{\text{srv}}$ is uncorrupted.
 5. Output $\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, c, d]\}$ to $\mathcal{P}_{\text{cli}}$.
- $\langle \text{Combine} : [\text{grp}, \text{sid}, c, \{d_j\}_{\text{cli}_j \in \text{grp}'}] \rangle$ from $\mathcal{P}_{\text{cli}}$.
 1. If there is no $(m ; \delta, c)$ stored, abort.
 2. If there exist $(c, m, d'_i ; \eta_i)_{\text{cli}_i \in \text{grp}''}$ such that $|\{d'_i\}_{\text{cli}_i \in \text{grp}''} \cap \{d_j\}_{\text{cli}_j \in \text{grp}'}| \geq \lceil \frac{|\text{grp}|}{t} \rceil$:
 - (a) Set the noise $\hat{\delta} := \delta + \sum_{\text{cli}_i \in \text{grp}'' \cap \text{Corrupt}} \eta_i + \sum_{\text{cli}_i \in \text{grp}'' \setminus \text{Corrupt}} \eta_i - \delta$.
 - (b) Output $\{\text{"pt"} : [\text{grp}, \text{sid}, c, m + \hat{\delta}]\}$ to $\mathcal{P}_{\text{cli}}$.
 3. Otherwise, abort.⁷
- $\langle \text{Corrupt} : [\text{pid}] \rangle$ from $\mathcal{S}$:
 1. Label $\mathcal{P}_{\text{pid}}$ as corrupted.

Fig. 4: Functionality $\mathcal{F}_{\text{MGHE}}$ for multi-group HE. Highlighted components apply only when MGHE is approximate. Dot-boxed components appear exclusively when $\mathcal{S}$ simulates a *semi-malicious* adversary (for Section 3). Framed components appear exclusively when $\mathcal{S}$ simulates a *malicious* adversary (for Section 4).

(c, m) , depending on corruption, and receives the simulated partial decryption d . Once a sufficient number of partial decryptions have been collected, the client submits them to $\mathcal{F}_{\text{MGHE}}$ to reconstruct the message. We note that this reconstruction step does not involve $\mathcal{S}$, highlighting that it is a non-interactive process.

To model a semi-malicious adversary, we allow the adversary to choose the random coins (from an arbitrary distribution) whenever it instructs a corrupted party to provide input to $\mathcal{F}_{\text{MGHE}}$. To account for approximate HE, the functionality also tracks the noise incurred during encryption, evaluation, and partial decryption, along with the corresponding plaintext-ciphertext pairs. For this purpose, we define two efficiently computable functions:

- **GetEncNoise** $(\omega) \rightarrow \delta$: on input the randomness ω used in encryption, output the corresponding noise δ .
- **GetEvalNoise** $(f, \{\delta_j\}_{j \in [\ell]}, \sigma) \rightarrow \hat{\delta}$: on input a circuit f , the noise values $\{\delta_j\}_{j \in [\ell]}$ of input ciphertexts $\{c_j\}_{j \in [\ell]}$, and the server’s additional noise σ (for circuit privacy), output the noise $\hat{\delta}$ of the evaluated ciphertext.

We emphasize that only ciphertexts generated by $\mathcal{F}_{\text{MGHE}}$ are eligible for evaluation and decryption. This restriction allows the functionality to maintain a lookup table of message-ciphertext pairs when $\mathcal{S}$ simulates a *semi-malicious* adversary. This analogizes to the mechanisms used in the IND-CPA^{pD}, IND-CIRC, and robustness (Rob) security games (Figures 5, 6, and 7) presented later in Section 3.2 and facilitates reductions to these security notions.

3.2 What Are the “Right” Security Notions for MGHE?

The primary security requirement of homomorphic encryption is *semantic security*, formalized with the IND-CPA notion, as established by Gentry [33]. However in practice, IND-CPA security alone is too weak, as (i) it does not protect against disclosure of partial-decryption, which is a necessary requirement in the multi-party setting, (ii) does not give any confidentiality guarantees to the circuit evaluator, and (iii) does not cover the security in a generalized threshold setting. In the following, we address this by introducing our notions that are tailored to building on-the-fly MPC.

IND-CPA^{pD} Security. The security notion that Kwak et al. [44] first defined for multi-group HE is IND-CPA. However, it is not sufficient given that several attacks [1,24,36,22] have been introduced targeting both approximate and exact HE schemes by exploiting the disclosure of (partial) decryption.. We introduce a variant of IND-CPA^D security [45], which we call *Indistinguishability of Chosen Plaintext Attack under Partial-Decryption*, or IND-CPA^{pD} for short (Fig. 5, Definition 2).

We extend the IND-CPA^D security [45] to the *semi-malicious* setting by introducing an oracle $\mathcal{O}_{\text{Enc}^*}$ that lets $\mathcal{A}$ fix the randomness used in encryptions of *non-challenge* messages, following Bernard et al. [8]. This enables a refinement

Game: $G_{\text{MGHE}}^{\text{IND-CPA}^{\text{pD}}-b}$

Procedure INIT

```

1:  $\text{pp} \leftarrow \text{MGHE.PGen}(1^\lambda, 1^\kappa)$ 
2:  $(\{\gamma_i\}_{i \in I_A}, I_A) \leftarrow \mathcal{A}(\text{pp}) \parallel I_A \subseteq [n]$ 
3: for  $i \in [n] \setminus I_A$  do
4:    $(\text{pk}_i, \text{sk}_i) \leftarrow \text{MGHE.KGen}(\text{pp})$ 
5: for  $i \in I_A$  do
6:    $(\text{pk}_i, \text{sk}_i) \leftarrow \text{MGHE.KGen}(\text{pp}; \gamma_i)$ 
7:  $\{I_1, \dots, I_k\} \leftarrow \mathcal{A}(\{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_i\}_{i \in I_A})$ 
8: for  $j \in [k]$  do
9:    $\text{jpk}_j \leftarrow \text{MGHE.KJoin}((\text{pk}_i, \text{sk}_i)_{i \in I_j})$ 
10:  $T := []$ 

```

Oracle $\mathcal{O}_{\text{Eval}}(f, L; \sigma)$

```

1:  $\hat{I} := \cup_{l \in L} T[l].I$ 
2: if  $|\hat{I} \cap I_A| \geq \lceil \frac{|\hat{I}|}{t} \rceil$  then
3:   return  $\perp$ 
4:  $\hat{c} \leftarrow \text{MGHE.Eval}(f, (T[l].\text{pk}, T[l].c)_{l \in L}; \sigma)$ 
5:  $\hat{m}_0 := f((T[l].m_0)_{l \in L})$ 
6:  $\hat{m}_1 := f((T[l].m_1)_{l \in L})$ 
7:  $\text{jpk} \leftarrow \text{MGHE.KJoin}((\text{pk}_i, \text{sk}_i)_{i \in \hat{I}})$ 
8:  $T := T \bowtie (\hat{I}, \text{jpk}, \hat{m}_0, \hat{m}_1, \hat{c})$ 
9: return  $\hat{c}$ 

```

Oracle $\mathcal{O}_{\text{Enc}}(\text{jpk}_j, m_0, m_1)$

```

1: if  $|I_j \cap I_A| \geq \lceil \frac{|I_j|}{t} \rceil$  then
2:   return  $\perp$ 
3:  $c \leftarrow \text{MGHE.Enc}(\text{jpk}_j, m_b)$ 
4:  $T := T \bowtie (I_j, \text{jpk}_j, m_0, m_1, c)$ 
5: return  $c$ 

```

Oracle $\mathcal{O}_{\text{Enc}^*}(\text{jpk}_j, m; \omega)$

```

1:  $c \leftarrow \text{MGHE.Enc}(\text{jpk}_j, m; \omega)$ 
2:  $T := T \bowtie (I_j, \text{jpk}_j, m, m, c)$ 
3: return  $c$ 

```

Oracle $\mathcal{O}_{\text{Dec}}(l)$

```

1: if  $T[l].m_0 = T[l].m_1$  then
2:   for  $i \in T[l].I$  do
3:      $d_i \leftarrow \text{MGHE.PDec}(\text{sk}_i, T[l].c)$ 
4:   return  $\{d_i\}_{i \in T[l].I}$ 

```

Procedure FIN

```

1:  $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\{\text{Enc}, \text{Enc}^*, \text{Eval}, \text{Dec}\}}}_{\{\text{pk}_i\}_{i \in [n]}, \{\text{jpk}_j\}_{j \in [k]}, \{\text{sk}_i\}_{i \in I_A}}(b)$ 
2:
3: return  $b'$ 

```

Fig. 5: IND-CPA^{pD} game for a multi-group homomorphic encryption (MGHE) scheme. $\bowtie$ appends a tuple to a list.

of the *noise-flooding* countermeasure of Li et al. [46]: rather than scaling to worst-case noise, one can scale to average-case noise, thereby addressing attacks exploiting average-scale flooding [24,36] while improving efficiency. Additionally, while zero-knowledge arguments can be used to enforce that the randomness lies within a prescribed bound, proving that a random coin was honestly sampled from a specific distribution remains prohibitively expensive. Further, the ability to set randomness applies not only to encryption but also to key generation and to evaluations via queries to $\mathcal{O}_{\text{Eval}}$, thereby capturing cases where the evaluation server is also corrupted. We remark that our notion is also strictly stronger than that of [48] and [7], where the adversary is restricted to a semi-honest setting and only learns the random coins honestly sampled by $\mathcal{O}_{\text{Enc}}$ or $\mathcal{O}_{\text{Eval}}$, whereas in our notion the adversary is granted full control over this randomness at every

part of the protocol. As also noted in [57], both clients and the server may be corrupted. Thus, it is necessary to consider the case where the adversary controls both encryption and evaluation randomness. Finally, in the multi-group setting, $\mathcal{A}$ may define the initial k groups and fix a corrupted subset $I_{\mathcal{A}}$ *non-adaptively* at initialization. iv) Finally, since message reconstruction proceeds via group-wise partial decryptions, we modify $\mathcal{O}_{\text{Dec}}$ to return each party's share, yielding a special case of IND-CPA^D.

Definition 2 (IND-CPA^{PD} Security). *A multi-group homomorphic encryption scheme MGHE is said to satisfy $(q_e, q_{e^*}, q_v, q_d, t, \varepsilon)$ -indistinguishability under chosen plaintext attack with partial decryption query (IND-CPA^{PD}), if for any adversary $\mathcal{A}$ that makes q_e queries to $\mathcal{O}_{\text{Enc}}$, q_{e^*} queries to $\mathcal{O}_{\text{Enc}^*}$, q_v queries to $\mathcal{O}_{\text{Eval}}$, q_d queries to $\mathcal{O}_{\text{Dec}}$, and corrupts parties in a group up to the threshold with respect to the threshold factor t , the advantage $\text{Adv}_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}}(\mathcal{A}) \leq \varepsilon$ where*

$$\text{Adv}_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}}(\mathcal{A}) := \left| \Pr \left[G_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}-0}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[G_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}-1}(\mathcal{A}) \Rightarrow 1 \right] \right|.$$

Circuit Privacy. The notion of *circuit privacy* [33,40,13] requires that evaluated ciphertexts are (statistically) indistinguishable from those generated by a simulator, thereby ensuring that no information about the server's circuit is leaked. Subsequent work refined this notion, either by tailoring the security to specific techniques (e.g. ciphertext sanitization, noise-flooding) for achieving circuit privacy [30,39], or by considering relaxed variants [13,34] that allow partial disclosure of the circuit (e.g., its topology). Knabenhans et al. [43] further studied scenarios where the server possesses a private input in addition to the circuit.

In Definition 3, we formalize a simulation-based notion of circuit privacy (SIM-CIRC) for MGHE. Our definition follows the relaxed variant of [13], where the topology of the circuit is revealed. This relaxation is justified by the assumption that the clients and the server agree in advance on how each client's input is assigned to positions within the circuit. As a result, an adversary controlling corrupted participants can already infer the circuit topology during this consensus phase.

Definition 3 (SIM-CIRC Security). *A multi-group homomorphic encryption scheme MGHE satisfies circuit privacy if there exists a PPT simulator Sim_{circ} such that for any $c_1, \dots, c_\ell$ ciphertexts*

$$\text{Sim}_{\text{circ}}((\text{pk}_1, c_j)_{j \in [\ell]}, f(m_1, \dots, m_\ell)) \stackrel{s}{\approx} \text{MGHE.Eval}(f, (\text{pk}_j, c_j)_{j \in [\ell]})$$

where $c_j \leftarrow \text{MGHE.Enc}(\text{pk}_j, m_i; \omega_i)$, $\text{pk}_j \leftarrow \text{MGHE.KJoin}((\text{pk}_i, \text{sk}_i)_{i \in I_j})$ and $(\text{pk}_i, \text{sk}_i) \leftarrow \text{MGHE.KGen}(\text{pp}; \gamma_i)$ with $\text{pp} \leftarrow \text{MGHE.PGen}(1^\lambda, 1^\kappa)$ for some security parameter λ and circuit depth κ , where ω_i and γ_i are random coins that may be selected by an adversary from an arbitrary distribution.

Simulation-based definitions which use the evaluated plaintext result for simulation may not be a suitable formalization for schemes using *approximate*

evaluation. Klucznik and Santato [42] proposed a relaxation of circuit privacy by introducing a game-based notion IND-CIRC that defines an indistinguishability game with a challenge evaluation oracle. In this work, we similarly extend their notion of *circuit indistinguishability* (IND-CIRC) to the multi-group setting (Fig. 6). We note that the adversary is allowed to corrupt all the clients since we only consider server-side confidentiality with this notion.

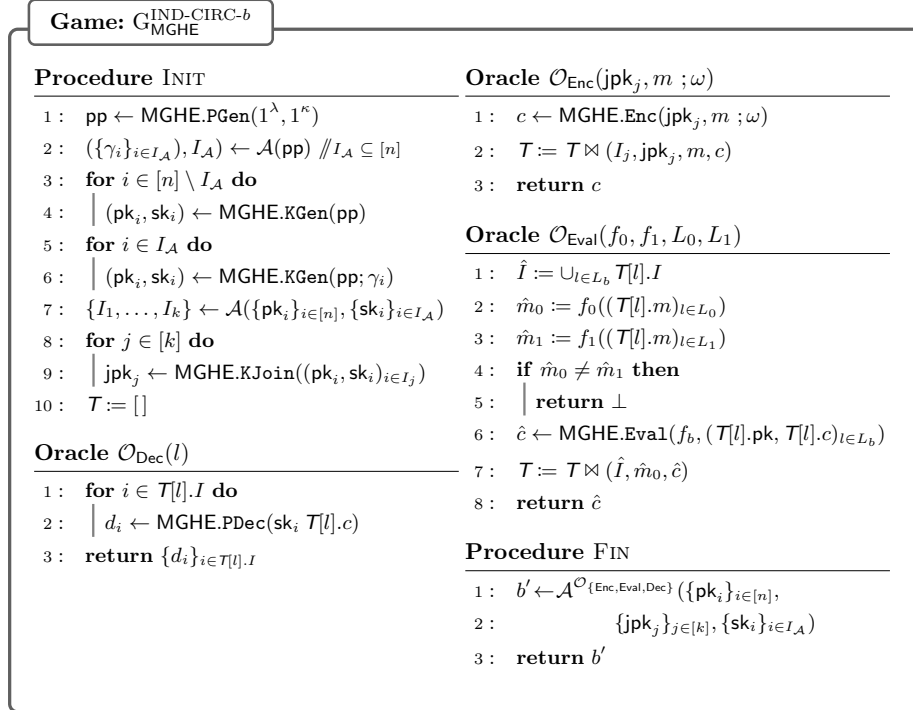


Fig. 6: IND-CIRC game for a multi-group homomorphic encryption scheme. $\bowtie$ appends a tuple to a list.

We observe that at $\mathcal{O}_{\text{Eval}}$, the adversary is allowed to select two sets L_0 and L_1 of input ciphertexts. As noted by Li and Micciancio [45], if only a single set L were permitted, the circuit topology would be revealed to the adversary. Our formalization in Fig. 6 captures the more general case yet we still define the adversary such that it only selects one set of input ciphertexts when building reductions (details in Section 3.3).

Definition 4 (IND-CIRC Security). *A multi-group homomorphic encryption scheme MGHE is said to satisfy $(q_e, q_v, q_d, \varepsilon)$ -circuit indistinguishability (IND-CIRC), if for any adversary $\mathcal{A}$ that makes q_e queries to $\mathcal{O}_{\text{Enc}}$, q_v queries*

to $\mathcal{O}_{\text{Eval}}$, q_d queries to $\mathcal{O}_{\text{Dec}}$, the advantage $\text{Adv}_{\text{MGHE}}^{\text{IND-CIRC}}(\mathcal{A}) \leq \varepsilon$ where

$$\text{Adv}_{\text{MGHE}}^{\text{IND-CIRC}}(\mathcal{A}) := \left| \Pr \left[G_{\text{MGHE}}^{\text{IND-CIRC-0}}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[G_{\text{MGHE}}^{\text{IND-CIRC-1}}(\mathcal{A}) \Rightarrow 1 \right] \right|.$$

Partial-Decryption Simulatability. We assume that the multi-group HE supports partial decryptions. This necessitates an additional security property such that partial decryption must not leak any information about the (individual) secret keys of each client. The *simulatability of partial decryption* was first formalized by Boneh et al. [10] such that there exists a simulator that simulates the partial decryption of *uncorrupted client* without the knowledge of their individual secret keys. In Definition 5, we restate the notion to fit our multi-group setting.

Definition 5 (SIM-PDEC Security). Let I be an index set representing all the clients in a group and let $I_{\mathcal{A}} \subsetneq I$ denote the index set of corrupted parties. A multi-group homomorphic encryption scheme MGHE satisfies partial-decryption simulatability (SIM-PDEC) if there exists a PPT simulator Sim_{th} such that

$$\forall i \in I \setminus I_{\mathcal{A}}, \text{Sim}_{th}(c, m, \{\text{sk}_j\}_{j \in I_{\mathcal{A}}}, \{d_j\}_{j \in I_{\mathcal{A}}}) \stackrel{s}{\approx} \text{MGHE.PDec}(\text{sk}_i, c)$$

where $d_j \leftarrow \text{MGHE.PDec}(\text{sk}_j, c)$ for $j \in I_{\mathcal{A}}$ and $m \leftarrow \text{MGHE.DistDec}(\{\text{sk}_i\}_{i \in I}, c)$.

Remark 1 (On necessity of IND-CPA^{PD} + SIM-PDEC). We stress that IND-CPA^{PD} security remains necessary in addition to SIM-PDEC security. At first glance, the oracle $\mathcal{O}_{\text{Dec}}$ may appear redundant once partial decryptions are simulatable. Several works [54,44] have combined SIM-PDEC with IND-CPA in their security proofs. However, IND-CPA + SIM-PDEC does *not* in general imply IND-CPA^{PD}, since such simulation typically depends on the underlying message m , while not every ciphertext is guaranteed to be decryptable. A concrete counterexample is the *bootstrapping failure* attack of Cheon et al. [24], where decryption failures are exploited after the (real) partial decryptions are combined. Crucially, this attack remains applicable even to CKKS with the noise-smudging countermeasure, which suffices for SIM-PDEC but fails to guarantee IND-CPA^{PD}.

Robustness. In this work, we consider the general setting of (t, n) -threshold decryption. To capture security in this setting, we introduce the notion of *robustness*, inspired by prior work [10,12]. Intuitively, robustness guarantees that the correct plaintext⁸ can always be recovered as long as a party collects a sufficient number of valid partial decryptions, even in the presence of adversarially crafted malformed shares. The first formalization of this notion in the context of *threshold* HE was given by Boudgoust and Scholl [12]. However, their definition does not account for ciphertexts resulting from homomorphic evaluation, which are central in many applications including MPC. Moreover, their model does not capture the *semi-malicious* setting, where corrupted parties may still follow the prescribed algorithm but are allowed to choose their own randomness.

⁸ Thus this notion also defines the correctness of a MGHE scheme.

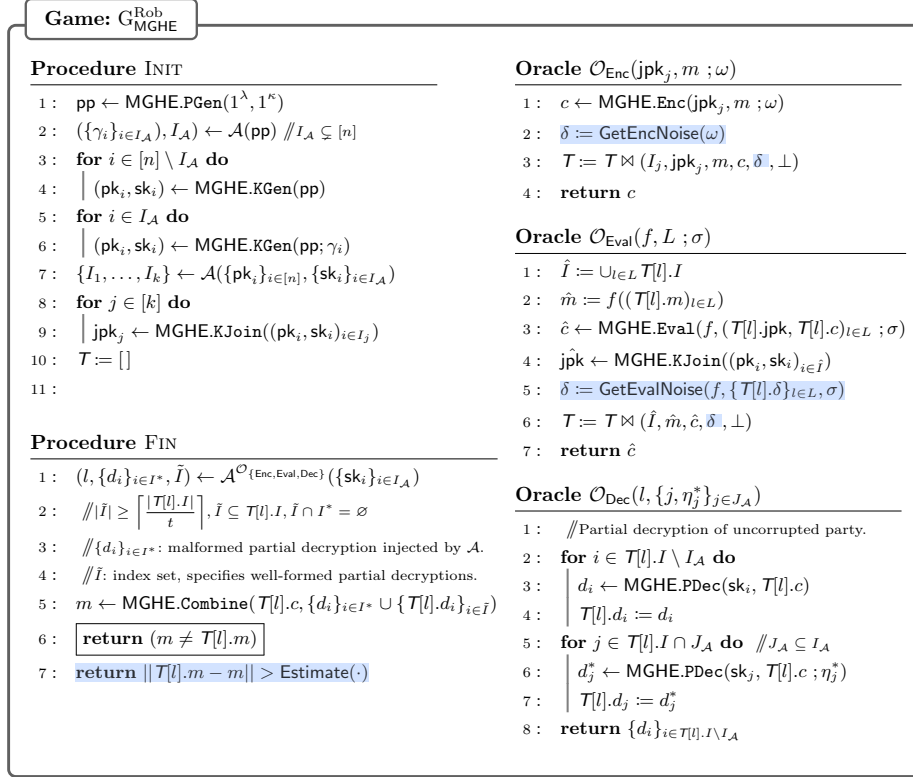


Fig. 7: Robustness (Rob) game for a multi-group homomorphic encryption scheme. Framed components appear only when MGHE is exact and perfectly correct. Highlighted components appear only when MGHE is approximate or imperfectly correct. $\bowtie$ appends a tuple to a list.

In Fig. 7, we formalize robustness for multi-group HE via an indistinguishability-style security game. The structure of the game parallels those defined earlier. First, the $\mathcal{O}_{\text{Eval}}$ oracle extends the framework of [12] to evaluated ciphertexts. Second, to capture semi-malicious behavior, we further allow the adversary to specify the randomness used in queries to both $\mathcal{O}_{\text{Enc}}$ and $\mathcal{O}_{\text{Eval}}$. Third, when interacting with $\mathcal{O}_{\text{Dec}}$, we also model a case where (some) corrupted parties in a group, defined by a set J_A , perform the partial decryption algorithm *honestly* but with randomness chosen adversarially⁹.

In the finalization phase, the adversary outputs an index set $\tilde{I}$ corresponding to the partial decryptions that are *honestly* generated (though potentially under adversarial randomness). The adversary may additionally provide a collection of

⁹ Here we still require that the adversary's randomness lies within the parameter bounds of the scheme, but without imposing adherence to any specific distribution (e.g., Gaussian).

malformed shares $\{d_i\}_{i \in I^*}$. The correctness requirement is that whenever at least $\lceil \frac{|\text{grp}|}{t} \rceil$ honest partial decryptions are available, i.e., $|\tilde{I}| \geq \lceil \frac{|\text{grp}|}{t} \rceil$, reconstruction must always yield the correct message, irrespective of any adversarially supplied partial-decryptions.

For approximate HE, we further refine the definition to account for noise growth. Specifically, we track the encryption and evaluation noise using functions $\text{GetEncNoise}(\cdot)$ and $\text{GetEvalNoise}(\cdot)$, as in our ideal functionality $\mathcal{F}_{\text{MGHE}}$. The winning condition is then defined such that the difference between the original message and reconstructed message exceeds the bound B . We define this bound using a *publicly computable* function $B \leftarrow \text{Estimate}(f, B_{\text{Enc}}, B_{\text{Eval}}, B_{\text{Dec}})$ that depends on the circuit f , the encryption error bound B_{Enc} , the evaluation error bound B_{Eval} , and the decryption error bound B_{Dec} as in [42].

Definition 6 (Robustness). *A multi-group homomorphic encryption scheme MGHE is said to satisfy $(q_e, q_v, q_d, t, \varepsilon)$ -robustness (Rob), if for any adversary $\mathcal{A}$ that makes q_e queries to $\mathcal{O}_{\text{Enc}}$, q_v queries to $\mathcal{O}_{\text{Eval}}$, q_d queries to $\mathcal{O}_{\text{Dec}}$, and corrupts a group up to the threshold with respect to a threshold factor t , the advantage $\text{Adv}_{\text{MGHE}}^{\text{Rob}}(\mathcal{A}) \leq \varepsilon$ where*

$$\text{Adv}_{\text{MGHE}}^{\text{Rob}}(\mathcal{A}) := \Pr [\mathcal{G}_{\text{MGHE}}^{\text{Rob}}(\mathcal{A}) \Rightarrow 1].$$

3.3 Realizing $\mathcal{F}_{\text{MGHE}}$

We prove that Π_{MGHE} securely realizes $\mathcal{F}_{\text{MGHE}}$ with respect to the simulator $\mathcal{S}_{\text{MGHE}}$ against a semi-malicious adversary under two corruption scenarios. In Theorem 2, where both the server and a subset of clients are corrupted, only client-side confidentiality must be ensured. In contrast, Theorem 3 together with Corollary 1 address the case of an uncorrupted server, where both client- and server-side confidentiality are required.

Theorem 2. *Let MGHE be a multi-group homomorphic encryption scheme with threshold factor t . Then, the protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global subroutine $\mathcal{G}_{\text{KRK}}$ against a semi-malicious adversary that non-adaptively corrupts the clients and the server, and injects malformed partial-decryption, if:*

- MGHE satisfies robustness (Rob), and the adversary corrupts no more than $\lceil \frac{|\text{grp}|}{t} \rceil$ clients in any group.
- MGHE is $\text{IND-CPA}^{\text{pD}}$ -secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group grp under which it provides input contains no more than $\lceil \frac{|\text{grp}|}{t} \rceil$ corrupted clients.
- MGHE satisfies SIM-PDEC security with respect to a simulator Sim_{th} .

We give a full proof of Theorem 2 in Appendix F.1.

In Theorem 3 and Corollary 1, we consider the semi-malicious setting where only a subset of clients is corrupted and the server remains uncorrupted. In this case, beyond client-side security, we must also establish server-side confidentiality, which we capture via an additional hybrid game reducible to IND-CIRC security.

Protocol: Π_{MGHE}

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\text{srv}}$; an ideal-process adversary $\mathcal{S}$.

- $\langle \text{RegParam} : [\text{sid}] \rangle$ to $\mathcal{P}_{\text{cli}}$: Forward this input to $\mathcal{G}_{\text{KRR}}$.
- $\langle \text{RegKey} : [\text{sid}] \rangle$ to $\mathcal{P}_{\text{cli}}$: Forward this input to $\mathcal{G}_{\text{KRR}}$.
- $\langle \text{JoinKey} : [\text{sid}] \rangle$ to $\mathcal{P}_{\text{grp}}$: Forward this input to $\mathcal{G}_{\text{KRR}}$.
- $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 1. Retrieve jpg' for grp from $\mathcal{G}_{\text{KRR}}$. If $\text{jpg}' \neq \text{jpg}$, abort.
 2. Otherwise, compute $c \leftarrow \text{MGHE.Enc}(\text{jpg}, m)$ and output $\{\text{"ct"} : [\text{grp}, \text{sid}, c]\}$.
- $\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}] \rangle$ to $\mathcal{P}_{\text{srv}}$:
 1. Retrieve jpg'_j for each grp_j . If there is $j \in [\ell]$ such that $\text{jpg}'_j \neq \text{jpg}_j$, abort.
 2. If $\text{sid} \neq \text{sid}_j$ for any $j \in [\ell]$, abort.
 3. Set $\text{grp} = \cup_{j \in [\ell]} \text{grp}_j$, compute $\hat{c} \leftarrow \text{MGHE.Eval}(f, (\text{jpg}_j, c_j)_{j \in [\ell]})$, and output $\{\text{"ct"} : [\text{grp}, \text{sid}, \hat{c}]\}$.
- $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 - Retrieve (pk, sk) from $\mathcal{G}_{\text{KRR}}$ and compute $d \leftarrow \text{MGHE.PDec}(\text{sk}, c)$.
 1. Output $\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, c, d]\}$.
- $\langle \text{Combine} : [\text{grp}, \text{sid}, c, \{d_i\}_{\text{cli}_i \in \text{grp}'}] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 - If $\text{grp}' \not\subseteq \text{grp}$ or $|\text{grp}' \cap \text{grp}| < \lceil \frac{|\text{grp}|}{t} \rceil$, abort. Otherwise, compute $m \leftarrow \text{MGHE.Combine}(c, \{d_i\}_{i \in \text{grp}'})$ and output $\{\text{"pt"} : [\text{grp}, \text{sid}, c, m]\}$.

Fig. 8: Protocol Π_{MGHE} for multi-group homomorphic encryption.

Theorem 3. *The protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global subroutine $\mathcal{G}_{\text{KRR}}$ against a semi-malicious adversary that non-adaptively corrupts only clients and injects malformed partial-decryptations if an approximate MGHE is additionally IND-CIRC secure.*

We give a full proof of Theorem 3 in Appendix F.2.

We also consider a stronger guarantee by requiring statistical indistinguishability for circuit privacy in the case of exact HE schemes. In Corollary 1, we establish server-side confidentiality under SIM-CIRC security.

Corollary 1. *The protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global subroutine $\mathcal{G}_{\text{KRR}}$ against a semi-malicious adversary that non-adaptively corrupts only clients and injects malformed partial-decryptations if an exact MGHE is additionally SIM-CIRC secure.*

We give a full proof of Corollary 1 in Appendix F.3.

4 UC-Secure On-the-Fly MPC from MGHE and zkSNARK/NIZK

4.1 Functionality $\mathcal{F}_{\text{OTF-MPC}}$

In Fig. 10, we present the ideal functionality $\mathcal{F}_{\text{OTF-MPC}}$ for on-the-fly MPC. At a high level, this functionality acts as a registry service and proceeds in three phases.

Simulator: $\mathcal{S}_{\text{MGHE}}$

- $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, |m|] \rangle$ from $\mathcal{F}_{\text{MGHE}}$:
 1. Sample a random coin ω from the default distribution $\mathcal{D}_\omega$.
 2. Compute $c \leftarrow \text{MGHE.Enc}(\text{jpg}, 0^{|m|}; \omega)$.
 3. Return $(c; \omega)$ to $\mathcal{F}_{\text{MGHE}}$.
- $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m] \rangle$ from $\mathcal{F}_{\text{MGHE}}$:
 1. Sample a random coin ω from the default distribution $\mathcal{D}_\omega$.
 2. Compute $c \leftarrow \text{MGHE.Enc}(\text{jpg}, m; \omega)$.
 3. Return $(c; \omega)$ to $\mathcal{F}_{\text{MGHE}}$.
- $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m; \omega] \rangle$ from $\mathcal{F}_{\text{MGHE}}$:
 1. Compute $c \leftarrow \text{MGHE.Enc}(\text{jpg}, m; \omega)$.
 2. Return $(c; \omega)$ to $\mathcal{F}_{\text{MGHE}}$.
- $\langle \text{Evaluate} : [\text{grp}, \text{sid}, |f|, \hat{m}, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}] \rangle$ from $\mathcal{F}_{\text{MGHE}}$:
 1. Compute $(\hat{c}, \sigma) \leftarrow \text{Sim}_{\text{circ}}(\{\text{jpg}_j\}_{j \in [\ell]}, \hat{m})$. *// Sim_{circ} also returns random coins σ .*
 2. Sample a circuit f^* s.t. $|f^*| = |f|$ and sample a random coin $\sigma \leftarrow \mathcal{D}_\sigma$.
 3. Compute $\hat{c} \leftarrow \text{MGHE.Eval}(f^*, (\text{jpg}_j, c_j)_{j \in [\ell]}; \sigma)$.
 4. Return $(\hat{c}, f^*; \sigma)$ to $\mathcal{F}_{\text{MGHE}}$.
- $\langle \text{Evaluate} : [\text{grp}, \text{sid}, f, \hat{m}, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}; \sigma] \rangle$ from $\mathcal{F}_{\text{MGHE}}$:
 1. Compute $\hat{c} \leftarrow \text{MGHE.Eval}(f^*, (\text{jpg}_j, c_j)_{j \in [\ell]}; \sigma)$.
 2. Return $(\hat{c}, f; \sigma)$ to $\mathcal{F}_{\text{MGHE}}$.
- $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c, m] \rangle$:
 1. Compute $(d; \eta) \leftarrow \text{Sim}_{\text{th}}(c, m, \{\text{sk}_i\}_{\text{cli}_i \in \text{Corrupt} \cap \text{grp}})$.
 2. Return $(d; \eta)$ to $\mathcal{F}_{\text{MGHE}}$.
- $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c; \eta] \rangle$ from $\mathcal{F}_{\text{MGHE}}$:
 1. Retrieve sk of $\mathcal{P}_{\text{cli}}$ from $\mathcal{G}_{\text{KRK}}$.
 2. Compute $d \leftarrow \text{MGHE.PDec}(\text{sk}, c; \eta)$.
 3. Return $(d; \eta)$ to $\mathcal{F}_{\text{MGHE}}$.

Fig. 9: Simulator $\mathcal{S}_{\text{MGHE}}$ for the functionality $\mathcal{F}_{\text{MGHE}}$ simulating a *semi-malicious* adversary. Framed components apply only when MGHE is exact and perfectly correct. Highlighted components apply only when MGHE is approximate or imperfectly correct.

In the *input phase*, clients register their data at $\mathcal{F}_{\text{OTF-MPC}}$ for later evaluation. Depending on the corruption status of the client and its group, the adversary learns either the message content or only its length. In the *computation phase*, the server submits a circuit f , and $\mathcal{F}_{\text{MGHE}}$ evaluates f over the registered inputs without revealing them to the server; the output of this evaluation is then stored in the registry. If the server is uncorrupted, the adversary learns nothing about the circuit. Finally, in the *output phase*, clients in a designated group may retrieve the evaluation result, provided that a sufficient number of uncorrupted group members participate.

To capture the multi-group setting, we introduce a “virtual party” $\mathcal{P}_{\text{grp}}$ representing each group. After the initialization of all n clients, we assume k initial groups $\mathcal{P}_{\text{grp}_1}, \dots, \mathcal{P}_{\text{grp}_k}$ are formed, where $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ for $I_j \subseteq [n]$ and $j \in [k]$.

Functionality: $\mathcal{F}_{\text{OTF-MPC}}$

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\text{srv}}$; an ideal-process adversary $\mathcal{S}$.

- $\langle \text{Upload} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \rangle$ from $\mathcal{P}_{\text{cli}}$:
 1. If fewer than $\lceil \frac{|\text{grp}|}{t} \rceil$ clients in grp have been corrupted and $\mathcal{P}_{\text{cli}}$ is not corrupted, output $\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m]\}$ to $\mathcal{S}$.
 2. Otherwise, output $\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m]\}$ to $\mathcal{S}$.
 3. If there is a record under the label $(\text{grp}, \text{sid}, \text{uid})$, abort.
 4. Otherwise, record $\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m]\}$.
- $\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, \hat{m}^*, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle$ from $\mathcal{P}_{\text{srv}}$:
 1. Retrieve $\{\text{" " : " , grp}_j, \text{sid}_j, \text{uid}_j, m_j\}$ under the label $(\text{grp}_j, \text{sid}_j, \text{uid}_j)$ for $j \in [\ell]$. If there exist $m_j = \perp$, abort.
 2. If $\text{sid} \neq \text{sid}_j$ for any $j \in [\ell]$, abort.
 3. Set $\hat{m} = f(\{m_j\}_{j \in [\ell]})$ and $\hat{\text{grp}} = \cup_{j \in [\ell]} \text{grp}_j$.
 4. If $\mathcal{P}_{\text{srv}}$ is not corrupted:
 - (a) If there is a record under the label $(\hat{\text{grp}}, \text{sid}, \text{uid})$, abort.
 - (b) Otherwise, record $\{\text{"eval"} : [\text{srv}, \hat{\text{grp}}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{m}]\}$ under the label $(\text{grp}, \text{sid}, \text{uid})$.
 - (c) Output $\{\text{"eval"} : [\text{srv}, \hat{\text{grp}}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{m}]\}$ at $\mathcal{S}$.
 5. If $\mathcal{P}_{\text{srv}}$ is corrupted:
 - (a) If there is a record under the label $(\hat{\text{grp}}, \text{sid}, \text{uid})$, abort.
 - (b) If $\hat{m}^*$ is not provided:
 - (i) Record $\{\text{"eval"} : [\text{srv}, \hat{\text{grp}}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{m}]\}$ under the label $(\text{grp}, \text{sid}, \text{uid})$.
 - (ii) Output $\{\text{"eval"} : [\text{srv}, \hat{\text{grp}}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{m}]\}$ at $\mathcal{S}$.
 - (c) Otherwise, record $\{\text{"eval"} : [\text{srv}, \hat{\text{grp}}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{m}^*]\}$ under the label $(\text{grp}, \text{sid}, \text{uid})$.
- $\langle \text{Retrieve} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle$ from $\mathcal{P}_{\text{cli}}$:
 1. Retrieve message $\{\text{typ} : \text{ " , grp}, \text{sid}, \text{uid}, \text{ " , } m\}$ under the label $(\text{grp}, \text{sid}, \text{uid})$.
 2. If $\mathcal{P}_{\text{cli}}$ is not corrupted:
 - (a) If $\text{cli} \notin \text{grp}$, abort.
 - (b) If $\text{typ} = \text{"up"}$, then abort.
 - (c) If $\text{typ} = \text{"eval"}$ and fewer than $\lceil \frac{|\text{grp}|}{t} \rceil$ clients in grp have been corrupted, output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$ to $\mathcal{P}_{\text{cli}}$.
 3. If $\mathcal{P}_{\text{cli}}$ is corrupted:
 - (a) If $\text{typ} = \text{"up"}$ and more than $\lceil \frac{|\text{grp}|}{t} \rceil$ clients in grp have been corrupted, then output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$ to $\mathcal{P}_{\text{cli}}$.
 - (b) If $\text{typ} = \text{"eval"}$, output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$ to $\mathcal{P}_{\text{cli}}$.
- $\langle \text{Corrupt} : [\text{pid}] \rangle$ from $\mathcal{S}$:
 1. Label $\mathcal{P}_{\text{pid}}$ as corrupted.

Fig. 10: Functionality $\mathcal{F}_{\text{OTF-MPC}}$ for on-the-fly MPC in group-wise setting. Dot-boxed inputs are exclusively from the corrupted party (on behalf of the adversary).

New groups created during computation are modeled as unions of existing ones¹⁰, i.e., $\hat{\text{grp}} = \cup_{j \in [\ell]} \text{grp}_j$, where each grp_j contributes an input to the computation of

¹⁰ We remark that MGHE supports dynamic group formation via a standard *keyswitching* operation, whereby a ciphertext can be transformed to be decryptable under the joint public key of a newly formed group, as also noted by Mouchet et al. [53].

a circuit. Since groups may be formed dynamically and vary in size, reconstruction is parameterized by a *threshold factor* t fixed for all groups¹¹ i.e., for any group grp , at least $\lceil \frac{|\text{grp}|}{t} \rceil$ members are required to reconstruct a secret.

We assume *pid-wise corruption* in this functionality, and in all subsequent ones, following the definition in [17]. In this model, a party $\text{pid} \in \{\text{cli}_1, \dots, \text{cli}_n, \text{srv}\}$ is corrupted once the adversary sends a corruption message to the functionality; this message has no further effect beyond marking pid as corrupted. From that point onward, all inputs from and outputs to $\mathcal{P}_{\text{pid}}$ are instead provided by, or delivered to, the adversary.

Remark 2. We observe that the functionality $\mathcal{F}_{\text{OtF-MPC}}$ captures *exact* secure multiparty computation. This functionality can be naturally extended to support *secure approximation*, in the spirit of the $\mathcal{F}_{\text{MGHE}}$ functionality (Fig. 4), by explicitly tracking the error incurred at each step and aggregating it over the course of the computation. This approach aligns with the notion of *liberal security* for approximate computation, as introduced by Feigenbaum et al. [31]. To achieve *standard security*, Hwang et al. [38] proposed a formalism that fixes the total error prior to protocol execution and distributes “shares of” noise to each party. We refer the reader to these works for further technical details.

4.2 Realizing $\mathcal{F}_{\text{OtF-MPC}}$ against Semi-Malicious Adversary

We recall the result from [49,54,44]: multi-party HE alone suffices to obtain an on-the-fly MPC protocol secure against a semi-malicious (SM) adversary. From a UC perspective, in Fig. 11, we present $\Pi_{\text{OtF-MPC}}^{\text{SM}}$ ¹² as a $\mathcal{F}_{\text{MGHE}}$ -hybrid protocol, where all computation are through our ideal functionality $\mathcal{F}_{\text{MGHE}}$.

Theorem 4. *The protocol $\Pi_{\text{OtF-MPC}}^{\text{SM}}$ UC-realizes $\mathcal{F}_{\text{OtF-MPC}}$ in $\mathcal{F}_{\text{MGHE}}$ -hybrid model in presence of a global subroutine $\mathcal{G}_{\text{KRK}}$ against a semi-malicious adversary that corrupts a group up to the threshold and possibly the server.*

We give a full proof of Theorem 4 in Appendix F.4.

4.3 Compiling with zkSNARK for Malicious Security

In Fig. 12, we present the protocol $\Pi_{\text{OtF-MPC}}^{\text{SE-1P}}$, which leverages a (straightline) *simulation-extractable* zkSNARK to enforce integrity. Intuitively, simulation-extractability guarantees that any valid proof can only be produced by a party that knows the corresponding witness, thus preventing any tampering or mauling of ciphertexts. Consequently, parties cannot deviate from the protocol undetected:

¹¹ For simplicity, we adopt a fixed threshold factor across all groups. Supporting group-specific thresholds could be achieved through inter-group consensus when they form a new group.

¹² We assume, in $\Pi_{\text{OtF-MPC}}^{\text{SM}}$ and all subsequent protocols, each group entity $\mathcal{P}_{\text{grp}}$ invokes $\mathcal{G}_{\text{KRK}}$ to combine the keys of its members into a group public key jpk upon formation of the group. For simplicity, this step is omitted from the protocol descriptions.

Protocol: $\Pi_{\text{OTF-MPC}}^{\text{SM}}$

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\text{srv}}$.

- $\langle \text{Upload} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 1. Call $\mathcal{G}_{\text{KRK}}$ in sid to get jpK of grp .
 2. Call $\mathcal{F}_{\text{MGHE}}$ in sid with m to obtain ciphertext c .
 3. Output $\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, c]\}$.
- $\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle$ to $\mathcal{P}_{\text{srv}}$:
 1. If $\text{sid} \neq \text{sid}_j$ for any $j \in [\ell]$, abort.
 2. For $j \in [\ell]$ do:
 - (a) Call $\mathcal{G}_{\text{KRK}}$ in sid to get jpK_j .
 - (b) Retrieve c_j recorded under $(\text{grp}_j, \text{sid}_j, \text{uid}_j)$.
 3. Set $\text{grp} = \cup_{i \in [\ell]} \text{grp}_j$.
 4. Call $\mathcal{F}_{\text{MGHE}}$ in sid with $(\text{jpK}_j, c_j)_{j \in [\ell]}$ to obtain the evaluated ciphertext $\hat{c}$.
 5. Output $\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{c}]\}$.
- $\langle \text{Retrieve} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 1. If $\text{cli} \notin \text{grp}$, abort.
 2. Retrieve message $\{\text{typ} : \cdot, \text{grp}, \text{sid}, \text{uid}, \cdot, c\}$.
 3. If $\text{typ} = \text{"up"}$, abort. //Partially decrypt evaluated ctxt only.
 4. Call $\mathcal{F}_{\text{MGHE}}$ in session sid to obtain partial decryption d .
 5. Output $\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, d]\}$.
 6. Set $\mathcal{S}_{\text{sh}} := \emptyset$.
 7. For $\text{cli}' \in \text{grp} \setminus \{\text{cli}\}$ do:
 - (a) Receive message $\{\text{"sh"} : [\text{cli}', \text{grp}, \text{sid}, \text{uid}, d]\}$.
 - (b) Set $\mathcal{S}_{\text{sh}} := \mathcal{S}_{\text{sh}} \cup \{d\}$.
 8. If $|\mathcal{S}_{\text{sh}}| \geq \lceil \frac{|\text{grp}|}{t} \rceil$:
 - (a) Call $\mathcal{F}_{\text{MGHE}}$ in session sid with $\mathcal{S}_{\text{sh}}$ to obtain a message m .
 - (b) Output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$.
 9. Otherwise, abort.

Fig. 11: Protocol $\Pi_{\text{OTF-MPC}}^{\text{SM}}$ in $\mathcal{F}_{\text{MGHE}}$ -hybrid model that is secure against a semi-malicious adversary.

at each phase of input, computation, and output, the designated party invokes $\mathcal{F}_{\text{zkSNARK}}$ instantiated with a phase-specific relation to prove that its respective MGHE operations were performed honestly and correctly.

We utilize the functionality $\mathcal{F}_{\text{zkSNARK}}$ (Appendix C, Fig. 15) defined by Chiesa and Fenzi [25], which is realizable in the presence of a global programmable random oracle $\mathcal{G}_{\text{RO}}$ [14] (Appendix C, Fig. 14). We make a modification to it to require *any* simulation security [29]. Specifically, a binary relation need not be satisfied when the zkSNARK simulator generates an argument. This is because, for example, when composed with $\mathcal{F}_{\text{MGHE}}$, the ideal-process adversary $\mathcal{S}$ only learns the message length $|m|$ when simulating a ciphertext for an uncorrupted client. Consequently, the resulting plaintext-ciphertext pair may not satisfy the relation, yet it is still considered an honest execution (following the completeness of zkSNARK).

In the input phase, we define the relation R_{Enc} to enforce the well-formedness of ciphertexts, ensuring that every submitted ciphertext indeed encrypts some

Protocol: $\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}$

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\text{srv}}$.

- $\langle \text{Upload} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 1. Call $\mathcal{G}_{\text{KRK}}$ in sid with grp to get jpk.
 2. Call $\mathcal{F}_{\text{MGHE}}$ in sid with m to get ciphertext c .
 3. Call $\mathcal{F}_{\text{MGHE}}$ in sid to get randomness ω .
 4. Call $\mathcal{F}_{\text{zkSNARK}}^{\text{Renc}}$ in sid with instance (jpk, c) and witness (m, ω) to get π .
 5. Output $\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c, \pi)]\}$ to $\mathcal{F}_{\text{AUTH}}$.
- $\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle$ to $\mathcal{P}_{\text{srv}}$:
 1. If $\text{sid} \neq \text{sid}_j$ for any $j \in [\ell]$, abort.
 2. For $j \in [\ell]$ do:
 - (a) Get the message msg under label $(\text{grp}_j, \text{sid}_j, \text{uid}_j)$ from $\mathcal{F}_{\text{AUTH}}$.
 - (b) If $\text{msg} = \{\text{"up"} : [\text{cli}, \text{grp}_j, \text{sid}_j, \text{uid}_j, (c_j, \pi_j)]\}$.
 - (i) Call $\mathcal{G}_{\text{KRK}}$ in sid to get jpk_j of grp_j .
 - (ii) Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ to verify π_j with instance (jpk_j, c_j) . Abort if verification fails.
 - (c) Else if $\text{msg} = \{\text{"eval"} : [\text{srv}, \text{grp}_j, \text{sid}_j, \text{uid}_j, (\text{grp}_{j'}, \text{sid}_{j'}, \text{uid}_{j'})_{j' \in [\ell']}, (c_j, \pi_j)]\}$.
 - (i) Call $\mathcal{G}_{\text{KRK}}$ in sid to get $\text{jpk}_{j'}$ of $\text{grp}_{j'}$ for $j' \in [\ell']$.
 - (ii) Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ to verify π_j with instance $((\text{jpk}_{j'}, c_{j'})_{j' \in [\ell']}, c_j)$.
 - (iii) Abort if verification fails or any of $c_{j'}$ for $j' \in [\ell']$ is invalid.
 3. Set $\text{grp} = \cup_{i \in [\ell]} \text{grp}_j$.
 4. Call $\mathcal{F}_{\text{MGHE}}$ in sid with f and $(\text{jpk}_j, c_j)_{j \in [\ell]}$ to obtain $\hat{c}$.
 5. Call $\mathcal{F}_{\text{MGHE}}$ in sid to obtain randomness σ .
 6. Call $\mathcal{F}_{\text{zkSNARK}}^{\text{Renc}}$ in sid with instance $((\text{jpk}_j, c_j)_{j \in [\ell]}, \hat{c})$ and witness σ to get π .
 7. Output $\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{c}, \pi]\}$ to $\mathcal{F}_{\text{AUTH}}$.
- $\langle \text{Retrieve} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle$ to $\mathcal{P}_{\text{cli}}$:
 1. If $\text{cli} \notin \text{grp}$, abort.
 2. Get the message msg under label $(\text{grp}, \text{sid}, \text{uid})$ from $\mathcal{F}_{\text{AUTH}}$.
 3. Abort if $\text{msg} = \{\text{"up"} : [\text{cli}', \text{grp}, \text{sid}, \text{uid}, (c, \pi)]\}$.

// Partially decrypt evaluated ciphertext only.
 4. Parse msg as $\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \hat{c}, \pi]\}$.
 5. Call $\mathcal{G}_{\text{KRK}}$ in sid to get jpk_j of grp_j for $j \in [\ell]$.
 6. Call $\mathcal{F}_{\text{zkSNARK}}^{\text{Renc}}$ in sid to verify π with instance $((\text{jpk}_j, c_j)_{j \in [\ell]}, \hat{c})$.
 7. Abort if verification fails or any input ciphertext c_j is invalid.
 8. Call $\mathcal{G}_{\text{KRK}}$ in sid with cli to get sk.
 9. Call $\mathcal{F}_{\text{MGHE}}$ in sid to get partial decryption d .
 10. Call $\mathcal{F}_{\text{MGHE}}$ in sid to get randomness η .
 11. Call $\mathcal{F}_{\text{zkSNARK}}^{\text{Rdec}}$ in sid with instance $(\hat{c}, d)$ and witness (sk, η) to get π .
 12. Output $\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, d, \pi]\}$ to $\mathcal{F}_{\text{AUTH}}$.
 13. Set $S_{\text{sh}} := \emptyset$.
 14. For $\text{cli}' \in \text{grp} \setminus \{\text{cli}\}$ do:
 - (a) Get message $\{\text{"sh"} : [\text{cli}', \text{grp}, \text{sid}, \text{uid}, d, \pi]\}$ from $\mathcal{F}_{\text{AUTH}}$.
 - (b) Call $\mathcal{F}_{\text{zkSNARK}}^{\text{Rdec}}$ in session sid with (d, π) .
 - (c) If π is valid, set $S_{\text{sh}} := S_{\text{sh}} \cup \{d\}$.
 15. If $|S_{\text{sh}}| \geq \lceil \frac{|\text{grp}|}{t} \rceil$:
 - (a) Call $\mathcal{F}_{\text{MGHE}}$ in sid with S_{sh} to get message m .
 - (b) Output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$.
 16. Otherwise, abort.

Fig. 12: Protocol $\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{zkSNARK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model that is secure against a (full) malicious adversary.

message m under valid randomness ¹³ ω bounded by B_{Enc} defined by the scheme. Formally,

$$R_{\text{Enc}} = \left\{ ((\text{jpgk}, c), (m, \omega)) \mid \begin{array}{l} c = \text{MGHE.Enc}(\text{jpgk}, m; \omega) \\ \wedge -B_{\text{Enc}} \leq \omega \leq B_{\text{Enc}} \end{array} \right\}.$$

Here, the instance is the pair (jpgk, c) consisting of the joint public key and ciphertext, while the witness (m, ω) consists of the underlying plaintext and the randomness used for encryption. Intuitively, this guarantees that a party can only produce a valid proof if it knows both a message and valid randomness that open the ciphertext, thereby preventing malformed inputs.

In the computation phase, given a circuit f , we define the relation R_{Eval} to certify the correctness of homomorphic evaluation:

$$R_{\text{Eval}}^f = \left\{ ((\hat{c}, (\text{jpgk}_j, c_j)_{j \in [\ell]}), \sigma) \mid \begin{array}{l} \hat{c} = \text{MGHE.Eval}(f, (\text{jpgk}_j, c_j)_{j \in [\ell]}; \sigma) \\ \wedge -B_{\text{Eval}} \leq \sigma \leq B_{\text{Eval}} \end{array} \right\}.$$

Here, the instance consists of the evaluated ciphertext $\hat{c}$ together with the input ciphertexts and their associated joint public keys $(\text{jpgk}_j, c_j)_{j \in [\ell]}$, while the witness is the randomness σ used in the evaluation. Intuitively, this ensures that a party can only generate a valid proof if $\hat{c}$ is a faithful homomorphic evaluation of f on the given inputs, thereby preventing adversaries from injecting inconsistent or malformed computations.

In the output phase, we define the relation R_{Dec} to certify the validity of partial decryptions.¹⁴ We assume that each individual public key pk is derived from its corresponding secret key sk via the algorithm $\text{MGHE.PKGen}(\cdot)$.¹⁵ This linkage ensures that the correct secret key was used in the partial decryption. Formally,

$$R_{\text{Dec}} = \left\{ ((\text{pp}, \text{pk}, c, d), (\text{sk}, \gamma, \eta)) \mid \begin{array}{l} d = \text{MGHE.PDec}(\text{sk}, c; \eta) \\ \wedge \text{pk} = \text{MGHE.PKGen}(\text{pp}, \text{sk}; \gamma) \\ \wedge -B_{\text{Dec}} \leq \eta \leq B_{\text{Dec}} \end{array} \right\}.$$

¹³ Parties may obtain randomness ω from $\mathcal{F}_{\text{MGHE}}$ by similarly sending it an input. We treat this act as a “utility functionality”: it is not part of the core definition of $\mathcal{F}_{\text{MGHE}}$, but it supports the description of our protocol that composes with $\mathcal{F}_{\text{zkSNARK}}$ which requires the input of the randomness. For brevity, we omit it from the formal specification of $\mathcal{F}_{\text{MGHE}}$. We emphasize that the environment $\mathcal{Z}$ does not perform such requests. Otherwise an uncorrupted (dummy) party would simply output this value obtained from $\mathcal{F}_{\text{MGHE}}$, leading to trivial distinguishing. These values remain internal and are never disclosed externally as an output.

¹⁴ Using zero-knowledge arguments with R_{Dec} is one approach to enforce *robustness* (Fig. 7), as in [10]. Other methods exist, such as error-correcting techniques in LSSS [12]. Our notion of robustness is intended to cover both such approaches, as well as the semi-malicious setting where zero-knowledge arguments are not employed.

¹⁵ Equivalently, one could commit directly to the secret key sk . UC-secure commitment schemes have been studied in several models, including the CRS model [19], the ROM [37], the stateless-token model, and the PUF model [28]. Extending our analysis to incorporate such UC-secure commitments is left for future work.

Here, the instance consists of the parameters $(\text{pp}, \text{pk}, c, d)$, while the witness $(\text{sk}, \gamma, \eta)$ contains the secret key, randomness for key generation, and randomness for partial decryption. Intuitively, this relation guarantees both correctness of the partial decryption and consistency between the public and secret keys, preventing the adversary from using malformed or inconsistent partial decryptions.

Theorem 5. *The protocol $\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}$ UC-realizes $\mathcal{F}_{\text{OTF-MPC}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{zkSNARK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of a global subroutine $\mathcal{G}_{\text{KRK}}$ and a global random oracle $\mathcal{G}_{\text{RO}}$ against a full malicious adversary that corrupts a group up to the threshold and possibly the server.*

We give a full proof of Theorem 5 in Appendix F.5.

4.4 Compiling with NIZK/zkSNARG for Malicious Security

Simulation extractability is a strong requirement, and several schemes of interest are not known to satisfy it in certain idealized models [47]. In practice, however, it often suffices to guarantee merely the *existence* of a valid witness certifying that a ciphertext is well-formed. This motivates relaxing the assumption to a *simulation-sound* NIZK.¹⁶

Inspired by Camenisch et al. [15], in $\mathcal{F}_{\text{NIZK}}$ (Appendix C, Fig. 15), we define a witness as a pair (w, w^*) such that the component w is subct to knowledge soundness, while w^* is required to exist. During verification, the adversary provides w , while the functionality checks for the existence of a valid w^* with respect to the instance x . If the verifier Vfy accepts the argument but no such w^* exists, the functionality flags this as a violation of simulation soundness.

In Fig. 13, we present the protocol $\Pi_{\text{OTF-MPC}}^{\text{SS-2P}}$, which employs a *two-pass* Naor–Yung structure [55] across the input, computation, and output phases. Intuitively, duplicating the ciphertexts ensures that soundness holds even in the absence of explicit knowledge extraction: a party must produce two consistent encryptions and prove the validity of both ciphertexts, which is infeasible unless the party knows the underlying message.

In the input phase, we adopt the Naor–Yung double-encryption paradigm [55]. This structure ensures that any valid ciphertext tuple must correspond to a *known* plaintext by the party who generate the ciphertext, without relying on extractability of the accompanying argument. We formalize this via the relation

$$R_{\text{Enc}} = \left\{ \left(\begin{pmatrix} \text{jpgk}_1, c_1 \\ \text{jpgk}_2, c_2 \end{pmatrix}, (m, \omega_1, \omega_2) \right) \mid \begin{array}{l} \forall u \in \{u, 2\} : \\ c_u = \text{MGHE.Enc}(\text{jpgk}_u, m; \omega_u) \\ \wedge -B_{\text{Enc}} \leq \omega_u \leq B_{\text{Enc}} \end{array} \right\},$$

where the instance consists of the two joint public keys $\text{jpgk}_1, \text{jpgk}_2$ and the corresponding ciphertexts c_1, c_2 , and the witness includes the plaintext m and the random coins ω_1, ω_2 .

¹⁶ We write $\mathcal{F}_{\text{NIZK}}$ to stress that it is not an argument of knowledge. For efficiency, one may instantiate it with a succinct non-interactive argument (SNARG) that also provides zero knowledge.

Protocol: $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\text{srv}}$.

- **Upload** : $[\text{cli}, \text{grp}, \text{sid}, \text{uid}, m]$ to $\mathcal{P}_{\text{cli}}$:
 1. Call $\mathcal{G}_{\text{KRR}}$ in (sid, 1) and (sid, 2) with grp to get jpg_1 and jpg_2 .
 2. Call $\mathcal{F}_{\text{MGHE}}$ in (sid, 1) and (sid, 2) with m to get ciphertexts c_1 and c_2 .
 3. Call $\mathcal{F}_{\text{MGHE}}$ in (sid, 1) and (sid, 2) to get randomness ω_1 and ω_2 .
 4. Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ in sid with instance $(\text{jpg}_1, \text{jpg}_2, c_1, c_2)$ and witness (m, ω_1, ω_2) to get π .
 5. Output $\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c_1, c_2, \pi)]\}$ to $\mathcal{F}_{\text{AUTH}}$.
- **Evaluate** : $[\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}]$ to $\mathcal{P}_{\text{srv}}$:
 1. If $\text{sid} \neq \text{sid}_j$ for any $j \in [\ell]$, abort.
 2. For $j \in [\ell]$:
 - (a) Get the message msg under label $(\text{grp}_j, \text{sid}_j, \text{uid}_j)$ from $\mathcal{F}_{\text{AUTH}}$
 - (b) If $\text{msg} = \{\text{"up"} : [\text{cli}, \text{grp}_j, \text{sid}_j, \text{uid}_j, (c_{j,1}, c_{j,2}, \pi_j)]\}$:
 - (i) Call $\mathcal{G}_{\text{KRR}}$ in (sid, 1) and (sid, 2) to get $\text{jpg}_{j,1}$ and $\text{jpg}_{j,2}$ of grp_j .
 - (ii) Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ to verify π_j with instance $(\text{jpg}_{j,1}, \text{jpg}_{j,2}, c_{j,1}, c_{j,2})$. Abort if verification fails.
 - (c) Else if $\text{msg} = \{\text{"eval"} : [\text{srv}, \text{grp}_j, \text{sid}_j, \text{uid}_j, (\text{grp}_{j'}, \text{sid}_{j'}, \text{uid}_{j'})_{j' \in [\ell]}, (c_{j,1}, c_{j,2}, \pi_j)]\}$:
 - (i) Call $\mathcal{G}_{\text{KRR}}$ in (sid, 1) and (sid, 2) to get $\text{jpg}_{j',1}$ and $\text{jpg}_{j',2}$ of $\text{grp}_{j'}$ for $j' \in [\ell]$.
 - (ii) Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ to verify π_j with instance $((\text{jpg}_{j',1}, \text{jpg}_{j',2}, c_{j',1}, c_{j',2})_{j' \in [\ell]}, c_{j,1}, c_{j,2})$.
 - (iii) Abort if verification fails or any of $c_{j',1}$ or $c_{j',2}$ for $j' \in [\ell]$ is invalid.
 3. Set $\text{grp} = \cup_{i \in [\ell]} \text{grp}_i$.
 4. For $k \in \{1, 2\}$: Call $\mathcal{F}_{\text{MGHE}}$ in (sid, k) with f and $(\text{jpg}_{j,k}, c_{j,k})$ to get $\hat{c}_k$.
 5. Call $\mathcal{F}_{\text{MGHE}}$ in (sid, 1) and (sid, 2) to get randomness σ_1 and σ_2 .
 6. Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ in sid with $((\text{jpg}_{j,1}, c_{j,1}, \text{jpg}_{j,2}, c_{j,2})_{j \in [\ell]}, \hat{c}_1, \hat{c}_2)$ and (σ_1, σ_2) to get π .
 7. Output $\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, (\hat{c}_1, \hat{c}_2, \pi)]\}$ to $\mathcal{F}_{\text{AUTH}}$.
- **Retrieve** : $[\text{cli}, \text{grp}, \text{sid}, \text{uid}]$ to $\mathcal{P}_{\text{cli}}$:
 1. If $\text{cli} \notin \text{grp}$, abort.
 2. Get the message msg under label $(\text{grp}, \text{sid}, \text{uid})$ from $\mathcal{F}_{\text{AUTH}}$
 3. Abort if $\text{msg} = \{\text{"up"} : [\text{cli}', \text{grp}, \text{sid}, \text{uid}, (c, \pi)]\}$.
// Partially decrypt evaluated ciphertext only.
 4. Parse msg as $\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, (\hat{c}_1, \hat{c}_2, \pi)]\}$.
 5. Call $\mathcal{G}_{\text{KRR}}$ in (sid, 1) and (sid, 2) to get $\text{jpg}_{j,1}$ and $\text{jpg}_{j,2}$ of grp_j for $j \in [\ell]$.
 6. Call $\mathcal{F}_{\text{NIZK}}^{\text{Renc}}$ in session sid to verify π with instance $((\text{jpg}_{j,1}, \text{jpg}_{j,2}, c_{j,1}, c_{j,2})_{j \in [\ell]}, \hat{c}_1, \hat{c}_2)$.
 7. Abort if verification fails, or any input ciphertexts $(c_{j,1}, c_{j,2}, \pi_j)$ is invalid.
 8. Call $\mathcal{G}_{\text{KRR}}$ in (sid, 1) and (sid, 2) with cli to get the secret key sk_1 and sk_2 .
 9. Call $\mathcal{F}_{\text{MGHE}}$ in session (sid, 1) and (sid, 2) to obtain partial decryptions d_1 and d_2 .
 10. Call $\mathcal{F}_{\text{MGHE}}$ in session (sid, 1) and (sid, 2) to obtain the randomness η_1 and η_2 .
 11. Call $\mathcal{F}_{\text{NIZK}}^{\text{Rdec}}$ in session sid with $(\hat{c}_1, \hat{c}_2, d_1, d_2)$ and $(\text{sk}_1, \text{sk}_2, \eta_1, \eta_2)$ to get π .
 12. Output $\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, d_1, d_2, \pi]\}$ to $\mathcal{F}_{\text{AUTH}}$.
 13. Set $\mathcal{S}_{\text{sh}_1} := \emptyset$ and $\mathcal{S}_{\text{sh}_2} := \emptyset$.
 14. For $\text{cli}' \in \text{grp} \setminus \{\text{cli}\}$ do:
 - (a) Receive message $\{\text{"sh"} : [\text{cli}', \text{grp}, \text{sid}, \text{uid}, d_1, d_2, \pi]\}$ from $\mathcal{F}_{\text{AUTH}}$.
 - (b) Call $\mathcal{F}_{\text{NIZK}}^{\text{Rdec}}$ in session sid with (d_1, d_2, π) .
 - (c) If π is valid, set $\mathcal{S}_{\text{sh}_1} := \mathcal{S}_{\text{sh}_1} \cup \{d_1\}$ and $\mathcal{S}_{\text{sh}_2} := \mathcal{S}_{\text{sh}_2} \cup \{d_2\}$.
 15. If $|\mathcal{S}_{\text{sh}_1}| < \lceil \frac{|\text{grp}|}{t} \rceil$, abort. Otherwise:
 - (a) For $u \in \{1, 2\}$: Call $\mathcal{F}_{\text{MGHE}}$ in session (sid, u) with $\mathcal{S}_{\text{sh}_u}$ to obtain a message m_u .
 - (b) If $m_1 \neq m_2$ $|m_1 - m_2| \leq \text{Estimate}(\cdot)$, output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$, and abort otherwise.

Fig. 13: Protocol $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$ in the $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model, secure against a (full) malicious adversary. Framed parts appear exclusively when MGHE is exact and perfectly correct. Highlighted parts appear only when MGHE is approximate or imperfectly correct.

In the computation phase, we define the relation R_{Eval}^f to certify that the evaluation of f is faithful. We perform a double evaluation on the input ciphertexts to preserve the security guarantees of Naor–Yung double encryption. While knowledge of witness is not strictly required here since the existence of a valid evaluation random coin for correct evaluation would suffice, the double evaluation is necessary to prevent an adversary from tampering with either of ciphertext in Naor–Yung pair through homomorphic operations. This ensures that the integrity enforced by the Naor–Yung structure carries through the evaluation phase. Formally, we define:

$$R_{\text{Eval}}^f = \left\{ \left(\left(\begin{pmatrix} \hat{c}_1, (\text{jp}k_{j,1}, c_{j,1})_{j \in [\ell]} \\ \hat{c}_2, (\text{jp}k_{j,2}, c_{j,2})_{j \in [\ell]} \end{pmatrix}, \begin{pmatrix} \sigma_1 \\ \sigma_2 \end{pmatrix} \right) \mid \begin{array}{l} \forall u \in \{1, 2\} : \\ \hat{c}_u = \text{MGHE.Eval}(f, (\text{jp}k_{j,u}, c_{j,u})_{j \in [\ell]}; \sigma_u) \\ \wedge -B_{\text{Eval}} \leq \sigma_u \leq B_{\text{Eval}} \end{array} \right) \right\}.$$

Here, the instance contains the evaluated ciphertexts and the input ciphertexts under the two sets of joint public keys, and the witness consists of the corresponding evaluation randomness.

Analogously, in the output phase, we employ a double (partial) decryption strategy to certify that each partial decryption is generated honestly. Formally, we define the relation

$$R_{\text{Dec}} = \left\{ \left(\left(\begin{pmatrix} \text{pp}_1, \text{pk}_1, c_1, d_1 \\ \text{pp}_2, \text{pk}_2, c_2, d_2 \end{pmatrix}, \begin{pmatrix} \text{sk}_1, \gamma_1, \eta_1 \\ \text{sk}_2, \gamma_2, \eta_2 \end{pmatrix} \right) \mid \begin{array}{l} \forall u \in \{1, 2\} : \\ d_u = \text{MGHE.PDec}(\text{sk}_u, c_u; \eta_u) \\ \wedge \text{pk}_u = \text{PKGen}(\text{pp}_u, \text{sk}_u; \gamma_u) \\ \wedge -B_{\text{Dec}} \leq \eta_u \leq B_{\text{Dec}} \end{array} \right) \right\}.$$

The instance consists of the public parameters, ciphertexts, and partial decryptions, and the witness contains the secret keys and randomness. After collecting sufficient partial decryptions, the client obtains two decrypted messages m_1 and m_2 . It then checks whether $m_1 = m_2$. If the messages match, the client outputs either one; otherwise, it aborts the protocol. In case of approximate HE, we let the client check if the difference between m_1 and m_2 are within the “range of acceptance” defined by a $\text{Estimate}(\cdot)$ function as in the *robustness* (Rob) game (Fig. 7). If yes, it can accept either of it; otherwise it aborts the protocol.

Theorem 6. *The protocol $\Pi_{\text{OTF-MPC}}^{\text{SS-2P}}$ UC-realizes $\mathcal{F}_{\text{OTF-MPC}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of a global subroutine $\mathcal{G}_{\text{KRK}}$ and a global random oracle $\mathcal{G}_{\text{RO}}$ against a full malicious adversary that corrupts a group up to the threshold and possibly the server.*

We give a full proof of Theorem 6 in Appendix F.6.

References

1. Akavia, A., Vald, M.: On the privacy of protocols based on CPA-secure homomorphic encryption. Cryptology ePrint Archive, Report 2021/803 (2021), <https://eprint.iacr.org/2021/803>
2. Aloufi, A., Hu, P.: Collaborative homomorphic computation on data encrypted under multiple keys. arXiv preprint arXiv:1911.04101 (2019)
3. Asharov, G., Jain, A., López-Alt, A., Tromer, E., Vaikuntanathan, V., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold FHE. In: Pointcheval, D., Johansson, T. (eds.) EUROCRYPT 2012. LNCS, vol. 7237, pp. 483–501. Springer, Berlin, Heidelberg (Apr 2012). https://doi.org/10.1007/978-3-642-29011-4_29
4. Atapoor, S., Baghery, K., Pereira, H.V.L., Spiessens, J.: Verifiable FHE via lattice-based SNARKs. CiC 1(1), 24 (2024). <https://doi.org/10.62056/a6ksdkp10>
5. Badertscher, C., Canetti, R., Hesse, J., Tackmann, B., Zikas, V.: Universal composition with global subroutines: Capturing global setup within plain UC. In: Pass, R., Pietrzak, K. (eds.) TCC 2020, Part III. LNCS, vol. 12552, pp. 1–30. Springer, Cham (Nov 2020). https://doi.org/10.1007/978-3-030-64381-2_1
6. Barak, B., Canetti, R., Nielsen, J.B., Pass, R.: Universally composable protocols with relaxed set-up assumptions. In: 45th FOCS. pp. 186–195. IEEE Computer Society Press (Oct 2004). <https://doi.org/10.1109/FOCS.2004.71>
7. Bergamaschi, F., Costache, A., Dachman-Soled, D., Kippen, H., LaBuff, L., Tang, R.: Revisiting the security of approximate FHE with noise-flooding countermeasures. pp. 102–134. LNCS, Springer, Cham (May 2025). https://doi.org/10.1007/978-3-031-91832-2_4
8. Bernard, O., Joye, M., Smart, N.P., Walter, M.: Drifting towards better error probabilities in fully homomorphic encryption schemes. pp. 181–211. LNCS, Springer, Cham (Jun 2025). https://doi.org/10.1007/978-3-031-91101-9_7
9. Boneh, D., Boyen, X., Halevi, S.: Chosen ciphertext secure public key threshold encryption without random oracles. In: Pointcheval, D. (ed.) CT-RSA 2006. LNCS, vol. 3860, pp. 226–243. Springer, Berlin, Heidelberg (Feb 2006). https://doi.org/10.1007/11605805_15
10. Boneh, D., Gennaro, R., Goldfeder, S., Jain, A., Kim, S., Rasmussen, P.M.R., Sahai, A.: Threshold cryptosystems from threshold fully homomorphic encryption. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part I. LNCS, vol. 10991, pp. 565–596. Springer, Cham (Aug 2018). https://doi.org/10.1007/978-3-319-96884-1_19
11. Boneh, D., Segev, G., Waters, B.: Targeted malleability: homomorphic encryption for restricted computations. In: Goldwasser, S. (ed.) ITCS 2012. pp. 350–366. ACM (Jan 2012). <https://doi.org/10.1145/2090236.2090264>
12. Boudgoust, K., Scholl, P.: Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part I. LNCS, vol. 14438, pp. 371–404. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8721-4_12
13. Bourse, F., del Pino, R., Minelli, M., Wee, H.: FHE circuit privacy almost for free. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 62–89. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_3
14. Camenisch, J., Drijvers, M., Gagliardoni, T., Lehmann, A., Neven, G.: The wonderful world of global random oracles. In: Nielsen, J.B., Rijmen, V. (eds.) EURO-

- CRYPT 2018, Part I. LNCS, vol. 10820, pp. 280–312. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78381-9_11
15. Camenisch, J., Krenn, S., Shoup, V.: A framework for practical universally composable zero-knowledge protocols. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 449–467. Springer, Berlin, Heidelberg (Dec 2011). https://doi.org/10.1007/978-3-642-25385-0_24
 16. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. Cryptology ePrint Archive, Report 2000/067 (2000), <https://eprint.iacr.org/2000/067>
 17. Canetti, R.: Universally composable security. J. ACM **67**(5) (Sep 2020). <https://doi.org/10.1145/3402457>, <https://doi.org/10.1145/3402457>
 18. Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally composable security with global setup. In: Vadhan, S.P. (ed.) TCC 2007. LNCS, vol. 4392, pp. 61–85. Springer, Berlin, Heidelberg (Feb 2007). https://doi.org/10.1007/978-3-540-70936-7_4
 19. Canetti, R., Fischlin, M.: Universally composable commitments. In: Kilian, J. (ed.) CRYPTO 2001. LNCS, vol. 2139, pp. 19–40. Springer, Berlin, Heidelberg (Aug 2001). https://doi.org/10.1007/3-540-44647-8_2
 20. Canetti, R., Raghuraman, S., Richelson, S., Vaikuntanathan, V.: Chosen-ciphertext secure fully homomorphic encryption. In: Fehr, S. (ed.) PKC 2017, Part II. LNCS, vol. 10175, pp. 213–240. Springer, Berlin, Heidelberg (Mar 2017). https://doi.org/10.1007/978-3-662-54388-7_8
 21. Chatel, S., Mouchet, C., Sahin, A.U., Pyrgelis, A., Troncoso, C., Hubaux, J.P.: PELTA - shielding multiparty-FHE against malicious adversaries. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 711–725. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3623139>
 22. Checri, M., Sirdey, R., Boudguiga, A., Bultel, J.P.: On the practical CPAD security of “exact” and threshold FHE schemes and libraries. Cryptology ePrint Archive, Report 2024/116 (2024), <https://eprint.iacr.org/2024/116>
 23. Cheon, J.H., Choe, H., Kim, S., Yeo, Y.: Reusable dynamic multi-party homomorphic encryption. Cryptology ePrint Archive, Report 2025/581 (2025), <https://eprint.iacr.org/2025/581>
 24. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks against the IND-CPA^D security of exact FHE schemes. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 2505–2519. ACM Press (Oct 2024). <https://doi.org/10.1145/3658644.3690341>
 25. Chiesa, A., Fenzi, G.: zkSNARKs in the ROM with unconditional UC-security. In: Boyle, E., Mahmoody, M. (eds.) TCC 2024, Part I. LNCS, vol. 15364, pp. 67–89. Springer, Cham (Dec 2024). https://doi.org/10.1007/978-3-031-78011-0_3
 26. Chiesa, A., Yogev, E.: Building Cryptographic Proofs from Hash Functions (2024), <https://github.com/hash-based-snargs-book>
 27. Cramer, R., Damgård, I.: Multiparty Computation, an Introduction, pp. 41–87. Birkhäuser Basel, Basel (2005). https://doi.org/10.1007/3-7643-7394-6_2, https://doi.org/10.1007/3-7643-7394-6_2
 28. Damgård, I., Scafuro, A.: Unconditionally secure and universally composable commitments from physical assumptions. In: Sako, K., Sarkar, P. (eds.) ASIACRYPT 2013, Part II. LNCS, vol. 8270, pp. 100–119. Springer, Berlin, Heidelberg (Dec 2013). https://doi.org/10.1007/978-3-642-42045-0_6
 29. Dodis, Y., Haralambiev, K., López-Alt, A., Wichs, D.: Efficient public-key cryptography in the presence of key leakage. In: Abe, M. (ed.) ASIACRYPT 2010. LNCS, vol. 6477, pp. 613–631. Springer, Berlin, Heidelberg (Dec 2010). https://doi.org/10.1007/978-3-642-17373-8_35

30. Ducas, L., Stehlé, D.: Sanitization of FHE ciphertexts. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part I. LNCS, vol. 9665, pp. 294–310. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49890-3_12
31. Feigenbaum, J., Ishai, Y., Malkin, T., Nissim, K., Strauss, M., Wright, R.N.: Secure multiparty computation of approximations. In: Orejas, F., Spirakis, P.G., van Leeuwen, J. (eds.) ICALP 2001. LNCS, vol. 2076, pp. 927–938. Springer, Berlin, Heidelberg (Jul 2001). https://doi.org/10.1007/3-540-48224-5_75
32. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953 (2019), <https://eprint.iacr.org/2019/953>
33. Gentry, C.: A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University (2009), crypto.stanford.edu/craig
34. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Mitzenmacher, M. (ed.) 41st ACM STOC. pp. 169–178. ACM Press (May / Jun 2009). <https://doi.org/10.1145/1536414.1536440>
35. Groth, J., Ostrovsky, R., Sahai, A.: Perfect non-interactive zero knowledge for NP. In: Vaudenay, S. (ed.) EUROCRYPT 2006. LNCS, vol. 4004, pp. 339–358. Springer, Berlin, Heidelberg (May / Jun 2006). https://doi.org/10.1007/11761679_21
36. Guo, Q., Nabokov, D., Suvanto, E., Johansson, T.: Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. In: Balzarotti, D., Xu, W. (eds.) USENIX Security 2024. USENIX Association (Aug 2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/guo-qian>
37. Hofheinz, D., Müller-Quade, J.: Universally composable commitments using random oracles. In: Naor, M. (ed.) TCC 2004. LNCS, vol. 2951, pp. 58–76. Springer, Berlin, Heidelberg (Feb 2004). https://doi.org/10.1007/978-3-540-24638-1_4
38. Hwang, I., Hwang, Y., Kim, M., Lee, D., Song, Y.: Provably secure approximate computation protocols from CKKS. Cryptology ePrint Archive, Report 2025/395 (2025), <https://eprint.iacr.org/2025/395>
39. Hwang, I., Min, S., Song, Y.: Ciphertext-simulatable HE from BFV with randomized evaluation. Cryptology ePrint Archive, Paper 2025/203 (2025), <https://eprint.iacr.org/2025/203>
40. Ishai, Y., Paskin, A.: Evaluating branching programs on encrypted data. In: Vadhan, S.P. (ed.) TCC 2007. LNCS, vol. 4392, pp. 575–594. Springer, Berlin, Heidelberg (Feb 2007). https://doi.org/10.1007/978-3-540-70936-7_31
41. Jakobsen, T.P., Nielsen, J.B., Orlandi, C.: A framework for outsourcing of secure computation. Cryptology ePrint Archive, Report 2016/037 (2016), <https://eprint.iacr.org/2016/037>
42. Klucznik, K., Santato, G.: On circuit private, multikey and threshold approximate homomorphic encryption. CiC **2**(1), 9 (2025). <https://doi.org/10.62056/a69qgy17s>
43. Knabenhans, C., Viand, A., Merino-Gallardo, A., Hithnawi, A.: VFHE: Verifiable fully homomorphic encryption. In: 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC) (2024)
44. Kwak, H., Lee, D., Song, Y., Wagh, S.: A general framework of homomorphic encryption for multiple parties with non-interactive key-aggregation. In: Pöpper, C., Batina, L. (eds.) ACNS 24 International Conference on Applied Cryptography and Network Security, Part II. LNCS, vol. 14584, pp. 403–430. Springer, Cham (Mar 2024). https://doi.org/10.1007/978-3-031-54773-7_16

45. Li, B., Micciancio, D.: On the security of homomorphic encryption on approximate numbers. In: Canteaut, A., Standaert, F.X. (eds.) EUROCRYPT 2021, Part I. LNCS, vol. 12696, pp. 648–677. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_23
46. Li, B., Micciancio, D., Schultz, M., Sorrell, J.: Securing approximate homomorphic encryption using differential privacy. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part I. LNCS, vol. 13507, pp. 560–589. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15802-5_20
47. Lipmaa, H., Parisella, R., Siim, J.: Algebraic group model with oblivious sampling. In: Rothblum, G.N., Wee, H. (eds.) TCC 2023, Part IV. LNCS, vol. 14372, pp. 363–392. Springer, Cham (Nov / Dec 2023). https://doi.org/10.1007/978-3-031-48624-1_14
48. Liu, Z., Wang, Y., Fisch, B.: IND-CPA-d of relaxed functional bootstrapping: A new attack, a general fix, and a stronger model. Cryptology ePrint Archive, Paper 2025/1627 (2025), <https://eprint.iacr.org/2025/1627>
49. López-Alt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: Karloff, H.J., Pitassi, T. (eds.) 44th ACM STOC. pp. 1219–1234. ACM Press (May 2012). <https://doi.org/10.1145/2213977.2214086>
50. Lysyanskaya, A., Rosenbloom, L.N.: Efficient and universally composable non-interactive zero-knowledge proofs of knowledge with security against adaptive corruptions. Cryptology ePrint Archive, Report 2022/1484 (2022), <https://eprint.iacr.org/2022/1484>
51. Manulis, M., Nguyen, J.: Fully homomorphic encryption beyond IND-CCA1 security: Integrity through verifiability. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, vol. 14652, pp. 63–93. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58723-8_3
52. Mouchet, C., Troncoso-Pastoriza, J., Bossuat, J.P., Hubaux, J.P.: Multiparty homomorphic encryption from ring-learning-with-errors. Proceedings on Privacy Enhancing Technologies **2021**(4), 291–311 (2021)
53. Mouchet, C., Troncoso-Pastoriza, J.R., Bossuat, J.P., Hubaux, J.P.: Multiparty homomorphic encryption from ring-learning-with-errors. PoPETs **2021**(4), 291–311 (Oct 2021). <https://doi.org/10.2478/popets-2021-0071>
54. Mukherjee, P., Wichs, D.: Two round multiparty computation via multi-key FHE. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 735–763. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_26
55. Naor, M., Yung, M.: Public-key cryptosystems provably secure against chosen ciphertext attacks. In: 22nd ACM STOC. pp. 427–437. ACM Press (May 1990). <https://doi.org/10.1145/100216.100273>
56. Park, J.: Homomorphic encryption for multiple users with less communications. Cryptology ePrint Archive, Report 2021/1085 (2021), <https://eprint.iacr.org/2021/1085>
57. Passelègue, A., Stehlé, D.: Low communication threshold fully homomorphic encryption. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part I. LNCS, vol. 15484, pp. 297–329. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0875-1_10
58. Sav, S., Bossuat, J.P., Troncoso-Pastoriza, J.R., Claassen, M., Hubaux, J.P.: Privacy-preserving federated neural network learning for disease-associated cell classification. Patterns **3**(5) (2022)

59. Schoenmakers, B., Veeningen, M., de Vreede, N.: Trinocchio: Privacy-preserving outsourcing by distributed verifiable computation. In: Manulis, M., Sadeghi, A.R., Schneider, S. (eds.) ACNS 16 International Conference on Applied Cryptography and Network Security. LNCS, vol. 9696, pp. 346–366. Springer, Cham (Jun 2016). https://doi.org/10.1007/978-3-319-39555-5_19
60. Thangam, V., Chandrasekaran, K.: Elliptic curve based secure outsourced computation in multi-party cloud environment. In: Security in Computing and Communications: 4th International Symposium, SSCC 2016, Jaipur, India, September 21-24, 2016, Proceedings 4. pp. 199–212. Springer (2016)
61. Viand, A.: Useable Fully Homomorphic Encryption. Ph.D. thesis, ETH Zurich (2023)
62. Xu, G., Han, X., Xu, S., Zhang, T., Li, H., Huang, X., Deng, R.H.: Hercules: Boosting the performance of privacy-preserving federated learning. *IEEE Transactions on Dependable and Secure Computing* **20**(5), 4418–4433 (2022)

A Related Work

HE-based MPC. López-Alt et al. [49] initiated the study on on-the-fly MPC by using multi-key HE that enables the computation on data encrypted under different keys. They show that their construction is secure against a *semi-malicious* adversary who is allowed to use arbitrarily chosen random coins during the execution of the protocol if the underlying MKHE scheme satisfies *semantic security*. Later, Mukherjee and Wichs [54] extended the analysis by considering the setting involving *partial decryption* which is generated by each participant using their secret key. These partial decryptions can then be combined to reconstruct the message without requiring every participant to be online at the same time, thereby improving practicality in distributed settings. To improve the efficiency of MKHE-based MPC, Mouchet et al. [53] proposed an MPC protocol by adding the dynamic inclusion property of MKHE to threshold HE through a *key-switching* operation. This technique is later generalized to the definition of *multi-group HE* defined by Kwak et al. [44] by clustering participants into groups. More recently, Cheon et al. [23] further improved the efficiency by making the ciphertext size independent of the number of participants. Still, we note that the conclusion from these later works is that, with HE alone, one can construct an MPC protocol that is secure against a semi-malicious or a semi-honest adversary (also known as honest-but-curious adversary), without further discussing the security against a fully malicious adversary. Additionally, these works typically focus on the privacy of inputs provided by the clients and assume that the server evaluating the circuit is corrupted by default without further discussing the privacy of the inputs provided by the server.

Composition of HE and Zero-Knowledge Arguments. The composition of (zk)SNARKs or NIZKs with HE has been studied in the literature multiple times, primarily to achieve *chosen-ciphertext* (CCA) security for HE. Boneh et al. [11] employ the Naor–Yung paradigm [55] using a *two-pass* structure: the same message is encrypted twice under independent randomness, and a NIZK proof is appended to attest to the consistency of the two ciphertexts. The evaluation similarly follows a two-pass structure, where the evaluation algorithm is applied separately to each component of the ciphertext pair from Naor–Yung paradigm, with a SNARG proof attesting to the correctness of the evaluation. Later, Canetti et al. [20] proposed a similar construction based on the Naor–Yung approach, but replaced the NIZK with a zkSNARK to improve efficiency. Both works prove that their transforms yield CCA1-secure HE schemes. Viand et al. [61,43] introduced the notion of *verifiable FHE*, where a CPA-secure FHE scheme is combined with a SNARK that proves only the correctness of the evaluation. However, their construction offers weaker guarantees than standard CCA1 security, as it does not prevent malleability of ciphertexts prior to evaluation. More recently, Manulis and Nguyen [51] revisited the Naor–Yung-based approach [11,20], but used a *single-pass* evaluation strategy instead of two. They showed that this approach achieves the vCCA security notion, which is stronger than CCA1 but slightly weaker than full CCA2. Atapoor et al. [4] proposed a verifiable HE construction

using lattice-based SNARKs. Although they do not provide a formal proof of security, they argue that their scheme inherits security from the CPA-security of the underlying HE and the knowledge soundness of the SNARK.

B Universal Composability (UC) with Global Subroutines

In this work, we prove security using the *Universal Composability* (UC) framework introduced by Canetti [16]. The main advantage of the UC framework is that protocols are guaranteed to maintain their security in any context, even in the presence of arbitrarily many protocol sessions that run concurrently in an adversarially controlled manner.

B.1 System of ITMs

The UC framework is a *simulation-based* framework where security is defined in terms of a comparison between a “real world” versus an “ideal world”. In the real world, a multi-party protocol Π is executed among *parties* $\mathcal{P}_1, \dots, \mathcal{P}_n$ who are modeled as *Interactive Turing Machines* (ITM). An adversary $\mathcal{A}$, who is also modeled as an ITM, carries attacks on the protocol Π . The adversary can corrupt parties, gaining access to their current and past states as well as the contents of all their tapes. Once a party is corrupted, the adversary gains full control over its future actions.

In the ideal world, a trusted *functionality* $\mathcal{F}$, modeled as an ITM and representing the desired security property, interacts with (dummy) parties $\mathcal{P}_1, \dots, \mathcal{P}_n$. These dummy parties can only forward inputs to them to $\mathcal{F}$ and outputs responses from $\mathcal{F}$ to them as their outputs. The protocol in which $\mathcal{F}$ interacts with the dummy parties is referred as an *ideal protocol* $\mathcal{I}_{\mathcal{F}}$. Additionally, there is an *ideal-process adversary* $\mathcal{S}$, also commonly called the *simulator*. The simulator’s capabilities are restricted to corrupting parties, delaying messages, or suppressing messages sent from $\mathcal{F}$ to a party.

An *environment* $\mathcal{Z}$, who is sometimes referred as a *distinguisher* and also modeled as an ITM, chooses protocol inputs to all parties and may read all outputs locally made by the parties and communicate with the adversary. Informally, we say a protocol is UC-secure if there is no environment that can distinguish between whether it is interacting with the real adversary $\mathcal{A}$ or the ideal-process adversary $\mathcal{S}$.

B.2 Global Subroutines

For many protocols, a global setup is needed, for example, a common reference string (CRS) or a random oracle. For this purpose, Badertscher et al. [5] proposed UC with *global subroutine* (UCGS) by using a transformation named *management protocol* [5, Section 3.1] to allow plain UC framework capture the global setup. A management protocol $\text{Mgmt}[\Pi, \mathcal{G}]$ for a protocol Π and a global subroutine $\mathcal{G}$ is a UC-protocol, such that one instance of $\text{Mgmt}[\Pi, \mathcal{G}]$ behaves like a single

instance of Π along with one or more instances of $\mathcal{G}$. Specifically, the incoming communication to $\text{Mgmt}[\Pi, \mathcal{G}]$ specifies a session sid , plus a session identifier for either Π or a specific instance of $\mathcal{G}$. The input is then passed to Π or the respective instance of $\mathcal{G}$.

UCGS can actually be viewed as a hybrid approach between basic UC and *generalized* UC [18]. Namely, the environment $\mathcal{Z}$ is allowed to invoke a single instance of the challenge protocol Π while it is allowed to invoke arbitrarily many instances of the global subroutine $\mathcal{G}$. This also allows us better flexibility when defining the global subroutine since we no longer need to fix the global functionality for the lifetime of a party.

B.3 UC Emulation and Realization

Informally, a protocol Π is said to *UC-emulate* another protocol Ψ in the UC framework if, for any adversary $\mathcal{A}$ attacking it, there exists a simulator $\mathcal{S}$ “attacking” Ψ , such that no *environment* $\mathcal{Z}$ can distinguish between these two executions. Note that we can consider two types of protocols. In the first case, the communication between the parties in this protocol is over an insecure channel. We also refer this as a *bare* model. In the second case, the protocol runs (multiple sessions of) an ideal functionality $\mathcal{F}$ as a subroutine. We refer such a protocol as a *$\mathcal{F}$ -hybrid protocol*. For the completeness, in Definitions 7 and 8, we consider a $\mathcal{F}$ -hybrid model. Essentially, a bare model can be considered as a $\mathcal{F}$ -hybrid model with $\mathcal{F}$ that simply forwards the message.

Definition 7 (UC-Emulation with Global Subroutine [5, Definition 3.1]).

Let Π, Ψ be protocols and let $\mathcal{G}$ be a global subroutine. Let Mgmt be a management protocol. If $\text{Mgmt}[\Pi, \mathcal{G}]$ UC-emulates $\text{Mgmt}[\Psi, \mathcal{G}]$, then we say that Π UC-emulates Ψ in presence of a global subroutine $\mathcal{G}$. We use the notation

$$\Delta_{\mathcal{Z}} \left(\{ \text{EXEC}_{\mathcal{F}, \Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}}(\mathbf{x}, \lambda, z) \}_{\mathbf{x}, \lambda, z}; \{ \text{EXEC}_{\Psi, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}}(\mathbf{x}, \lambda, z) \}_{\mathbf{x}, \lambda, z} \right) \leq \varepsilon(\lambda)$$

to denote the advantage of the environment in distinguishing between these two execution in presence of a global subroutine $\mathcal{G}$.

However, the emulation of a protocol alone may not adequately capture its security. For instance, by trivially setting $\Pi := \Psi$ and $\mathcal{S} := \mathcal{A}$, the environment $\mathcal{Z}$ is unable to distinguish between the two executions but it does not imply any security. Thus it is essential to consider the *ideal protocol* $\text{I}_{\mathcal{F}}$. We say that a protocol Π *UC-realizes* a functionality $\mathcal{F}$ if the protocol Π UC-emulates the ideal protocol $\text{I}_{\mathcal{F}}$. In Definition 8, we formally define UC-realization for an ideal functionality $\mathcal{F}$.

Definition 8 (UC-realization with Global Subroutine). Let Π be protocols. Let $\mathcal{F}$ be an ideal functionality. Let $\mathcal{G}$ be a global subroutine. Let Mgmt be a management protocol. If $\text{Mgmt}[\Pi, \mathcal{G}]$ UC-emulates $\text{Mgmt}[\text{I}_{\mathcal{F}}, \mathcal{G}]$ where $\text{I}_{\mathcal{F}}$ is the

ideal protocol of $\mathcal{F}$, then we say that Π UC-realizes $\mathcal{F}$ in presence of a global subroutine $\mathcal{G}$. We use the notation

$$\Delta_{\mathcal{Z}} \left(\{\mathbf{REAL}_{\mathcal{F}, \Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}}(\mathbf{x}, \lambda, z)\}_{\mathbf{x}, \lambda, z}; \{\mathbf{IDEAL}_{\mathcal{F}', \mathcal{S}, \mathcal{Z}}^{\mathcal{G}}(\mathbf{x}, \lambda, z)\}_{\mathbf{x}, \lambda, z} \right) \leq \varepsilon(\lambda)$$

to denote the advantage of the environment $\mathcal{Z}$ in distinguishing between these two executions in presence of a global subroutine $\mathcal{G}$.

B.4 UC Composition Theorem

The UC framework allows us to show the security when multiple protocols are composed for another protocol. We let the protocol Φ be a $\mathcal{F}$ -hybrid protocol and let Π be a protocol that UC-realizes an ideal functionality $\mathcal{F}$. The intuition is that we can then construct a *composed protocol* $\Phi^{\mathcal{F} \rightarrow \Pi}$ by changing Φ 's calls to a session of $\mathcal{F}$ to a session of Π . At a high level, the composition theorem states that running the protocol $\Phi^{\mathcal{F} \rightarrow \Pi}$ is *at least as secure* as running the original protocol Φ who invokes the ideal functionality $\mathcal{F}$.

We describe some properties of the protocols described by Canetti [16] for the composition theorem to hold. Informally, we say that a protocol Φ is (Π, Ψ) -compliant¹⁷ [16, Section 6.1] if it does not invoke both protocol Π and Ψ with the same session identifier. We say a protocol is *subroutine-respecting* [17, Definition 19] if the subprotocols in it communicate only with parties outside their session through the main protocol. We say a protocol is *subroutine-exposing* [17, Definition 21] if the adversary can obtain a list of subroutines invoked by the protocol.

For composition theorem involving global subroutines, we need two additional properties of the protocols. We say that a protocol Π is $\mathcal{G}$ -subroutine-respecting if its subprotocols are allowed to pass and receive output from $\mathcal{G}$. We say that $\mathcal{G}$ is Π -regular if $\mathcal{G}$ is not allowed to spawn new ITMs and from using Π as a subroutine. Note in Corollary 2, we directly state the theorem for UC-realization since it is more needed than emulation.

Corollary 2 (UC Composition with Global Subroutine [5, Theorem 3.5]). *Let $\mathcal{G}$ be a global subroutine. Let Mgmt be a management protocol. Let Φ , Ψ and Π be protocols. Assume that*

1. Φ is subroutine-exposing, (Π, Ψ) -compliant, $(\Pi, \text{Mgmt}[\Pi, \mathcal{G}])$ -compliant, and $(\Pi, \text{Mgmt}[\Psi, \mathcal{G}])$ -compliant.
2. $\mathcal{G}$ is subroutine-respecting and Π -regular.
3. Π and Ψ are $\mathcal{G}$ -subroutine-respecting.

If $\text{Mgmt}[\Pi, \mathcal{G}]$ UC-emulates $\text{Mgmt}[\Psi, \mathcal{G}]$, then the composed protocol $\Phi^{\Psi \rightarrow \Pi}$ UC-emulates protocol Φ .

¹⁷ In the original definition, an addition identity-bound predicate ξ is also introduced. To be a compliant protocol, the extended identities in an extended session of protocol Φ must also satisfy the predicate ξ . This is less related to this thesis so we omit the discussion on this part.

C UC-secure Zero-Knowledge Non-Interactive Arguments

In this work, we use zkSNARKs / NIZKs as a building block to provide integrity through verifiability. On the client-side, we prove the well-formedness of the ciphertext to guarantee its non-malleability. On the server-side, we prove that the evaluation of a circuit is honestly executed. We begin by revisiting the syntax of a non-interactive argument and reviewing some basic properties in the random oracle model, following the work of Chiesa and Yorgev [26].

Definition 9 (Non-Interactive Argument [26]). Let $\mathcal{O}_{\text{RO}}$ be a random oracle with output length λ . A non-interactive argument is a tuple of algorithms, denoted as $\text{NARG} = (\text{Prv}, \text{Vfy})$, where

- $\pi \leftarrow \text{Prv}^{\mathcal{O}_{\text{RO}}}(x, w)$: Given an instance x , a witness w , and access to a random oracle $\mathcal{O}_{\text{RO}}$, the argument prover algorithm outputs an argument string π .
- $b \leftarrow \text{Vfy}^{\mathcal{O}_{\text{RO}}}(x, \pi)$: Given an instance x , an argument string π , and access to the random oracle $\mathcal{O}_{\text{RO}}$, outputs a bit $b \in \{0, 1\}$, where $b = 1$ denotes acceptance and $b = 0$ denotes rejection.

To be efficient, a NARG must also satisfy *succinctness*.¹⁸ Observe that setting the argument π as the witness w yields a trivial NARG. Thus to be succinct, the argument size should be less than the witness size.

Definition 10 (Succinctness). A non-interactive argument $\text{NARG} = (\text{Prv}, \text{Vfy})$ is succinct if for any output size $\lambda \in \mathbb{N}$ of the random oracle $\mathcal{O}_{\text{RO}}$, any instance-witness pair $(x, w) \in R$ and $\pi \leftarrow \text{Prv}^{\mathcal{O}_{\text{RO}}}(x, w)$, then size of π is given by

$$|\pi| = \text{poly}(\lambda) \cdot \text{polylog}(|x| + |w|).$$

Definition 11 (SNARG). A non-interactive argument $\text{NARG} = (\text{Prv}, \text{Vfy})$ is a succinct non-interactive argument (SNARG) if it also satisfies succinctness.

Several works have proposed the ideal functionality for a zkSNARK and show the realization of the functionality given a global random oracle with restricted *programmability* and *observability*. In this work, we consider the global random oracle $\mathcal{G}_{\text{RO}}$ introduced by Camenisch et al. [14], and the ideal functionality $\mathcal{F}_{\text{zkSNARK}}$ of a zkSNARK introduced by Chiesa and Fenzi [25].

C.1 Global Random Oracle $\mathcal{G}_{\text{RO}}$

In Fig. 14, we depict the *restricted programmable and observable random oracle* (rpoRO) introduced by Camenisch et al. [14]. The oracle can be queried both by a party ITI ($\mathcal{P}_{\text{pid}}, \text{sid}$) and by the adversary (either the real-world adversary $\mathcal{A}$ or the ideal-world adversary $\mathcal{S}$). The random oracle is *domain-separated*, meaning that every query must be of the form (sid, x) . Queries issued by an out-of-session ITI or directly by the adversary are deemed *illegitimate*. The (ideal-process)

¹⁸ Chiesa and Yorgev [26] also discuss succinctness in verification. Since it is less related to the discussion in this work, we omit it here.

Functionality: $\mathcal{G}_{\text{RO}}$

Parameter: output length λ .

- $\langle \text{Query} : [x] \rangle$ from a machine $(\mathcal{P}_{\text{pid}}, \text{sid})$ or the adversary:
 1. If there does not exist v such that (x, v) is in the query list Qry .
 - (a) Sample v from $\{0, 1\}^\lambda$.
 - (b) Append (x, v) to Qry .
 2. Parse x as (sid', x') .
 3. If $\text{sid} \neq \text{sid}'$ or the query is from the adversary, append (x', v) to illegitimate query list Illegal under the label sid' .
 4. Output $\{\text{"val"} : [v]\}$ to the caller.
- $\langle \text{Observe} : [\text{sid}] \rangle$ from the adversary:
 1. Get the illegal trace tr from Illegal under the label sid .
 2. Output $\{\text{"obv"} : [\text{sid}, \text{tr}]\}$ to the adversary.
- $\langle \text{ProgRO} : [x, v] \rangle$ from the adversary:
 1. If there does not exist v' such that (x, v') in Qry and $v' \neq v$:
 - (a) Append (x, v) to Qry .
 - (b) Append x to the list of programmed queries Prog .
 - (c) Output $\{\text{"cfm"} : [1]\}$ to the adversary.
 2. Otherwise, output $\{\text{"cfm"} : [0]\}$ to the adversary.
- $\langle \text{IsProg} : [x] \rangle$ from a machine $(\mathcal{P}_{\text{pid}}, \text{sid})$ or the adversary:
 1. Parse x as (sid, x') .
 2. If it is from $(\mathcal{P}_{\text{pid}}, \text{sid})$ and $\text{sid} \neq \text{sid}'$, abort.
 3. Set $b := x \in ? \text{Prog}$.
 4. Output $\{\text{"chk"} : [b]\}$ to $(\mathcal{P}_{\text{pid}}, \text{sid})$ or the adversary.

Fig. 14: A restricted programmable and observable global random oracle (rpoRO) $\mathcal{G}_{\text{RO}}$ described by Camenisch et al. [14].

adversary $\mathcal{S}$ can thus observe the queries made by the real-world adversary $\mathcal{A}$ in other sessions for the purpose of simulation.

The adversary may also *program* the random oracle by prescribing specific query-answer pairs. Moreover, parties can check whether a query has been programmed. Intuitively, this feature allows a party to abort the protocol if a (maliciously) programmed query occurs during (real-world) execution. Importantly, in the ideal world, since the parties are dummy, and all their interaction with the random oracle is mediated by the simulator. Thus the environment can issue checks only through corrupted or out-of-session honest parties, which means the simulator fully controls the interface by which the environment interacts with the oracle to check for such programmed queries. This grants $\mathcal{S}$ the ability to program the oracle undetectably, thereby endowing it with greater power than the real-world adversary $\mathcal{A}$, making the functionality useful for proof.

Functionality: $\mathcal{F}_{\text{zkSNARK}}^R$ $\mathcal{F}_{\text{NIZK}}^R$

Parameter: a binary relation R , an instance bound n .

Participants: a prover $\mathcal{P}_{\text{pvr}}$, a verifier $\mathcal{P}_{\text{vfr}}$, and an ideal-process adversary $\mathcal{S}$.

- $\langle \text{Setup} : [\text{sid}] \rangle$ from $\mathcal{P}_{\text{pid}}$:
 1. If Algo has been set for sid or $\mathcal{P}_{\text{pid}}$ has been corrupted, abort.
 2. Otherwise, output $\langle \text{Setup} : [\text{sid}] \rangle$ to $\mathcal{S}$. Upon receiving a tuple of algorithms $(\text{Sim}, \text{Ext}, \text{Vfy})$ from $\mathcal{S}$. Set Algo as this tuple for sid .
- $\langle \text{Prove} : [\text{sid}, x, (w, w^*)] \rangle$ from the prover $\mathcal{P}_{\text{pvr}}$:
 1. If Algo has not been set, or $|x| > n$, or $\mathcal{P}_{\text{pvr}}$ has been corrupted, abort.
 2. Output $\langle \text{Observe} : [\text{sid}] \rangle$ to $\mathcal{S}$ and obtain the illegitimate query trace $\{\text{"obv"} : [\text{sid}, \text{tr}_z]\}$. Append tr_z to the trace list ExtTr for session sid .
 3. Call $(\pi, \text{pg}) \leftarrow \text{Sim}^{\mathcal{G}_{\text{RO}}[\text{sid}]}(x)$.
 4. For each entry (x, v) in pg , call $\mathcal{S}$ to program $\mathcal{G}_{\text{RO}}$.
 5. Append (x, v) to the list of proved argument Proved in session sid .
 6. Append pg to the list of honestly programmed points HProg in session sid .
 7. Append x to list of proved instance Inst .
 8. Output $\{\text{"pf"} : [\text{sid}, x, \pi]\}$ to $\mathcal{P}_{\text{vfr}}$.
- $\langle \text{Verify} : [\text{sid}, x, \pi] \rangle$ from the verifier $\mathcal{P}_{\text{vfr}}$:
 1. If Algo has not been set, or $|x| > n$, or $\mathcal{P}_{\text{vfr}}$ has been corrupted, abort.
 2. If $(x, \pi) \in \text{Proved}$, output $\{\text{"vf"} : [\text{sid}, x, \pi, 1]\}$ to $\mathcal{P}_{\text{vfr}}$ and halt.
 3. Call $b \xleftarrow{\text{tr}_v} \text{Vfy}^{\mathcal{G}_{\text{RO}}[\text{sid}]}(x, \pi)$. If $b = 0$, output $\{\text{"vf"} : [\text{sid}, x, \pi, 0]\}$ to $\mathcal{P}_{\text{vfr}}$ and halt.
 4. For each $(x, v) \in \text{tr}_v \setminus \text{HProg}[\text{sid}]$, query $\mathcal{G}_{\text{RO}}$ with $\langle \text{IsProg} : [(\text{sid}, x)] \rangle$ to get the response $\{\text{"chk"} : [b]\}$. If $b = 1$ for any (x, v) , output $\{\text{"vf"} : [\text{sid}, x, \pi, 0]\}$ to $\mathcal{P}_{\text{vfr}}$ and halt.
 5. If $x \notin \text{Inst}[\text{sid}]$:

- (a) Output $\langle \text{Observe} : [\text{sid}] \rangle$ to $\mathcal{S}$ and obtain the illegitimate query trace $\{\text{"obv"} : [\text{sid}, \text{tr}_z]\}$. Append tr_z to the trace list ExtTr for session sid .
 - (b) For each $(x, v) \in \text{ExtTr}[\text{sid}]$, query $\mathcal{G}_{\text{RO}}$ with $\langle \text{IsProg} : [(\text{sid}, x)] \rangle$. If x is programmed and $(x, v) \in \text{HProg}[\text{sid}]$, append (x, v) to trace tr'_z .
 - (c) Call $w \leftarrow \text{Ext}(x, \pi, \text{tr}'_z)$.
 - (d) If $(x, w) \notin R$ and output $\{\text{"vf"} : [\text{sid}, x, \pi, \perp]\}$ and halt.

- (a) Output $\langle \text{Witness} : [\text{sid}, x, \pi] \rangle$ to $\mathcal{S}$ and get $\{\text{"wit"} : [\text{sid}, x, \pi, w]\}$ as response.
 - (b) If $\nexists w^* : (x, (w, w^*)) \in R$, output $\{\text{"vf"} : [\text{sid}, x, \pi, \perp]\}$ and halt.
 6. Output $\{\text{"vf"} : [\text{sid}, x, \pi, 1]\}$ and halt.
- $\langle \text{Corrupt} : [\text{pid}] \rangle$ from $\mathcal{S}$:
 1. Label $\mathcal{P}_{\text{pid}}$ as corrupted.

Fig. 15: Ideal functionality $\mathcal{F}_{\text{zkSNARK}}$ for a simulation-extractable zkSNARK [25] and Ideal functionality $\mathcal{F}_{\text{NIZK}}$ for a simulation-sound NIZK; framed part is exclusively for $\mathcal{F}_{\text{zkSNARK}}$; the highlighted part is exclusively for $\mathcal{F}_{\text{NIZK}}$.

C.2 Ideal Functionality $\mathcal{F}_{\text{zkSNARK}}$ and $\mathcal{F}_{\text{NIZK}}$

In Figure 15, we present the ideal functionality $\mathcal{F}_{\text{zkSNARK}}$, which is a simplified version of the one proposed by Chiesa and Fenzi [25]. Specifically, they gave the definition of the functionality for a specific session. In this work, we define the input to $\mathcal{F}_{\text{zkSNARK}}$ as originating from a party $\mathcal{P}_{\text{pid}}$ rather than a session $(\mathcal{P}_{\text{pid}}, \text{sid})$. In their work [25], they also introduced a stronger model in which the random coins used for argument generation and verification is disclosed to the adversary when a party is corrupted. This is specifically for the case of *adaptive corruption*, which was also discussed by Lysyanskaya and Rosenbloom [50]. Since we only consider

static corruption, the simplified functionality described in Figure 15 suffice for the security proofs in this work. For more details on UC-secure zkSNARKs against adaptive corruption, we refer readers to [25].

Building on this functionality, we propose a relaxed variant compatible with simulation-sound NIZKs. Drawing inspiration from their approach to handling extraction failure, as well as the *gullible* zero-knowledge functionality introduced by Camenisch et al. [15] and the functionality by Groth et al. [35], we define a witness as a pair (w, w^*) such that the component w is subject to knowledge soundness, while w^* is required to exist. During verification, the adversary provides w , while the functionality checks for the existence of a valid w^* with respect to the instance x . If the verifier $\mathbf{Vfy}$ accepts the argument but no such w^* exists, the functionality flags this as a violation of simulation soundness.

We emphasize that we work with *any* simulation security [29]. In particular, a binary relation need not be satisfied when the simulator $\mathbf{Sim}_{zk}$ generates an argument. This is because, when composing with $\mathcal{F}_{\text{MGHE}}$, the ideal-process adversary $\mathcal{S}$ only learns the message length $|m|$ when producing a ciphertext for an uncorrupted client. Consequently, the resulting plaintext–ciphertext pair may not satisfy the relation, even though it is still considered "honest".

Remark 3. We remark that the existence of the witness w^* is not checked through exhaustive search but the simulator who simulates the functionality may obtain a state (from an external channel) to allow this. For example, when a simulator who simultaneously interacts with $\mathcal{F}_{\text{MGHE}}$ and $\mathcal{F}_{\text{NIZK}}$ can obtain the random coins as existence witness w^* from $\mathcal{F}_{\text{MGHE}}$ and check it for $\mathcal{F}_{\text{NIZK}}$.

D Global Key Registry $\mathcal{G}_{\text{KRK}}$

Inspired by the *key registration with knowledge* (KRK) setup in [6,18], we define a global subroutine $\mathcal{G}_{\text{KRK}}$ ¹⁹ for the registration and retrieval of public parameters and keys in a multi-group setting, as illustrated in Fig. 16.

To model the multi-group setting, we let a group $\mathcal{P}_{\text{grp}}$ provide inputs to $\mathcal{G}_{\text{KRK}}$ for key aggregation. If non-interactive key aggregation is available (as in Kwak et al. [44]), a client $\mathcal{P}_{\text{cli}}$ or server $\mathcal{P}_{\text{srv}}$ can provide its input directly, without relying on a group entity. Since key aggregation generally requires the secret keys, this step may need to be executed via an MPC protocol. Indeed, this “sub-functionality” of $\mathcal{G}_{\text{KRK}}$ closely resembles the simple MPC functionality $\mathcal{F}_{\text{MPC}}$ studied in prior works [27,49].

We remark that $\mathcal{G}_{\text{KRK}}$ can be viewed as a trusted party for public parameter and key generation, where the adversary’s influence is restricted to choosing

¹⁹ We note that $\mathcal{G}_{\text{KRK}}$ is *Π -regular*, meaning it communicates only with the calling parties and the subroutines explicitly declared in the protocol, and *subroutine-respecting* with respect to the global subroutine, meaning it never interacts outside its session or beyond the interface defined by the global subroutine. Thus $\mathcal{G}_{\text{KRK}}$ satisfies the preconditions required by the UC composition theorem.

Functionality: $\mathcal{G}_{\text{KRR}}$

- Participants:** n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\text{srv}}$; an adversary.
- $\langle \text{RegParam} : [\text{sid}, \lambda, \kappa] \rangle$ from $\mathcal{P}_{\text{cli}}$ or $\mathcal{P}_{\text{srv}}$:
 1. If a parameter has been set for sid , abort.
 2. Otherwise, run $\text{pp} \leftarrow \text{MGHE.PGen}(1^\lambda, 1^\kappa)$ and store pp under label sid .
 - $\langle \text{RetParam} : [\text{sid}] \rangle$ from $\mathcal{P}_{\text{cli}}$:
 1. If a parameter pp has not been set for sid , abort.
 2. Otherwise, output $\{\text{"param"} : [\text{sid}, \text{pp}]\}$ at $\mathcal{P}_{\text{cli}}$.
 - $\langle \text{RegKey} : [\text{cli}, \text{sid}, \text{pp}; \dot{\gamma}] \rangle$ from $\mathcal{P}_{\text{cli}}$:
 1. If pp does not match with sid , abort.
 2. If $\mathcal{P}_{\text{cli}}$ is not corrupted, run $(\text{pk}, \text{sk}) \leftarrow \text{MGHE.KGen}(\text{pp})$.
 3. If $\mathcal{P}_{\text{cli}}$ is corrupted, run $(\text{pk}, \text{sk}) \leftarrow \text{MGHE.KGen}(\text{pp}; \gamma)$.
 4. Store (pk, sk) under label (cli, sid) .
 - $\langle \text{RetKey} : [\text{pid}', \text{sid}] \rangle$ from $\mathcal{P}_{\text{pid}}$ or the adversary.
 1. If no key under label $(\text{pid}', \text{sid})$, abort.
 2. If $\text{pid} = \text{pid}'$, output $\{\text{"key"} : [\text{pid}', \text{sid}, (\text{pk}, \text{sk})]\}$ at $\mathcal{P}_{\text{pid}}$.
 3. Otherwise, output $\{\text{"key"} : [\text{pid}', \text{sid}, \text{pk}]\}$ at $\mathcal{P}_{\text{pid}}$ or the adversary.
 - $\langle \text{JoinKey} : [\text{grp}, \text{sid}] \rangle$ from $\mathcal{P}_{\text{grp}}$:
 1. If there are $(\text{pk}_i, \text{sk}_i)$ stored under label $(\text{cli}_i, \text{sid})$ for all $\text{cli}_i \in \text{grp}$:
 - (a) Run $\text{jpk} \leftarrow \text{MGHE.KJoin}((\text{pk}_i, \text{sk}_i)_{\text{cli}_i \in \text{grp}})$.
 - (b) Store jpk under label (grp, sid) .
 2. Otherwise, abort.
 - $\langle \text{Corrupt} : [\text{pid}] \rangle$ from the adversary:
 1. Label $\mathcal{P}_{\text{pid}}$ as corrupted.

Fig. 16: Global subroutine $\mathcal{G}_{\text{KRR}}$ with public parameter and key registry functionality. Dot-boxed inputs originate exclusively from the corrupted party (on behalf of the adversary). We use $\langle \cdot \rangle$ for inputs sent from parties to the functionality, and $\{\cdot\}$ for outputs from the functionality to parties (consistently across the following functionalities and protocols).

malicious randomness for corrupted parties. Thus, the adversary remains *semi-malicious* during the key-generation process throughout this work. To further capture a *malicious* adversary also in the key-generation process, one may additionally require a zkSNARK proof to ensure well-formed keys, beyond those considered in Sections 4.3 and 4.4. Our definition of $\mathcal{G}_{\text{KRR}}$ is intentionally streamlined to limit the number of hybrid games in the proofs, thereby improving readability.

We assume that $\mathcal{G}_{\text{KRR}}$ follows the *pid-wise corruption* model of [17]. During key retrieval, if a party $\mathcal{P}_{\text{pid}}$ is corrupted, the adversary may obtain its secret key for any session by specifying the session identifier sid . In this case, the functionality treats the request as if it were sent by the corrupted party.

Finally, we note that the ideal-process adversary $\mathcal{S}$ can obtain the public key of any uncorrupted party directly from $\mathcal{G}_{\text{KRR}}$, without going through a corrupted party. This reflects the real-world scenario in which public keys are publicly available and not restricted to members of a particular group.

E Authenticated Channel $\mathcal{F}_{\text{AUTH}}$

In Fig. 17, we illustrate an authenticated channel $\mathcal{F}_{\text{AUTH}}$ through which parties exchange messages, as a pre-assumption commonly made in existing works on MPC. In contrast to the standard authenticated channel defined by Canetti et al. [16], which models point-to-point message delivery, our ideal functionality $\mathcal{F}_{\text{AUTH}}$ is designed as a registry where parties post and retrieve messages. This abstraction more accurately captures the broadcast-style communication typically used in multi-party settings.

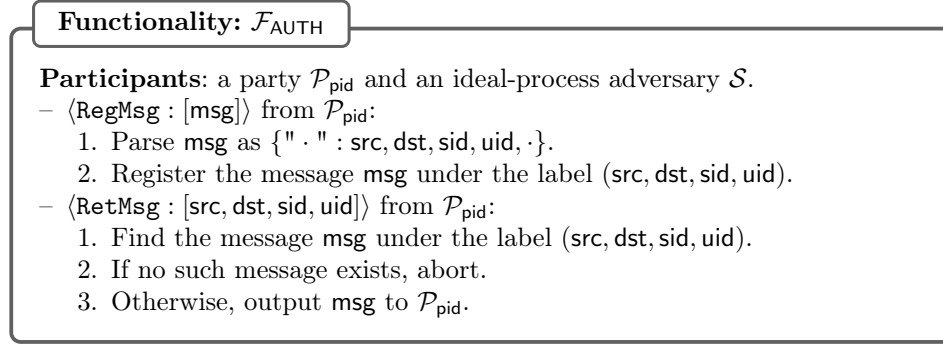


Fig. 17: An authenticated channel $\mathcal{F}_{\text{AUTH}}$.

We recall that under pid -wise corruption [17], once a party $\mathcal{P}_{\text{pid}}$ is corrupted, the functionality thereafter accepts inputs directly from the adversary, treating them as if they originated from $\mathcal{P}_{\text{pid}}$. In particular, when a party $\mathcal{P}_{\text{pid}}$ submits an input to register a message, the functionality simply logs it under the identifier pid , regardless of whether it was sent by an uncorrupted party or by the adversary on behalf of a corrupted one.

F Full Proofs

F.1 Full proof of Theorem 2

Theorem 2. *Let MGHE be a multi-group homomorphic encryption scheme with threshold factor t . Then, the protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global key registry $\mathcal{G}_{\text{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that non-adaptively corrupts the clients and the server, provided that:*

- MGHE satisfies robustness (Rob), and the adversary corrupts fewer than $\frac{|\text{grp}|}{t}$ clients in any group.
- MGHE is $\text{IND-CPA}^{\text{pD}}$ -secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group grp under which it provides input contains no more than $\frac{|\text{grp}|}{t}$ corrupted clients.

- MGHE satisfies SIM-PDEC security with respect to a simulator Sim_{th} .

Proof. We consider n clients $\mathcal{P}_{\text{cli}_1}, \dots, \mathcal{P}_{\text{cli}_n}$ and a server $\mathcal{P}_{\text{srv}}$ in the system. In the setup phase, we let a client $\mathcal{P}_{\text{cli}_i}$, for some $i \in [n]$, be first activated with the input $\langle \text{RegParam} : [\text{sid}] \rangle$ to initialize the system's public parameter at $\mathcal{G}_{\text{KRK}}$. Following this, each client $\mathcal{P}_{\text{cli}_i}$ receives the input $\langle \text{RegKey} : [\text{cli}, \text{sid}] \rangle$ to generate their respective key pairs $(\text{pk}_i, \text{sk}_i)$ at $\mathcal{G}_{\text{KRK}}$.

Next, we form k initial groups $\text{grp}_1, \dots, \text{grp}_k$, where each group $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ includes a subset of clients indexed by $I_j \subseteq [n]$ for $j \in [k]$. For every group, the corresponding “virtual” group entity $\mathcal{P}_{\text{grp}_j}$ is provided with input $\langle \text{JoinKey} : [\text{grp}_j, \text{sid}] \rangle$ to compute joint public key jpk_j at $\mathcal{G}_{\text{KRK}}$.

Then the adversary corrupts a subset of clients by sending backdoor messages and queries $\mathcal{G}_{\text{KRK}}$ to access the secret keys sk_i of the corrupted clients. Notably, during this setup phase, $\mathcal{G}_{\text{KRK}}$ does these operations for both of the real and ideal worlds. Thus these two worlds remain indistinguishable to the environment $\mathcal{Z}$ at this stage.

To prove the security of the protocol, we then define the following sequence of hybrid games:

- **REAL** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: This represents the execution of the protocol Π_{MGHE} in the real world and an adversary $\mathcal{A}$ attacking the protocol.
- **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1, \mathcal{G}_{\text{KRK}}}$: In this hybrid game, we keep track of the message-ciphertext tuples (m, c) that are encrypted and evaluated, as well as the partial decryptions d that are honestly generated with respect to (m, c) . When a client combines the partial decryptions, if it has obtained a sufficient number of honest partial decryptions, it outputs m . For schemes on approximate number, it outputs $m + \delta + \sigma + \sum_{i=1}^{| \text{grp} | / t} \eta_i$ where δ is calculated with $\text{GetEncNoise}(\cdot)$ and $\text{GetEvalNoise}(\cdot)$.
- **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2, \mathcal{G}_{\text{KRK}}}$: In this hybrid game, whenever an uncorrupted client $\mathcal{P}_{\text{cli}}$ is activated with the input $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpk}, m] \rangle$, the client outputs $c \leftarrow \text{MGHE.Enc}(\text{jpk}, 0^{|m|})$.
- **IDEAL** $_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: This is the ideal execution where the ideal functionality $\mathcal{F}_{\text{MGHE}}$ is attached to the simulator $\mathcal{S}_{\text{MGHE}}$ and interacts with dummy client $\mathcal{P}_{\text{cli}_1}, \dots, \mathcal{P}_{\text{cli}_n}$.

Claim 1.1. *If MGHE satisfies robustness, then **REAL** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1, \mathcal{G}_{\text{KRK}}}$ are computationally indistinguishable.*

Proof. We prove the claim by showing that if there is an environment $\mathcal{Z}$ that distinguishes between **REAL** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1, \mathcal{G}_{\text{KRK}}}$, then we can use it to construct an adversary $\mathcal{A}_{dc}$ that breaks the robustness of MGHE. For this, we let $\mathcal{A}_{cpa}$ simulate the interaction between the environment $\mathcal{Z}$ and the two worlds.

We let the adversary $\mathcal{A}_{dc}$ form the same groups $\text{grp}_1, \dots, \text{grp}_k$ by specifying the index sets $I_1, \dots, I_k$ in the $\mathcal{G}_{\text{MGHE}}^{\text{Rob}}$ game and corrupts the same clients by specifying the index set $I_{\mathcal{A}}$ as in the setup phase. When a client queries $\mathcal{G}_{\text{KRK}}$ to retrieve the group public key, we let $\mathcal{A}_{dc}$ provides the one generated by $\mathcal{G}_{\text{MGHE}}^{\text{Rob}}$. Similarly, when the adversary $\mathcal{A}$ inputs $\mathcal{G}_{\text{KRK}}$ for the key pairs of corrupted clients,

we let $\mathcal{A}_{dc}$ respond with (pk_i, sk_i) for $i \in I_{\mathcal{A}}$, which is also generated by the game G_{MGHE}^{Rob} .

- **On input** $\langle \text{Encrypt} : [grp, sid, jpk_j, m] \rangle$ **to an uncorrupted client** $\mathcal{P}_{cli}$: We then let $\mathcal{A}$ honestly sample a random coin r from the distribution and query its oracle $\mathcal{O}_{Enc}$ with $(m; \omega)$. We observe that $\mathcal{O}_{Enc}$ does $c \leftarrow \text{MGHE.Enc}(jpk_j, m; \omega)$, which is exactly the behavior of an uncorrupted client in both of these worlds. Thus we write c from $\mathcal{O}_{Enc}$ as output of $\mathcal{P}_{cli}$.
- **On input** $\langle \text{Encrypt} : [grp, sid, jpk_j, m; \omega] \rangle$ **to a corrupted client** $\mathcal{P}_{cli}$: We similarly let $\mathcal{A}$ query its oracle $\mathcal{O}_{Enc}$ with $(m; \omega)$ where ω is the random coin selected by $\mathcal{Z}$. Then upon getting c from $\mathcal{O}_{Enc}$, we write it as output of $\mathcal{P}_{cli}$.
- **On input** $\langle \text{Evaluate} : [sid, f, (grp_j, sid_j, jpk_j, c_j)_{j \in [\ell]}; \sigma] \rangle$ **to a corrupted server** $\mathcal{P}_{srv}$: We observe by definition of these two worlds, all ciphertexts to be evaluated must either be encrypted by the clients or result from an evaluation performed by the server. This implies that in G_{MGHE}^{Rob} , these ciphertexts are all entries in the lookup table T . Thus, we let $\mathcal{A}_{dc}$ identify these ciphertexts in the lookup table T by specifying an index set L and query the oracle $\mathcal{O}_{Eval}$ with $(f, L; \sigma)$. Observe that the oracle $\mathcal{O}_{Eval}$ returns the ciphertext $\hat{c} \leftarrow \text{MGHE.Eval}(f, (T[l].jpk, T[l].c)_{l \in L}; \sigma)$. This is exactly the behavior of the server in both of these worlds. Thus we write $\hat{c}$ as the output of $\mathcal{P}_{srv}$.
- **On input** $\langle \text{Share} : [cli, grp, sid, c] \rangle$ **to an uncorrupted** $\mathcal{P}_{cli}$: Observe that we require that c must be either encrypted by clients or an evaluation performed by the server. Thus we similarly let $\mathcal{A}_{dc}$ identify the index l of c in the lookup table T of G_{MGHE}^{Rob} . Then we let $\mathcal{A}_{dc}$ query $\mathcal{O}_{Dec}$ with l . Note that $\mathcal{O}_{Dec}$ does $d \leftarrow \text{MGHE.PDec}(sk, c)$ for where sk is the secret key of the client, which reflects the behavior of the client in these two worlds. Thus we write d as the output of $\mathcal{P}_{cli}$.
- **On input** $\langle \text{Share} : [cli, grp, sid, c; \eta] \rangle$ **to a corrupted** $\mathcal{P}_{cli}$: We similarly let $\mathcal{A}_{dc}$ identify the index l of c in the lookup table T of G_{MGHE}^{Rob} . Then we let $\mathcal{A}_{dc}$ query $\mathcal{O}_{Dec}$ with l and also with η . Note that $\mathcal{O}_{Dec}$ does $d \leftarrow \text{MGHE.PDec}(sk, c; \eta)$ for where sk is the secret key of the client, which reflects the behavior of the client in these two worlds. Thus we write d as the output of $\mathcal{P}_{cli}$.

We observe that in $\text{HYB}_{\Pi_{MGHE}, \mathcal{A}, \mathcal{Z}}^{1G_{KRK}}$, when an (uncorrupted) client $\mathcal{P}_{cli}$ is activated to combine the partial decryptions output by clients in a group grp' for a message m that is intended for the group grp , where $grp' \subseteq grp$. The message m is output if at least $\frac{|grp|}{t}$ partial decryptions are honestly obtained and output by clients in grp' .

It follows that the environment $\mathcal{Z}$ can only distinguish between these two worlds if the algorithm $\text{MGHE.Combine}(\cdot)$ reconstructs a message different from m using the same set of partial decryptions from grp' in $\text{REAL}_{\Pi_{MGHE}, \mathcal{A}, \mathcal{Z}}^{G_{KRK}}$. This corresponds to the winning condition of G_{MGHE}^{Rob} . Thus we let $\mathcal{A}_{dc}$ to output the same partial decryptions from grp' in the game G_{MGHE}^{Rob} .

$$\Delta_{\mathcal{Z}} \left(\text{REAL}_{\Pi_{MGHE}, \mathcal{A}, \mathcal{Z}}^{G_{KRK}}, \text{HYB}_{\Pi_{MGHE}, \mathcal{A}, \mathcal{Z}}^{1G_{KRK}} \right) \leq \text{Adv}_{MGHE}^{Rob}(\mathcal{A}_{dc}). \quad (1)$$

□

Claim 1.2. *If MGHE is IND-CPA^{PD} secure, then $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$ and $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$ are computationally indistinguishable.*

Proof. We prove the claim by demonstrating that if an environment $\mathcal{Z}$ can distinguish between $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$ and $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, then we can use $\mathcal{Z}$ to construct an adversary $\mathcal{A}_{\text{cpa}}$ to break the IND-CPA^{PD} security of MGHE. For this, we let $\mathcal{A}_{\text{cpa}}$ simulate the interaction between the environment $\mathcal{Z}$ and the two worlds. We let $\mathcal{A}_{\text{cpa}}$ follow a similar setup as in proof of Claim 1.1 but interacts with the game $\mathcal{G}_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}}$ instead. Then $\mathcal{A}_{\text{cpa}}$ handles its oracle queries as follows to simulate the responses based on the messages that the environment $\mathcal{Z}$ delivers to each party.

- **On input** $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}_j, m] \rangle$ **to an uncorrupted client** $\mathcal{P}_{\text{cli}}$: We consider the following two cases for this input.
 1. If the group grp_j is not corrupted (i.e., $|\text{grp}_j \cap \text{Corrupt}| < \frac{|\text{grp}_j|}{t}$), we let $\mathcal{A}_{\text{cpa}}$ query its encryption oracle $\mathcal{O}_{\text{Enc}}$ with the tuple (jpg_j, m_0, m_1) , where $m_0 = m$ and $m_1 = 0^{|m|}$ for the m chosen by $\mathcal{Z}$. When $\mathcal{O}_{\text{Enc}}$ responds with $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m)$, this simulates the behavior of an uncorrupted client in the world $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$, where m is encrypted, and the encryption is output by $\mathcal{P}_{\text{cli}}$. On the other hand, if $\mathcal{O}_{\text{Enc}}$ returns $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, 0^{|m|})$, it represents the scenario where the dummy party $\mathcal{P}_{\text{cli}}$ outputs the encryption of $0^{|m|}$, which is returned from $\mathcal{F}_{\text{MGHE}}$ and generated by $\mathcal{S}_{\text{MGHE}}$. Accordingly, we write c returned by $\mathcal{O}_{\text{Enc}}$ as the output of the client $\mathcal{P}_{\text{cli}}$.
 2. Otherwise, if grp_j is corrupted, we let $\mathcal{A}_{\text{cpa}}$ query its encryption oracle $\mathcal{O}_{\text{Enc}}$ with the tuple $(\text{jpg}_j, m; \omega)$, where ω is *honestly* sampled from the distribution since we are simulating the behavior of an *uncorrupted* client. Note that the oracle $\mathcal{O}_{\text{Enc}^*}$ returns $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m; \omega)$. In $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$, this is exactly the behavior of an uncorrupted client. Also, in $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, this is what the simulator $\mathcal{S}_{\text{MGHE}}$ does when it is given the actual message m by $\mathcal{F}_{\text{MGHE}}$. Thus, we then write c from $\mathcal{O}_{\text{Enc}^*}$ as the output of the client $\mathcal{P}_{\text{cli}}$.
- **On input** $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}_j, m; \omega] \rangle$ **to a corrupted client** $\mathcal{P}_{\text{cli}}$: We let $\mathcal{A}_{\text{cpa}}$ query its oracle with $(m; \omega)$, where ω is the random coins chosen by $\mathcal{Z}$. The oracle $\mathcal{O}_{\text{Enc}^*}$ returns $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m; \omega)$. In $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$, this input from $\mathcal{Z}$ is delivered to the adversary $\mathcal{A}$, who then computes $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m; \omega)$ using the random coins r provided by $\mathcal{Z}$, and writes c as the output of a corrupted client. Similarly, in $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, when the simulator $\mathcal{S}_{\text{MGHE}}$ sends the input to $\mathcal{F}_{\text{MGHE}}$, the functionality also performs the encryption honestly using the random coins chosen by $\mathcal{Z}$. The simulator $\mathcal{S}_{\text{MGHE}}$ then writes c received from $\mathcal{F}_{\text{MGHE}}$ as the output of the corrupted client. Accordingly, we write c from $\mathcal{O}_{\text{Enc}^*}$ as the output of a corrupted client $\mathcal{P}_{\text{cli}}$.
- **On input** $\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}, \sigma] \rangle$ **to a corrupted server** $\mathcal{P}_{\text{srv}}$: Note that by definition, all ciphertexts to be evaluated must either

be encrypted by the clients or result from an evaluation performed by the server. This implies that in $\text{G}_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}}$, these ciphertexts are all entries in the lookup table T . Thus, we let $\mathcal{A}_{cpa}$ first identify these ciphertexts in the lookup table T by specifying an index set L and query the oracle $\mathcal{O}_{\text{Eval}}$ with the circuit f , the index set L , and the random coin σ . If $\perp$ is returned, $\mathcal{A}_{dc}$ aborts, and nothing is written on any tape. Otherwise, the oracle $\mathcal{O}_{\text{Eval}}$ returns the ciphertext $\hat{c} \leftarrow \text{MGHE.Eval}(f, (T[l].\text{jpgk}, T[l].c)_{l \in L})$.

In $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$, since we consider a semi-malicious adversary, the adversary $\mathcal{A}$ evaluates the circuit f honestly to obtain the ciphertext $\hat{c}$. Similarly, in $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, by the definition of $\mathcal{F}_{\text{MGHE}}$, when the server is corrupted, the functionality also evaluates the circuit honestly. After $\hat{c}$ is obtained, it is written as the output of a corrupted client. Thus, the behavior of these two executions is identical for this input and aligns with the behavior of $\mathcal{O}_{\text{Eval}}$. Accordingly, we write $\hat{c}$ from $\mathcal{O}_{\text{Eval}}$ as the output of the corrupted server $\mathcal{P}_{\text{srv}}$.

- **On input** $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c] \rangle$ **to a uncorrupted** $\mathcal{P}_{\text{cli}}$: We let $\mathcal{A}_{cpa}$ query it oracle $\mathcal{O}_{\text{Dec}}$ with an index l identifying the position of c in the lookup table T . If the oracle returns $\perp$, then $\mathcal{A}_{cpa}$ aborts the query. This indicates that the messages being decrypted in $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$ and $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$ differ. Consequently, after the execution, the environment $\mathcal{Z}$ can trivially distinguish between these two worlds by inspecting the final output tape of a client. Otherwise, the oracle $\mathcal{O}_{\text{Dec}}$ returns $\{d_i\}_{\text{cli}_i \in \text{grp}}$ where $d_i \leftarrow \text{MGHE.PDec}(\text{sk}_i, c)$ for $\text{cli}_i \in \text{grp}$. Note that in both of the worlds $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$ and $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, the partial-decryptations output by clients are $\text{MGHE.PDec}(\cdot)$ in both two worlds. Therefore, we can then write the d_i from $\mathcal{O}_{\text{Dec}}$ as the outputs of the clients.
- **On input** $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c; \eta] \rangle$ **to a corrupted** $\mathcal{P}_{\text{cli}}$: We note that for corrupted client, the adversary $\mathcal{A}_{cpa}$ has the secret keys of the corrupted client. Thus we let $\mathcal{A}_{cpa}$ directly run $d \leftarrow \text{MGHE.PDec}(\text{sk}_i, c; \eta)$. Then we write d as the output of the corrupted client.
- **On input** $\langle \text{Combine} : [\text{grp}, \text{sid}, c, \{d_i\}_{\text{cli}_i \in \text{grp}'}] \rangle$ **to a (uncorrupted or corrupted) client** $\mathcal{P}_{\text{cli}}$: Note that in both $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$ and $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, if insufficient partial decryption is obtained, then $\mathcal{P}_{\text{cli}}$ outputs $\perp$. Otherwise, it outputs the original message m (plus an error term if approximate). Observe that by robustness, $m' \leftarrow \text{MGHE.Combine}(c, \{d_i\}_{\text{cli}_i \in \text{grp}'})$ is indistinguishable from m . Thus, we write $\perp$ if not enough partial decryptions are obtained; otherwise, we write m' as the output of $\mathcal{P}_{\text{cli}}$.

Finally, when $\mathcal{Z}$ outputs a bit b , we let $\mathcal{A}_{cpa}$ also output b . Thus $\mathcal{Z}$'s distinguishing advantage between these two worlds directly translates to $\mathcal{A}_{cpa}$'s advantage in the $\text{IND-CPA}^{\text{PD}}$ game i.e.,

$$\Delta_{\mathcal{Z}} \left(\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}, \text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}} \right) \leq \text{Adv}_{\text{MGHE}}^{\text{IND-CPA}^{\text{PD}}}(\mathcal{A}_{cpa}). \quad (2)$$

□

Claim 1.3. *If MGHE is SIM-PDEC secure with respect to a threshold simulator Sim_{th} , then $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$ and $\text{IDEAL}_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ are statistically indistinguishable.*

Proof. By the definition of SIM-PDEC, there exists a threshold simulator Sim_{th} such that a partial-decryption generated by an uncorrupted client using $d \leftarrow \text{MGHE.PDec}(\text{sk}_i, c)$ is statistically indistinguishable from a share generated via $d \leftarrow \text{Sim}_{th}(c, m, \{\text{sk}_i\}_{i \in I_A})$. For corrupted clients, the decryption process remains the same in both hybrid games since they are both computed as $d \leftarrow \text{MGHE.PDec}(\text{sk}_i, c; \eta)$, or they directly injects a fake partial decryption. Additionally, for approximate schemes, we know that δ_{ct} and δ are indistinguishable by previous hop. Thus we have that the reconstruct messages in these two worlds are also indistinguishable from each other in case of approximate schemes..

Therefore, we have that the partial-decryptions output by any client $\mathcal{P}_{cli}$ (whether uncorrupted or corrupted) in both of $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$ and $\text{IDEAL}_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ are statistically indistinguishable. $\square$

Therefore, by combing Claims 1.1, 1.2, and 1.3, we can then conclude the proof for the theorem. $\square$

F.2 Full proof of Theorem 3

Theorem 3. *Let MGHE be a multi-group homomorphic encryption scheme on approximate number with threshold factor t . Then, the protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global key registry $\mathcal{G}_{\text{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that non-adaptively corrupts parties, provided that:*

- MGHE satisfies robustness, and the adversary corrupts fewer than $\frac{|\text{grp}|}{t}$ clients in any group.
- MGHE is IND-CIRC secure and the server is not corrupted.
- MGHE is IND-CPA^{PD}-secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group grp under which it provides input contains no more than $\frac{|\text{grp}|}{t}$ corrupted clients.
- MGHE satisfies SIM-PDEC security with respect to a simulator Sim_{th} .

Proof. Note that the correctness of the protocol follows Theorem 2. We only show security in the corollary. We consider the same setting as in the proof of Theorem 2. To show the security of Π_{MGHE} , we consider the following sequence of hybrid games:

- $\text{REAL}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: This represents the execution of the protocol Π_{MGHE} in the real world, involving the environment $\mathcal{Z}$ and the adversary $\mathcal{A}$.
- $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$: In this hybrid game, we keep track of the message-ciphertext tuples (m, c) that are encrypted and evaluated, as well as the partial decryptions d that are honestly generated with respect to (m, c) . When a client combines the partial decryptions, if it has obtained a sufficient number of honest partial

decryptions, it outputs $m + \delta + \sigma + \sum_{i=1}^{|grp|/t} \eta_i$ where δ is calculated with $\text{GetEncNoise}(\cdot)$ and $\text{GetEvalNoise}(\cdot)$.

- **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRR}}}$: When $\mathcal{P}_{\text{srv}}$ is activated with $\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}] \rangle$, we let $\mathcal{P}_{\text{srv}}$ output the ciphertext

$$\hat{c}^* \leftarrow \text{MGHE.Eval}(f^*, (\text{jpg}_j, c_j)_{j \in [\ell]})$$

where f^* is a randomly sampled circuit from the circuit space $\mathcal{F}_\kappa$. The noise δ is then re-calculated with respect to f^* using $\text{GetEncNoise}(\cdot)$ and $\text{GetEvalNoise}(\cdot)$.

- **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{3\mathcal{G}_{\text{KRR}}}$: In this hybrid game, whenever an uncorrupted client $\mathcal{P}_{\text{cli}}$ is activated with the input $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m] \rangle$, the client outputs $c \leftarrow \text{MGHE.Enc}(\text{jpg}, 0^{|m|})$.
- **IDEAL** $_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRR}}}$: This is the ideal execution where the ideal functionality $\mathcal{F}_{\text{MGHE}}$ is attached to the simulator $\mathcal{S}_{\text{MGHE}}$ and interacts with dummy client $\mathcal{P}_{\text{cli}_1}, \dots, \mathcal{P}_{\text{cli}_n}$.

Note that by Claim 1.1, we know that **REAL** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRR}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRR}}}$ are computational indistinguishable by the robustness of MGHE. Now we claim that:

Claim 2.1. *If MGHE is IND-CIRC secure, then **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRR}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRR}}}$ are computationally indistinguishable.*

Proof. We show that if there exists an environment $\mathcal{Z}$ that can distinguish between **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRR}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRR}}}$, we can construct an IND-CIRC adversary $\mathcal{A}_{\text{circ}}$ against MGHE using this environment. To do so, we let $\mathcal{A}_{\text{circ}}$ simulate the interaction between $\mathcal{Z}$ and these two worlds. Specifically, $\mathcal{A}_{\text{circ}}$ forms the same groups by specifying the index set $I_1, \dots, I_k$ in the game $\text{G}_{\text{MGHE}}^{\text{IND-CIRC}}$ and corrupts the same clients by defining the index set $I_{\mathcal{A}}$ as in the setup phase. Then, $\mathcal{A}_{\text{cpa}}$ handles its oracle queries based on the input provided by the environment $\mathcal{Z}$ to each party.

- **On input** $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}_j, m] \rangle$ **to an uncorrupted client** $\mathcal{P}_{\text{cli}}$: We let $\mathcal{A}_{\text{cpa}}$ query its oracle $\mathcal{O}_{\text{Enc}}$ with $(\text{jpg}_j, m; \omega)$ where ω is an honestly sampled random coins. Note that in both **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRR}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRR}}}$. An uncorrupted client runs and outputs $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m; \omega)$ for an honestly sampled random coins ω , which is exactly the behavior of the oracle $\mathcal{O}_{\text{Enc}}$. Thus we write c output from $\mathcal{O}_{\text{Enc}}$ as the output of an uncorrupted client $\mathcal{P}_{\text{cli}}$.
- **On input** $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}_j, m; \omega] \rangle$ **to a corrupted client** $\mathcal{P}_{\text{cli}}$: We let $\mathcal{A}_{\text{cpa}}$ query its oracle $\mathcal{O}_{\text{Enc}}$ with $(\text{jpg}_j, m; \omega)$, where ω is the random coins provided by $\mathcal{Z}$ to a corrupted client. In both hybrid games, since we consider a semi-malicious adversary, the ciphertext is generated as $c \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m; \omega)$ and is written as the output of a corrupted client. This behavior corresponds to that of the oracle $\mathcal{O}_{\text{Enc}}$. Therefore, we write the ciphertext c output by $\mathcal{O}_{\text{Enc}}$ as the output of a corrupted client $\mathcal{P}_{\text{cli}}$.

- **On input** $\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}] \rangle$ **to an uncorrupted server** $\mathcal{P}_{\text{srv}}$: Similarly as in the proof of Theorem 2, we let $\mathcal{A}_{\text{cpa}}$ first identify these ciphertexts in the lookup table T of $\text{G}_{\text{MGHE}}^{\text{IND-CIRC}}$ by specifying an index set L . Then, we let $\mathcal{A}_{\text{circ}}$ queries the oracle $\mathcal{O}_{\text{Eval}}$ with circuit-index set pair (f, L, f^*, L) where f^* is a circuit sampled uniformly at random from the circuit space $\mathcal{F}_\kappa$. If the oracle outputs $\perp$, we let $\mathcal{A}_{\text{cpa}}$ abort this query and nothing is written on any tape. This ensures that the output of a client $\mathcal{P}_{\text{cli}}$ during decryption remains the same in both worlds, thereby preventing trivial distinguishing. Otherwise, the evaluated ciphertext output from $\mathcal{O}_{\text{Eval}}$ is either

$$\hat{c} \leftarrow \text{MGHE.Eval}(f, (T[l].\text{jpg}, T[l].c)_{l \in L})$$

or

$$\hat{c}^* \leftarrow \text{MGHE.Eval}(f^*, (T[l].\text{jpg}, T[l].c)_{l \in L}).$$

Observe that in $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$, an uncorrupted server $\mathcal{P}_{\text{srv}}$ outputs $\hat{c}$, while in $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, $\hat{c}^*$ corresponds to the output of an uncorrupted server. Therefore, we write $\hat{c}$ or $\hat{c}^*$ returned by $\mathcal{O}_{\text{Eval}}$ as the output of an uncorrupted server $\mathcal{P}_{\text{srv}}$.

- **On input** $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c] \rangle$ **to an uncorrupted client** $\mathcal{P}_{\text{cli}}$: This is handled the same way as in the proof of Claim 1.2.
- **On input** $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c; \eta] \rangle$ **to an corrupted client** $\mathcal{P}_{\text{cli}}$: This is handled the same way as in the proof of Claim 1.2.
- **On input** $\langle \text{Combine} : [\text{grp}, \text{sid}, c, \{d_i\}_{\text{cli}_i \in \text{grp}}] \rangle$ **to a client** $\mathcal{P}_{\text{cli}}$: This is handled the same way as in the proof of Claim 1.2.

Finally, we let $\mathcal{A}_{\text{circ}}$ output the same bit output by $\mathcal{Z}$. Thus, the distinguishing advantage of $\mathcal{Z}$ translates directly to the advantage of $\mathcal{A}_{\text{circ}}$ in the IND-CIRC game:

$$\Delta_{\mathcal{Z}} \left(\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}, \text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}} \right) \leq \text{Adv}_{\text{MGHE}}^{\text{IND-CIRC}}(\mathcal{A}_{\text{circ}}). \quad (3)$$

□

Claim 2.2. *If MGHE is IND-CPA^{pD} secure, then $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$ and $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{3\mathcal{G}_{\text{KRK}}}$ are computationally indistinguishable.*

Proof. We consider the same IND-CPA^{pD} adversary $\mathcal{A}_{\text{cpa}}$ as defined in the proof of Theorem 3, but instead let $\mathcal{A}_{\text{cpa}}$ query its oracle $\mathcal{O}_{\text{Eval}}$ with a circuit f^* sampled from the circuit space $\mathcal{F}_\kappa$ at random. Note that In $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \text{MGHE.Eval}(f^*, \{\text{jpg}\}_{j \in [\ell]}, f(\{m_j\}_{j \in [\ell]}))$$

for some circuit f^* selected by $\mathcal{A}_{\text{cpa}}$, where each m_j is encrypted by either an uncorrupted or a corrupted client. In contrast, in $\text{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{3\mathcal{G}_{\text{KRK}}}$, the ciphertext $\hat{c}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \text{MGHE.Eval}(f^*, \{\text{jpg}\}_{j \in [\ell]}, f(\{m'_j\}_{j \in [\ell]}))$$

where each m'_j is either $0^{|m_j|}$ or some plaintext m_j if it is from an uncorrupted client or m_j if it is from a corrupted client or encrypted for a corrupted group. The remaining part follows the proof of Claim 1.2. $\square$

Also, by Claim 1.3, we have that $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{3\mathcal{G}_{\text{KRK}}}$ and $\mathbf{IDEAL}_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ statistically indistinguishable by SIM-PDEC of MGHE. Therefore, by combining Claim 2.1, 2.2, and Claim 1.1 and 1.3, we can conclude the proof for this theorem. $\square$

We can then consider a stronger security by requiring the statistical indistinguishability of the circuit for schemes with exact evaluation. In Corollary 1, we show the server-side confidentiality with the SIM-CIRC security.

F.3 Full proof of Corollary 1

Corollary 1. *Let MGHE be a multi-group homomorphic encryption scheme on exact number with threshold factor t . Then, the protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\text{MGHE}}$ in the presence of a global key registry $\mathcal{G}_{\text{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that non-adaptively corrupts parties, provided that:*

- MGHE satisfies robustness, and the adversary corrupts fewer than $\frac{|\text{grp}|}{t}$ clients in any group.
- MGHE is SIM-CIRC secure and the server is not corrupted.
- MGHE is IND-CPA^{pD}-secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group grp under which it provides input contains no more than $\frac{|\text{grp}|}{t}$ corrupted clients.
- MGHE satisfies SIM-PDEC security with respect to a simulator Sim_{th} .

Proof. We consider the same setting as in the proof of Theorem 2. From Theorem 3, we only replace the last two games. For completeness, we also all of the hybrid worlds here.

- $\mathbf{REAL}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: This represents the execution of the protocol Π_{MGHE} in the real world, involving the environment $\mathcal{Z}$ and the adversary $\mathcal{A}$.
- $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{1\mathcal{G}_{\text{KRK}}}$: In this hybrid game, we keep track of the message-ciphertext tuples (m, c) that are encrypted and evaluated, as well as the partial decryptions d that are honestly generated with respect to (m, c) . When a client combines the partial decryptions, if it has obtained a sufficient number of honest partial decryptions, it outputs m .
- $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{2\mathcal{G}_{\text{KRK}}}$: When $\mathcal{P}_{\text{srv}}$ is activated with $\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]}] \rangle$, we let $\mathcal{P}_{\text{srv}}$ output the ciphertext $\hat{c} \leftarrow \text{Sim}_{\text{circ}}(\{\text{jpg}_j\}_{j \in [\ell]}, \hat{m})$ produced by the circuit privacy simulator Sim_{circ} where $\hat{m} = f(m_1, \dots, m_j)$ and $c_j \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m_j)$ for $j \in [\ell]$.
- $\mathbf{HYB}_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{3\mathcal{G}_{\text{KRK}}}$: In this hybrid game, whenever an uncorrupted client $\mathcal{P}_{\text{cli}}$ is activated with the input $\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m] \rangle$, the client outputs $c \leftarrow \text{MGHE.Enc}(\text{jpg}, 0^{|m|})$.

- **IDEAL** $_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: This is the ideal execution where the ideal functionality $\mathcal{F}_{\text{MGHE}}$ is attached to the simulator $\mathcal{S}_{\text{MGHE}}$ and interacts with dummy client $\mathcal{P}_{\text{cli}_1}, \dots, \mathcal{P}_{\text{cli}_n}$.

Similarly, by Claim 1.1, we know that **REAL** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^1{}^{\mathcal{G}_{\text{KRK}}}$ are computational indistinguishable by the robustness of MGHE. Now we claim that:

Claim 2.3. *If MGHE is SIM-CIRC secure with respect to a circuit-privacy simulator Sim_{circ} , then **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^1{}^{\mathcal{G}_{\text{KRK}}}$ and **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^2{}^{\mathcal{G}_{\text{KRK}}}$ are statistically indistinguishable.*

Proof. By the definition of SIM-CIRC, there exists a circuit-privacy simulator Sim_{circ} such that an evaluated ciphertext output by $\hat{c} \leftarrow \text{MGHE.Eval}(f, (\text{jpg}_j, c_j)_{j \in [\ell]})$ is statistically indistinguishable from $c \leftarrow \text{Sim}_{\text{circ}}(\{\text{jpg}_j\}_{j \in [\ell]}, \hat{m})$ where $\hat{m} = f(m_1, \dots, m_\ell)$ and $c_j \leftarrow \text{MGHE.Enc}(\text{jpg}_j, m_j)$ for $j \in [\ell]$. Observe that these two outputs correspond to the outputs of an uncorrupted server when it is activated to evaluate a circuit f in these two worlds. Thus we have that the distributions of the two worlds are statistically indistinguishable. $\square$

Claim 2.4. *If MGHE is IND-CPA^{pD} secure, then **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^3{}^{\mathcal{G}_{\text{KRK}}}$ and **IDEAL** $_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ are computationally indistinguishable.*

Proof. We consider the same IND-CPA^{pD} adversary $\mathcal{A}_{\text{cpa}}$ as defined in the proof of Claim 3, but with a change of how $\mathcal{A}_{\text{cpa}}$ handles its queries to $\mathcal{O}_{\text{Eval}}$. Note that in **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^2{}^{\mathcal{G}_{\text{KRK}}}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \text{Sim}_{\text{circ}}(\{\text{jpg}\}_{j \in [\ell]}, f(\{m_j\}_{j \in [\ell]}))$$

for some circuit f , where each m_j is encrypted by either an uncorrupted or a corrupted client. In contrast, in **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^3{}^{\mathcal{G}_{\text{KRK}}}$, the ciphertext $\hat{c}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \text{Sim}_{\text{circ}}(\{\text{jpg}\}_{j \in [\ell]}, f(\{m'_j\}_{j \in [\ell]}))$$

where each m'_j is either $0^{|m_j|}$ or some plaintext m_j if it is from an uncorrupted client or m_j if it is from a corrupted client or encrypted for a corrupted group.

Furthermore, by Claim 2.3, we know that the ciphertext $\hat{c}$ obtained from $\text{MGHE.Eval}(\cdot)$ and $\text{Sim}_{\text{circ}}(\cdot)$ are statistically indistinguishable. Thus, the output of $\mathcal{P}_{\text{srv}}$ in these two worlds corresponds to the behavior of $\mathcal{O}_{\text{Eval}}$ when $b = 0$ or $b = 1$ in the IND-CPA^{pD} game, respectively. Upon receiving $\hat{c}$ from $\mathcal{O}_{\text{Eval}}$, we can then write it as the output of $\mathcal{P}_{\text{srv}}$. The remaining part follows the proof of Claim 1.2. $\square$

Also, by Claim 1.3, we have that **HYB** $_{\Pi_{\text{MGHE}}, \mathcal{A}, \mathcal{Z}}^3{}^{\mathcal{G}_{\text{KRK}}}$ and **IDEAL** $_{\mathcal{F}_{\text{MGHE}}, \mathcal{S}_{\text{MGHE}}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ are statistically indistinguishable by SIM-PDEC of MGHE. Therefore, by combining Claims 2.3 and 2.4, and Claims 1.1 and 1.3, we can then conclude the proof for this corollary. $\square$

F.4 Full proof of Theorem 4

Theorem 4. *The protocol $\Pi_{\text{OtF-MPC}}^{\text{SM}}$ UC-realizes $\mathcal{F}_{\text{OtF-MPC}}$ in $\mathcal{F}_{\text{MGHE}}$ -hybrid model with a global key registry $\mathcal{G}_{\text{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that corrupts a group up to the threshold.*

Proof. Let $\mathcal{A}$ be an adversary interacting with parties executing $\Pi_{\text{OtF-MPC}}^{\text{SM}}$ in $\mathcal{F}_{\text{MGHE}}$ -hybrid model. We construct a simulator $\mathcal{S}$ such that no environment $\mathcal{Z}$ can distinguish between

- **REAL** $_{\Pi_{\text{OtF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: the environment $\mathcal{Z}$ interacting with $\mathcal{A}$ and parties running $\Pi_{\text{OtF-MPC}}^{\text{SM}}$ in the $\mathcal{F}_{\text{MGHE}}$ -hybrid model
- **IDEAL** $_{\mathcal{F}_{\text{OtF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$: the environment $\mathcal{Z}$ interacting with $\mathcal{S}$ and the ideal functionality $\mathcal{F}_{\text{OtF-MPC}}$.

We let the simulator $\mathcal{S}$ runs an internal copy of $\mathcal{A}$ such that $\mathcal{S}$ forwards all communications from $\mathcal{Z}$ to $\mathcal{A}$ and back. $\mathcal{S}$ also runs internal copies of $\{\mathcal{P}_{\text{cli}_i}, \dots, \mathcal{P}_{\text{cli}_n}\}$ such that $\mathcal{S}$ simulates the interaction between $\mathcal{A}$ and $\{\mathcal{P}_{\text{cli}_i}, \dots, \mathcal{P}_{\text{cli}_n}, \mathcal{P}_{\text{srv}}\}$ as if $\mathcal{S}$ runs the functionality $\mathcal{F}_{\text{MGHE}}$. When $\mathcal{A}$ sends a (backdoor) message to corrupt a party, we let $\mathcal{S}$ send the same backdoor message to $\mathcal{F}_{\text{OtF-MPC}}$ to corrupt the same party.

The simulation of each phase is as follows:

- **Input Phase, Corrupted Client:** When a corrupted client is activated with the input

$$\langle \text{Upload} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \rangle,$$

we observe that the simulator $\mathcal{S}$ receives m from $\mathcal{F}_{\text{OtF-MPC}}$ in the world

IDEAL $_{\mathcal{F}_{\text{OtF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$.

Now in **REAL** $_{\Pi_{\text{OtF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, the adversary $\mathcal{A}$ sends (through a corrupted client) the input

$$\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m; \omega] \rangle$$

to $\mathcal{F}_{\text{MGHE}}$. We let the simulator forward this input back to $\mathcal{A}$ in the name of $\mathcal{F}_{\text{MGHE}}$. Upon receiving the ciphertext c from $\mathcal{A}$, we let $\mathcal{S}$ send c to (the corrupted) $\mathcal{P}_{\text{cli}}$ that it internally runs.

- **Input Phase, Uncorrupted Client:** When an uncorrupted client is activated with the input

$$\langle \text{Upload} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \rangle,$$

we observe that $\mathcal{S}$ receives either m or $|m|$, depending on the corruption status of grp , from $\mathcal{F}_{\text{OtF-MPC}}$ in the world **IDEAL** $_{\mathcal{F}_{\text{OtF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$.

To simulate the world **REAL** $_{\Pi_{\text{OtF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, we let $\mathcal{S}$ first retrieve jpg from $\mathcal{G}_{\text{KRK}}$. Then $\mathcal{S}$ sends to $\mathcal{A}$, in name of $\mathcal{F}_{\text{MGHE}}$, the input

$$\langle \text{Encrypt} : [\text{grp}, \text{sid}, \text{jpg}, m / |m|] \rangle.$$

Upon receiving a ciphertext c from $\mathcal{A}$, we let $\mathcal{S}$ send c (in the name of $\mathcal{F}_{\text{MGHE}}$) to the party $\mathcal{P}_{\text{cli}}$ that it internally runs.

- **Computation Phase, Corrupted Server:** When a corrupted server is activated with the input

$$\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle,$$

we observe that the simulator $\mathcal{S}$ knows the circuit f from the corrupted server $\mathcal{P}_{\text{srv}}$ and receives the evaluated plaintext $\hat{m} = f(\{m_j\}_{j \in [\ell]})$ ²⁰ from $\mathcal{F}_{\text{OtF-MPC}}$ in the world $\text{IDEAL}_{\mathcal{F}_{\text{OtF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$.

Now in $\text{REAL}_{\Pi_{\text{OtF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, the adversary $\mathcal{A}$ sends (through a corrupted client) the input

$$\langle \text{Evaluate} : [\text{sid}, f, (\text{grp}_j, \text{sid}_j, \text{jpg}_j, c_j)_{j \in [\ell]; \sigma}] \rangle$$

to $\mathcal{F}_{\text{MGHE}}$, we let $\mathcal{S}$ forward this back to $\mathcal{A}$ in the name of $\mathcal{F}_{\text{MGHE}}$. When the adversary then outputs an evaluated ciphertext $\hat{c}$, we let $\mathcal{S}$ send $\hat{c}$ to (the corrupted) server $\mathcal{P}_{\text{srv}}$ to match with the behavior of $\mathcal{F}_{\text{MGHE}}$.

- **Computation Phase, Uncorrupted Server:** When an uncorrupted server is activated with the input

$$\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle,$$

we observe that the simulator $\mathcal{S}$ only receives evaluated plaintext $\hat{m} = f(\{m_j\}_{j \in [\ell]})$ from $\mathcal{F}_{\text{OtF-MPC}}$ in the world $\text{IDEAL}_{\mathcal{F}_{\text{OtF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$.

To simulate the world $\text{REAL}_{\Pi_{\text{OtF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, we let the simulator retrieve $\{\text{jpg}_j\}_{j \in [\ell]}$ from $\mathcal{G}_{\text{KRK}}$. Then $\mathcal{S}$ send (in name of $\mathcal{F}_{\text{MGHE}}$) to $\mathcal{A}$ the input

$$\langle \text{Evaluate} : [\text{sid}, \perp, \hat{m}, (\text{grp}_j, \text{sid}_j, \text{jpg}_{j,u}, c_{j,u})_{j \in [\ell]}] \rangle.$$

When the adversary returns an evaluated ciphertext $\hat{c}$, we then let $\mathcal{S}$ send $\hat{c}$ to $\mathcal{P}_{\text{srv}}$ that it internally runs.

- **Output Phase, Corrupted Client:** When a corrupted client is activated with the input

$$\langle \text{Retrieve} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle,$$

we observe that $\mathcal{S}$ learns the message m if it is an evaluated message from the server, and $\mathcal{S}$ learns nothing if it is uploaded by a client since we assume that the adversary has not corrupted the group, from $\mathcal{F}_{\text{OtF-MPC}}$ in the world $\text{IDEAL}_{\mathcal{F}_{\text{OtF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$.

Now in $\text{REAL}_{\Pi_{\text{OtF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$ to $\mathcal{F}_{\text{MGHE}}$, when the adversary sends (through the corrupted client) the input

$$\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c; \eta] \rangle$$

to $\mathcal{F}_{\text{MGHE}}$, we first let the simulator check if it has seen the ciphertext c before. If yes, then $\mathcal{S}$ sends back this input to $\mathcal{A}$. Upon getting the partial decryption d from $\mathcal{A}$, we let $\mathcal{S}$ keep a record of it and send it to the (corrupted) client $\mathcal{P}_{\text{cli}}$.

²⁰ We note that the adversary does not inject $\hat{m}^*$ since we consider a semi-malicious adversary now.

Later, when the adversary sends (through the corrupted client) the input

$$\langle \text{Combine} : [\text{grp}, \text{sid}, c, \{d_i\}_{i \in \text{grp}}] \rangle,$$

we let simulator checks whether there are enough partial decryptions recorded before in $\{d_i\}_{i \in \text{grp}'}$. If so, we let $\mathcal{S}$ output m that it receives from $\mathcal{F}_{\text{OTF-MPC}}$ earlier to $\mathcal{P}_{\text{cli}}$ in the message

$$\{ \text{"pt"} : [\text{grp}, \text{sid}, c, m] \}.$$

- **Output Phase, Uncorrupted Client:** When an uncorrupted client $\mathcal{P}_{\text{cli}}$ is activated with the input

$$\langle \text{Retrieve} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle,$$

we observe that the simulator receives an evaluated plaintext m from $\mathcal{F}_{\text{OTF-MPC}}$ from $\mathcal{F}_{\text{OTF-MPC}}$ in the world $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$.

To simulate the world $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SM}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, we let $\mathcal{S}$ find the corresponding ciphertext c in its record. Then $\mathcal{S}$ sends to $\mathcal{A}$ in name of $\mathcal{F}_{\text{MGHE}}$ the input

$$\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, c, m] \rangle$$

Upon getting the partial decryption d from $\mathcal{A}$, we let $\mathcal{S}$ send it to the client $\mathcal{P}_{\text{cli}}$ that it internally runs.

Now we let $\mathcal{S}$ then check if it has obtained partial decryptions from $\mathcal{A}$. If yes, we then let $\mathcal{S}$ send to the client the plaintext m that it receives from $\mathcal{F}_{\text{OTF-MPC}}$ in a message

$$\{ \text{"pt"} : [\text{grp}, \text{sid}, c, m] \}.$$

We note that in each case, $\mathcal{S}$ simulates the interaction between $\mathcal{A}$, parties in the protocol $\Pi_{\text{OTF-MPC}}^{\text{SM}}$ in $\mathcal{F}_{\text{MGHE}}$ -hybrid model. Therefore, the environment's view is indistinguishable in the real and ideal executions. $\square$

F.5 Full proof of Theorem 5

Theorem 5. *The protocol $\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}$ UC-realizes $\mathcal{F}_{\text{OTF-MPC}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{zkSNARK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of a global key registry $\mathcal{G}_{\text{KRK}}$ and a global random oracle $\mathcal{G}_{\text{RO}}$ against a full malicious adversary that corrupts a group up to the threshold.*

Proof. We follow the proof of Theorem 4 for the confidential layer with $\mathcal{F}_{\text{MGHE}}$. Now we additionally prove the theorem by considering the interactions involving $\mathcal{F}_{\text{zkSNARK}}$ and $\mathcal{F}_{\text{Auth}}$.

- **Input Phase, Corrupted Client:** In $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, when $\mathcal{A}$ sends an a message to $\mathcal{F}_{\text{AUTH}}$ to register it (on behalf of the corrupted client). We let $\mathcal{S}$ parses the message as

$$\{ \text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c, \pi)] \}.$$

Then $\mathcal{S}$ stores this message internally for later use.

Next, $\mathcal{S}$ sends the input $\langle \text{Observe} : [\text{sid}] \rangle$ to $\mathcal{G}_{\text{RO}}$ to obtain the query trace tr_z made by the environment $\mathcal{Z}$. Then for each (x, v) in tr_z , we let $\mathcal{S}$ send the input $\langle \text{IsProg} : [x] \rangle$ to obtain a list of programmed points pg_z made by $\mathcal{Z}$. Using this information, $\mathcal{S}$ invokes the extractor to compute

$$w \leftarrow \text{Ext}((\text{jp}_k, c), \pi, \text{tr}_z \setminus \text{pg}_z).$$

We let $\mathcal{S}$ parse the witness w as (m, ω) with respect to R_{Enc} .

Then in $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, we let $\mathcal{S}$ send the following input to $\mathcal{F}_{\text{OTF-MPC}}$ on behalf of the corrupted client:

$$\{ \text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \}.$$

In all other cases, including when the message cannot be parsed or extraction cannot be made, $\mathcal{S}$ sets $m = \perp$ when sending the input to $\mathcal{F}_{\text{OTF-MPC}}$.

- **Input Phase, Uncorrupted Client:** In $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, upon receiving the ciphertext c from $\mathcal{A}$ (sending back to $\mathcal{F}_{\text{MGHE}}$ as in proof of Theorem 4), since $\mathcal{A}$ is not involved in $\mathcal{F}_{\text{zkSNARK}}$, we let $\mathcal{S}$ directly invoke $(\pi, \text{pg}) \leftarrow \text{Sim}_{\mathcal{Z}_k}^{\mathcal{G}_{\text{RO}}[\text{sid}]}(c)$ to generate an argument π for the relation R_{Enc} and program $\mathcal{G}_{\text{RO}}$ with pg respectively. We then let $\mathcal{S}$ generate a message

$$\{ \text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c_1, c_2, \pi)] \}$$

When a party sends an input to $\mathcal{F}_{\text{AUTH}}$ to retrieve this message, we let $\mathcal{S}$ deliver the message to the party (in the name of $\mathcal{F}_{\text{AUTH}}$).

- **Computation Phase, Corrupted Server:** In $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, when $\mathcal{A}$ sends a message to $\mathcal{F}_{\text{AUTH}}$ (through the corrupted server) to register the evaluation of a circuit, we let $\mathcal{S}$ parse it as

$$\{ \text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\hat{c}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \pi)] \}.$$

Similarly, $\mathcal{S}$ stores the message internally for later use.

Next, $\mathcal{S}$ queries $\mathcal{G}_{\text{RO}}$ to obtain the query trace tr_z made by the environment $\mathcal{Z}$, and the list of points pg_z programmed by $\mathcal{Z}$. Then we let $\mathcal{S}$ then runs the extractor

$$w \leftarrow \text{Ext}(\hat{c}, (\text{jp}_k, c_j)_{j \in [\ell]}, \pi, \text{tr}_z \setminus \text{pg}_z),$$

with respect to the relation R_{Eval}^f (where f is obtained by corrupting the server) and parses w as a random coin σ .

Then in $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, we let $\mathcal{S}$ send the following input to $\mathcal{F}_{\text{OTF-MPC}}$:

$$\{ \text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, \sqcup, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \},$$

where $\hat{m}^*$ sent to $\mathcal{F}_{\text{OTF-MPC}}$ is set as an empty placeholder $\sqcup$ to allow $\mathcal{F}_{\text{OTF-MPC}}$ to evaluate $\hat{m} = f(\{m_j\}_{j \in [\ell]})$ and record the result $\hat{m}$.

In all other cases including extraction failure or message cannot be parsed, $\mathcal{S}$ sends the input

$$\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, \perp, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}]\}$$

to $\mathcal{F}_{\text{OTF-MPC}}$ by setting $\perp$ as injected message to yield an invalid evaluated result $\hat{m} = \perp$.

- **Computation Phase, Uncorrupted Server:** In $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, when an uncorrupted server $\mathcal{P}_{\text{srv}}$ is activated with the input

$$\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle,$$

we observe that each message m_j for $j \in [\ell]$ must satisfy $m_j \neq \perp$ to proceed to the next step of evaluation by the definition of $\mathcal{F}_{\text{OTF-MPC}}$.

Now observe that in $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, when $\mathcal{P}_{\text{srv}}$ is activated with the input to evaluate a circuit, it first checks each input ciphertext that it receives from $\mathcal{F}_{\text{AUTH}}$. When the message at position $(\text{grp}_j, \text{sid}_j, \text{uid}_j)$ originates from a client, the uncorrupted server parses it as

$$\{\text{"up"} : [\text{cli}, \text{grp}_j, \text{sid}_j, (c_j, \pi)]\}$$

When $\mathcal{P}_{\text{cli}}$ (that is run internally by $\mathcal{S}$) verifies that π is a valid argument for the instance (pk_j, c_j) using $\mathcal{F}_{\text{zkSNARK}}^{R_{\text{Enc}}}$, we let $\mathcal{S}$ check if it has π in its record. If yes, π will always be accepted, meaning it is honestly generated. Otherwise, the behavior of $\mathcal{F}_{\text{zkSNARK}}^{R_{\text{Enc}}}$ is exactly the same as what we have the simulator $\mathcal{S}$ do in case of a corrupted client during the *input phase*. Recall that in case the verification is not successful, $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, the simulator $\mathcal{S}$ writes $m = \perp$ to $\mathcal{F}_{\text{OTF-MPC}}$, which causes $\mathcal{F}_{\text{OTF-MPC}}$ to abort during evaluation. Thus we have that the condition to abort an execution is the same in both of these two worlds.

On the other hand, if the message is from the server, in $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, the uncorrupted server parses it as

$$\{\text{"eval"} : [\text{srv}, \text{grp}_j, \text{sid}_j, \text{uid}_j, (c_j, (\text{grp}_{j'}, \text{sid}_{j'}, \text{uid}_{j'})_{j' \in [\ell']}, \pi)]\}.$$

Similarly, if π is generated honestly by the server, it means that the relation R_{Eval}^f is satisfied with respect to some circuit f . By the definition of $\mathcal{F}_{\text{MGHE}}$, then it has

$$m_j = f(\{m_{j'}\}_{j' \in [\ell']},$$

where m_j and $\{m_{j'}\}_{j' \in [\ell']}$ are the underlying messages of c_j and $\{c_{j'}\}_{j' \in [\ell']}$ respectively. We recall that when the verification fails, $\mathcal{S}$ is defined to write $\hat{m} = \perp$ to $\mathcal{F}_{\text{OTF-MPC}}$, which prevent $\mathcal{F}_{\text{OTF-MPC}}$ from further execution in $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$. Thus, the condition for proceeding to the next step of evaluation is equivalent in both the real and ideal worlds.

If all these checks pass, we then let $\mathcal{S}$ simulate $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$. We recall that the simulator $\mathcal{S}$ receives $\hat{m} = f(\{m_j\}_{j \in [\ell]})$ from $\mathcal{F}_{\text{OTF-MPC}}$ in the real

world, and the simulation of interaction between $\mathcal{A}$ and $\mathcal{F}_{\text{MGHE}}$ is the same as in proof of Theorem 4. Then upon receiving $\hat{c}$ from $\mathcal{A}$, we let $\mathcal{S}$ invoke $(\pi, \text{pg}) \leftarrow \text{Sim}_{zk}^{\mathcal{G}_{\text{RO}}[\text{sid}]}(\hat{c})$ to generate an argument π for the relation R_{Eval}^f and program $\mathcal{G}_{\text{RO}}$ with pg . We then let $\mathcal{S}$ store the message

$$\{ \text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\hat{c}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \pi)] \}.$$

When another party (that is internally run by $\mathcal{S}$) send input to $\mathcal{F}_{\text{AUTH}}$ to retrieve this message, we let $\mathcal{S}$ output it to the party in name of $\mathcal{F}_{\text{AUTH}}$.

- **Output Phase, Corrupted Client:** In $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, when $\mathcal{A}$ sends a message containing partial decryption to $\mathcal{F}_{\text{AUTH}}$, we let $\mathcal{S}$ parse the message as

$$\{ \text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c, d, \pi)] \}.$$

Then we similarly let $\mathcal{S}$ query $\mathcal{G}_{\text{RO}}$ to obtain the query trace tr_z made by the environment $\mathcal{Z}$, and the list of points pg_z programmed by $\mathcal{Z}$. Then $\mathcal{S}$ calls the extractor to obtain the witness

$$w \leftarrow \text{Ext}((c, d), \pi, \text{tr}_z \setminus \text{pg}_z),$$

and parse $w = (\text{sk}, \gamma, \eta)$. Note that if $w = \perp$, we then let $\mathcal{S}$ ignore this message. Otherwise, we then let $\mathcal{S}$ record this partial decryption and mark it as valid one in its record for later use.

- **Output Phase, Uncorrupted Client:** Note that we assume the group grp is not corrupted over the threshold. In $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, if the message originates from a client (i.e., it begins with the identifier $\{ \text{"up"} : [\cdot] \}$), $\mathcal{F}_{\text{OTF-MPC}}$ aborts the execution. The same is followed by an uncorrupted client $\mathcal{P}_{\text{cli}}$ executing $\Pi_{\text{OTF-MPC}}$ in $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SE-1P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$. On the other hand, if the message originates from the server (beginning with $\{ \text{"eval"} : [\cdot] \}$), the embedded message $\hat{m}$ is considered valid only if $\hat{m} = f(\{m_j\}_{j \in [\ell]})$, where $\{m_j\}_{j \in [\ell]}$ are the input messages referenced in the message.

In $\Pi_{\text{OTF-MPC}}$, this process is divided into two sub-phases. When $\mathcal{P}_{\text{cli}}$ is activated with the sub-input $\langle \text{Share} : [\text{grp}, \text{sid}, \text{uid}] \rangle$, it first retrieves the corresponding message from $\mathcal{F}_{\text{AUTH}}$ and parses it as

$$\{ \text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\hat{c}, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \pi)] \}.$$

If π are generated honestly by the server, they will be accepted. Otherwise, by the definition of $\mathcal{F}_{\text{zkSNARK}}$, we can extract a circuits f such that

$$\hat{c} = \text{MGHE.Eval}(f, (\text{jp}_{\text{pk}_j}, c_j)_{j \in [\ell]}).$$

If verification of π fails, the client aborts the execution. Also recall from the earlier definition of $\mathcal{S}$ for a corrupted server that, when $\hat{c}$ is an incorrect evaluation, we let $\mathcal{S}$ write $\hat{m} = \perp$ to $\mathcal{F}_{\text{OTF-MPC}}$, causing the protocol to abort during output phase. This results in consistent abortion conditions in both the ideal and real worlds.

The simulation of interaction between $\mathcal{A}$ and $\mathcal{F}_{\text{MGHE}}$ follows the proof of Theorem 4. Now, upon receiving the partial decryptions d from $\mathcal{A}$, then $\mathcal{S}$ invokes $(\pi, \text{pg}) \leftarrow \text{Sim}_{zk}^{\mathcal{G}_{\text{RO}}[\text{sid}]}$ to generate an argument π for the relation R_{Dec} and programs $\mathcal{G}_{\text{RO}}$ with pg . Then $\mathcal{S}$ constructs the message

$$\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c, d, \pi)]\}$$

and delivers it to a party in the name of $\mathcal{F}_{\text{AUTH}}$ when a party sends input to $\mathcal{F}_{\text{AUTH}}$ to retrieve it.

Combining of partial decryptions similarly follows proof of Theorem 4. Now, by the definition of $\mathcal{F}_{\text{zkSNARK}}$, if the arguments π are generated honestly by clients, they will always be accepted. Since we assume that the adversary does not corrupt more clients than allowed by the threshold, an uncorrupted client will possess enough partial decryption d , which are honestly generated using the correct secret keys. Consequently, by the definition of $\mathcal{F}_{\text{MGHE}}$, the pair (c, d) will reconstruct to an evaluated message $\hat{m}$, such that

$$\hat{m} = f(\{m_j\}_{j \in [\ell]}),$$

which is later output to $\mathcal{P}_{\text{cli}}$. Notably, in $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, the functionality $\mathcal{F}_{\text{OTF-MPC}}$ simply outputs the same evaluated plaintext $\hat{m}$ to an uncorrupted client. Therefore, the outputs in both the ideal and real worlds remain consistent. □

F.6 Full proof of Theorem 6

Theorem 6. *The protocol $\Pi_{\text{OTF-MPC}}^{\text{SS-2P}}$ UC-realizes $\mathcal{F}_{\text{OTF-MPC}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of a global key registry $\mathcal{G}_{\text{KRK}}$ and a global random oracle $\mathcal{G}_{\text{RO}}$ against a full malicious adversary that corrupts a group up to the threshold.*

Proof. We only prove for the cases when a client or the server is corrupted. For the uncorrupted clients and server, the proof generally follows the proof of Theorem 5 since an argument is always accepted if it is honestly generated by an uncorrupted party by the completeness of NIZK. To reduce to the soundness of NIZK, we now let the simulator $\mathcal{S}$ attached to $\mathcal{F}_{\text{OTF-MPC}}$ simulate the interaction between the adversary (through corrupted parties) and functionalities $\mathcal{F}_{\text{MGHE}}$ and $\mathcal{F}_{\text{NIZK}}$ as follows.

– **Input Phase, Corrupted Client:** In $\text{REAL}_{\Pi_{\text{OTF-MPC}}^{\text{SS-2P}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRK}}}$, when $\mathcal{A}$ sends an input to $\mathcal{F}_{\text{AUTH}}$ to register a message (on behalf of the corrupted client), we let $\mathcal{S}$ parse the message as

$$\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (c_1, c_2, \pi)]\}.$$

Then $\mathcal{S}$ stores this message internally for later use.

Next, $\mathcal{S}$ sends to $\mathcal{A}$ in name of $\mathcal{F}_{\text{NIZK}}$ to obtain m as witness. Then $\mathcal{S}$ checks in its record if $\mathcal{A}$ has sent input to $\mathcal{F}_{\text{MGHE}}$ (through the corrupted client) to encrypt a message m to c_1 and c_2 using the random coins ω_1 and ω_2 . If yes, in $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRRK}}}$, the simulator $\mathcal{S}$ then sends the input

$$\{\text{"up"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m]\}.$$

to $\mathcal{F}_{\text{OTF-MPC}}$ through the corrupted client. In all other cases, including failure to verify the existence of random coins used for encryption, or failure to parse the message, we let $\mathcal{S}$ set $m = \perp$ when sending the input to $\mathcal{F}_{\text{OTF-MPC}}$.

- **Computation Phase, Corrupted Server:** In $\text{REAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}_{\text{KRRK}}}$, when $\mathcal{A}$ sends an input to $\mathcal{F}_{\text{AUTH}}$ to register a message containing the evaluation of a circuit, we let $\mathcal{S}$ parse it as

$$\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, (\hat{c}_1, \hat{c}_2, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}, \pi)]\}.$$

Then $\mathcal{S}$ stores the message internally for later use.

Similarly, $\mathcal{S}$ checks in its record if $\mathcal{A}$ has sent inputs to $\mathcal{F}_{\text{MGHE}}$ with two random coins σ_1 and σ_2 , and a circuit f , such that $\hat{c}_1$ and $\hat{c}_2$ are the results of evaluating f on the respective input ciphertexts using σ_1 and σ_2 . If yes, in $\text{IDEAL}_{\mathcal{F}_{\text{OTF-MPC}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}_{\text{KRRK}}}$, the simulator $\mathcal{S}$ then sends the input

$$\{\text{"eval"} : [\text{srv}, \text{grp}, \text{sid}, \text{uid}, f, \perp, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}]\},$$

to $\mathcal{F}_{\text{OTF-MPC}}$ to evaluate $\hat{m} = f(\{m_j\}_{j \in [\ell]})$ and record the result $\hat{m}$.

In all other cases, $\mathcal{S}$ sends an input to $\mathcal{F}_{\text{OTF-MPC}}$ to use $\perp$ to directly inject an invalid evaluated result $\hat{m} = \perp$.

We note that after the input and computation phase, if a ciphertext tuple in the real world is invalid, then its respective plaintext in the ideal world is marked with $\perp$ in $\mathcal{F}_{\text{OTF-MPC}}$, which is the same as in the proof of Theorem 5. Thus the proof for the output phase generally follows the proof of Theorem 5 when we assume that a group is not corrupted over the threshold. $\square$

G Protocol with Single-Round Decryption

In Fig. 18, we further simplify the protocol by reducing to a single decryption. Specifically, upon receiving an evaluated ciphertext in the form of (c_1, c_2, π) . We let each client only (partially) decrypt c_1 instead. In this case, we then modify the relation R_{Dec} as

$$R_{\text{Dec}} = \left\{ ((\text{pp}_1, \text{pk}_1, c_1, d_1), (\text{sk}_1, \gamma_1, \eta_1)) \left| \begin{array}{l} d_u = \text{MGHE.PDec}(\text{sk}_u, c_u; \eta_u) \\ \wedge \text{pk}_u = \text{PKGen}(\text{pp}_u, \text{sk}_u; \gamma_u) \end{array} \right. \right\}.$$

We observe that with the relation $\mathcal{R}_{\text{Eval}}^f$, we ensure that the evaluated ciphertexts c_1 and c_2 correspond to the same underlying plaintext m , or are within

Protocol: $\Pi_{\text{OtF-MPC}}^{\text{SS-2i2c1o}}$

Participants: n clients $\mathcal{P}_{\text{cli}_1}, \mathcal{P}_{\text{cli}_2}, \dots, \mathcal{P}_{\text{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\text{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\text{grp}_j}$ with $\text{grp}_j = \{\text{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\text{srv}}$.

- $\langle \text{Upload} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, m] \rangle$ to $\mathcal{P}_{\text{cli}}$: The same as in $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$.
 - $\langle \text{Evaluate} : [\text{srv}, \text{sid}, \text{uid}, f, (\text{grp}_j, \text{sid}_j, \text{uid}_j)_{j \in [\ell]}] \rangle$ to $\mathcal{P}_{\text{srv}}$: The same as in $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$.
 - $\langle \text{Retrieve} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle$ to $\mathcal{P}_{\text{cli}}$: The same as in $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$.
 - $\langle \text{Share} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}] \rangle$:
 1. If $\text{cli} \notin \text{grp}$, abort.
 2. Retrieve message $\{\text{typ} : \cdot, \text{grp}, \text{sid}, \text{uid}, \cdot, (c_1, c_2, \pi)\}$ under the label $(\text{grp}, \text{sid}, \text{uid})$ from $\mathcal{F}_{\text{AUTH}}$.
 3. If $\text{typ} = \text{"up"}$, abort.
 4. Call $\mathcal{F}_{\text{zkSNARK}}^{\text{R}_{\text{eval}}^f}$ to verify the validity of π .
 5. Check all the input ciphertexts with respect to $(\text{grp}_j, \text{sid}_j, \text{uid}_j)$ for $j \in [\ell]$ are valid.
 6. Retrieve the secret keys sk_1 from $\mathcal{G}_{\text{KRK}}$.
 7. Call $\mathcal{F}_{\text{MGHE}}$ in sessions $(\text{sid}, 1)$ and to obtain partial decryption d_1 .
 8. Call $\mathcal{F}_{\text{NIZK}}^{\text{R}_{\text{Dec}}}$ to generate an argument π with (c_1, d_1) as instance and $(\text{sk}_1, \eta_1, \gamma_1)$ as witness where η_1 is the random coin used at call to $\mathcal{F}_{\text{MGHE}}$, and γ_1 is random coin used during key generation.
 9. Output $\{\text{"sh"} : [\text{cli}, \text{grp}, \text{sid}, \text{uid}, (d_1, \pi)]\}$ via $\mathcal{F}_{\text{AUTH}}$.
 - $\langle \text{Combine} : [\text{grp}, \text{sid}, \text{uid}] \rangle$:
 1. Obtain partial decryption d and the respective argument π from $\mathcal{F}_{\text{AUTH}}$.
 2. Call $\mathcal{F}_{\text{NIZK}}^{\text{R}_{\text{Dec}}}$ to verify the validity of π .
 3. Check sufficient number of partial decryptions has been obtained. Collect them into a set $\{d_i\}_{\text{cli}_i \in \text{grp}'}$.
 4. Call $\mathcal{F}_{\text{MGHE}}$ in sessions $(\text{sid}, 1)$ to combine these partial decryptions to messages m .
- Output $\{\text{"ret"} : [\text{grp}, \text{sid}, \text{uid}, m]\}$.

Fig. 18: Protocol $\Pi_{\text{OtF-MPC}}^{\text{SS-2P}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model that is secure against a (full) malicious adversary.

an acceptable error bound in the case of approximate HE schemes. Additionally, we introduce the relation $\mathcal{R}_{\text{Dec}}$ to certify the validity of each client's partial decryption. Assuming the adversary cannot corrupt more clients than the threshold allows, these relations together guarantee that the output of the decryption process is consistent with an honestly evaluated ciphertext, even when only one of the two evaluated ciphertexts is actually decrypted.

Theorem 6. *The protocol $\Pi_{\text{OtF-MPC}}^{\text{SS-2i2c1o}}$ UC-realizes $\mathcal{F}_{\text{OtF-MPC}}$ in $[\mathcal{F}_{\text{MGHE}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{AUTH}}]$ -hybrid model in presence of a global key registry $\mathcal{G}_{\text{KRK}}$ and a global random oracle $\mathcal{G}_{\text{RO}}$ against a full malicious adversary that corrupts a group up to the threshold and possibly the server.*

Proof (Sketch). The proof follows the proof of Theorem 6 for the input and evaluation phases. After the evaluation of a circuit, the tuple $(\hat{c}_1, \hat{c}_2, \pi)$ is delivered to a party $\mathcal{P}_{\text{cli}}$. As in Theorem 6, note that during the computation phase, the functionality $\mathcal{F}_{\text{MGHE}}$ records an evaluated message $\hat{m}$ that is consistently associated with both $\hat{c}_1$ and $\hat{c}_2$. Hence, decryption of either $\hat{c}_1$ or $\hat{c}_2$ yields the same $\hat{m}$. $\square$